

Studentų Projektas v3.0 – Vector<T>
3.0

Sugeneruota Doxygen 1.9.8

skyrius 1

Hierarchijos Indeksas

1.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

PushBackResult	??
ReallocResult	??
Skirstymorezultatas	??
TimerResults	??
Tyrimolrasas	??
Vector< T, Allocator >	??
Vector< Studentas >	??
Zmogus	??
Studentas	??

skyrius 2

Klasės Indeksas

2.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

PushBackResult	??
ReallocResult	??
Skirstymorezultatas	
Saugo skirstymo į grupes rezultatus	??
Studentas	
Konkreči klasė, paveldinti iš Zmogus	??
TimerResults	
Saugo kiekvienos programos fazės laiko matavimus (sekundėmis)	??
Tyrimolrasas	??
Vector< T, Allocator >	
Dinaminio masyvo konteineris – std::vector analogas	??
Zmogus	
Abstrakti bazinė klasė, apibrėžianti žmogaus sąsają	??

skyrius 3

Failo Indeksas

3.1 Failai

Visų failų sąrašas su trumpais aprašymais:

Generavimas.cpp	??
Generavimas.h	
Studentų failų generavimo ir skirstymo į grupes funkcijų deklaracijos	??
konteineris.h	??
laikai.cpp	??
laikai.h	
Laiko matavimų struktūra ir pagalbinės funkcijos	??
main.cpp	??
menu.cpp	??
menu.h	
Pagrindinio programos menu funkcijos deklaracija	??
skaitymas.cpp	??
skaitymas.h	
Studentų duomenų skaitymo funkcijų deklaracijos	??
spartos_analize.cpp	
Spartos analizė: std::vector vs Vector<T>	??
spausdinam.cpp	??
spausdinam.h	
Studentų duomenų rikiavimo ir išvedimo funkcijų deklaracijos	??
Studentas.cpp	
Studentas klasės metodų realizacija	??
Studentas.h	
Studento klasės apibrėžimas	??
tyrimas.cpp	??
tyrimas.h	
Konteinerių ir skirstymo strategijų greitimeikos tyrimo funkcijos	??
Vector.h	
Pilnavertė std::vector alternatyva – Vector<T, Allocator>	??
Zmogus.cpp	??
Zmogus.h	
Abstrakti bazinė klasė žmogui	??

skyrius 4

Klasės Dokumentacija

4.1 PushBackResult Struktūra

Vieši Atributai

- `std::size_t n`
- `double stdTime`
- `double myTime`

4.1.1 Smulkus aprašymas

Apibrėžimas failo [spartos_analyze.cpp](#) eilutėje 30.

4.1.2 Atributų Dokumentacija

4.1.2.1 myTime

```
double PushBackResult::myTime
```

Apibrėžimas failo [spartos_analyze.cpp](#) eilutėje 34.

4.1.2.2 n

```
std::size_t PushBackResult::n
```

Apibrėžimas failo [spartos_analyze.cpp](#) eilutėje 32.

4.1.2.3 stdTime

```
double PushBackResult::stdTime
```

Apibrėžimas failo [spartos_analyze.cpp](#) eilutėje 33.

Dokumentacija šiai struktūrai sugeneruota iš šio failo:

- [spartos_analyze.cpp](#)

4.2 ReallocResult Struktūra

Vieši Atributai

- `std::size_t n`
- `int stdReallocations`
- `int myReallocations`

4.2.1 Smulkus aprašymas

Apibrėžimas failo [spartos_analyze.cpp](#) eilutėje 92.

4.2.2 Atributų Dokumentacija

4.2.2.1 myReallocations

```
int ReallocResult::myReallocations
```

Apibrėžimas failo [spartos_analyze.cpp](#) eilutėje 96.

4.2.2.2 n

```
std::size_t ReallocResult::n
```

Apibrėžimas failo [spartos_analyze.cpp](#) eilutėje 94.

4.2.2.3 stdReallocations

```
int ReallocResult::stdReallocations
```

Apibrėžimas failo [spartos_analyze.cpp](#) eilutėje 95.

Dokumentacija šiai struktūrai sugeneruota iš šio failo:

- [spartos_analyze.cpp](#)

4.3 Skirstymorezultatas Struktūra

Saugo skirstymo į grupes rezultatus.

```
#include <Generavimas.h>
```

Vieši Atributai

- `StudContainer vargsiukai`
- `StudContainer protai`

4.3.1 Smulkus aprašymas

Saugo skirstymo į grupes rezultatus.

Apibrėžimas failo [Generavimas.h](#) eilutėje 41.

4.3.2 Atributų Dokumentacija

4.3.2.1 protai

```
StudContainer Skirstymorezultatas::protai
```

Apibrėžimas failo [Generavimas.h](#) eilutėje 44.

4.3.2.2 vargsiukai

```
StudContainer Skirstymorezultatas::vargsiukai
```

Apibrėžimas failo [Generavimas.h](#) eilutėje 43.

Dokumentacija šiai struktūrai sugeneruota iš šio failo:

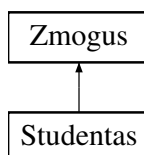
- [Generavimas.h](#)

4.4 Studentas Klasė

Konkreči klasė, paveldinti iš [Zmogus](#).

```
#include <Studentas.h>
```

Paveldimumo diagrama Studentas:



Vieši Metodai

Rule of Five

- `Studentas ()`
Numatytasis konstruktorius.
- `Studentas (std::istream &is)`
Srautinis konstruktorius.
- `Studentas (const Studentas &other)`
Kopijavimo konstruktorius.
- `Studentas & operator= (const Studentas &other)`
Kopijavimo priskyrimo operatorius.
- `Studentas (Studentas &&other)`
Perkėlimo konstruktorius.
- `Studentas & operator= (Studentas &&other)`
Perkėlimo priskyrimo operatorius.
- `~Studentas ()` `override=default`
Destruktorius.
- `const std::vector< int > & getND () const`
Grąžina namų darbų pažymių vektorių (tik skaitymui).
- `int getEgzaminas () const`
Grąžina egzamino pažymį.
- `float getVidurkis () const`
Grąžina namų darbų pažymių vidurkį.
- `float getgalrezVid () const`
Grąžina galutinį balą apskaičiuotą pagal vidurkį.
- `float getgalrezMed () const`
Grąžina galutinį balą apskaičiuotą pagal medianą.

Keitimo metodai (setters)

- `void setVardas (const std::string &v)`
Nustato studento vardą.
- `void setPavarde (const std::string &p)`
Nustato studento pavardę.
- `void setEgzaminas (int e)`
Nustato egzamino pažymį.

Funkcijos

- `std::istream & readStudent (std::istream &is)`
Nuskaito studento duomenis iš srauto.
- `void readNdInteractive ()`
Interaktyviai įveda namų darbų pažymius iš konsolės.
- `void parinktiAtsitiktinius ()`
Sugeneruoja atsitiktinius namų darbų ir egzamino pažymius.
- `void suskaiciuotiGalutini ()` `override`
Apskaiciuoja galutinį įvertinimą (vidurkio ir medianos variantus).
- `void spausdinti (std::ostream &os) const` `override`
Išveda studento duomenis į nurodytą srautą.

Vieši Metodai inherited from **Zmogus**

- **Zmogus** ()=default
Numatytasis konstruktorius.
- **Zmogus** (const std::string &v, const std::string &p)
Konstruktorius su parametrais.
- **Zmogus** (const **Zmogus** &)=default
Kopijavimo konstruktorius.
- **Zmogus** & **operator=** (const **Zmogus** &)=default
Kopijavimo priskyrimo operatorius.
- **Zmogus** (**Zmogus** &&)=default
Perkėlimo konstruktorius.
- **Zmogus** & **operator=** (**Zmogus** &&)=default
Perkėlimo priskyrimo operatorius.
- virtual ~**Zmogus** ()=default
Virtualus destruktorius.
- virtual std::string **getVardas** () const
- virtual std::string **getPavarde** () const
- void **setVardas** (const std::string &v)
- void **setPavarde** (const std::string &p)

Privatūs Atributai

- int **egzaminas**
- std::vector< int > **nd**
- float **vidurkis**
- float **galrezVid**
- float **galrezMed**

Susiję Funkcijos

Atkreipkite dėmesį, čia ne metodai

Laisvosios funkcijos

- std::ostream & **operator<<** (std::ostream &os, const **Studentas** &s)
Išvesties operatorius srautui.
- std::istream & **operator>>** (std::istream &is, **Studentas** &s)
Įvesties operatorius srautui.
- bool **comparePagalVarda** (const **Studentas** &a, const **Studentas** &b)
Lyginimo funkcija rikiavimui pagal vardą.
- bool **comparePagalPavarde** (const **Studentas** &a, const **Studentas** &b)
Lyginimo funkcija rikiavimui pagal pavardę.
- bool **comparePagalGalVid** (const **Studentas** &a, const **Studentas** &b)
Lyginimo funkcija rikiavimui pagal galutinį vidurkio balą.
- bool **comparePagalGalMed** (const **Studentas** &a, const **Studentas** &b)
Lyginimo funkcija rikiavimui pagal galutinį medianos balą.

Additional Inherited Members

Apsaugoti Atributai inherited from [Zmogus](#)

- `std::string` [vardas](#)
- `std::string` [pavarde](#)

4.4.1 Smulkus aprašymas

Konkreči klasė, paveldinti iš [Zmogus](#).

Saugo ir apdoroja studento namų darbų pažymius, egzamino rezultata, skaičiuoja galutinius įvertinimus (pagal vidurkį ir medianą).

4.4.1.0.1 Rule of Five

Klasė realizuoja visus penkis specialiuosius metodus:

- Numatytąjį konstruktorių
- Kopijavimo konstruktorių
- Kopijavimo priskyrimo operatorių
- Perkėlimo konstruktorių
- Perkėlimo priskyrimo operatorių
- Destruktorių (paveldi iš [Zmogus](#))

4.4.1.0.2 Galutinio balo formulė

$$galrezVid = 0.4 \cdot vidurkis + 0.6 \cdot egzaminas$$

$$galrezMed = 0.4 \cdot mediana + 0.6 \cdot egzaminas$$

Apibrėžimas failo [Studentas.h](#) eilutėje 46.

4.4.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.4.2.1 Studentas() [1/4]

```
Studentas::Studentas ( )
```

Numatytasis konstruktorius.

Inicializuoja visus skaičinius laukus į 0, o vektorių – į tuščią.

Inicializuoja skaičinius laukus į 0.

Apibrėžimas failo [Studentas.cpp](#) eilutėje 24.

4.4.2.2 Studentas() [2/4]

```
Studentas::Studentas (
    std::istream & is ) [explicit]
```

Srautinis konstruktorius.

Nuskaityto studento duomenis iš nurodyto įvesties srauto (pvz. failo eilutės pateiktos kaip `std::istringstream`).

Parametrai

<i>is</i>	Ivesties srautas.
-----------	-------------------

Nuskaityti eilutę iš srauto per `readStudent()`.

Parametrai

<i>is</i>	Ivesties srautas (pvz. <code>std::istringstream</code>).
-----------	---

Apibrėžimas failo `Studentas.cpp` eilutėje 34.

4.4.2.3 Studentas() [3/4]

```
Studentas::Studentas (
    const Studentas & other )
```

Kopijavimo konstruktorius.

Sukuria gilią kopiją kito `Studentas` objekto.

Parametrai

<i>other</i>	Kopijuojamas objektas.
--------------	------------------------

Sukuria gilią kopiją.

Parametrai

<i>other</i>	Kopijuojamas objektas.
--------------	------------------------

Apibrėžimas failo `Studentas.cpp` eilutėje 44.

4.4.2.4 Studentas() [4/4]

```
Studentas::Studentas (
    Studentas && other )
```

Perkėlimo konstruktorius.

Perkelia resursus iš `other` į šį objektą. Po perkėlimo `other` lieka tuščios būsenos.

Parametrai

<i>other</i>	Perkeliamas objektas (rvalue nuoroda).
--------------	--

Po perkėlimo šaltinio skaičiniai laukai nulinami, `nd` vektorius perkeliamas.

Parametrai

<i>other</i>	Perkeliamas objektas (rvalue).
--------------	--------------------------------

Apibrėžimas failo [Studentas.cpp](#) eilutėje 84.

4.4.2.5 ~Studentas()

```
Studentas::~~Studentas ( ) [override], [default]
```

Destruktorius.

Paveldi iš [Zmogus](#). Automatiškai atlaisvina `nd` vektoriaus atmintį.

4.4.3 Metodų Dokumentacija

4.4.3.1 getEgzaminas()

```
int Studentas::getEgzaminas ( ) const [inline]
```

Gražina egzamino pažymį.

Gražina

Egzamino pažymys (1–10).

Apibrėžimas failo [Studentas.h](#) eilutėje 135.

4.4.3.2 getgalrezMed()

```
float Studentas::getgalrezMed ( ) const [inline]
```

Gražina galutinį balą apskaičiuotą pagal medianą.

Gražina

Galutinis balas (medianos variantas).

Apibrėžimas failo [Studentas.h](#) eilutėje 153.

4.4.3.3 getgalrezVid()

```
float Studentas::getgalrezVid ( ) const [inline]
```

Gražina galutinį balą apskaičiuotą pagal vidurkį.

Gražina

Galutinis balas (vidurkio variantas).

Apibrėžimas failo [Studentas.h](#) eilutėje 147.

4.4.3.4 getND()

```
const std::vector< int > & Studentas::getND ( ) const [inline]
```

Gražina namų darbų pažymių vektorių (tik skaitymui).

Gražina

Konstantinė nuoroda į `nd` vektorių.

Apibrėžimas failo [Studentas.h](#) eilutėje 129.

4.4.3.5 getVidurkis()

```
float Studentas::getVidurkis ( ) const [inline]
```

Gražina namų darbų pažymių vidurkį.

Gražina

Vidurkis kaip `float`.

Apibrėžimas failo [Studentas.h](#) eilutėje 141.

4.4.3.6 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & other )
```

Kopijavimo priskyrimo operatorius.

Nukopijuoja visus duomenis iš `other` į šį objektą. Saugiai tvarko savipriskyrimo atvejį.

Parametrai

<i>other</i>	Priskiriamas objektas.
--------------	------------------------

Gražina

Nuoroda į šį objektą (`*this`).

Naudoja savipriskyrimo apsaugą (`if (this == &other)`).

Parametrai

<i>other</i>	Priskiriamas objektas.
--------------	------------------------

Gražina

Nuoroda į šį objektą.

Apibrėžimas failo [Studentas.cpp](#) eilutėje 60.

4.4.3.7 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && other )
```

Perkėlimo priskyrimo operatorius.

Perkelia resursus iš `other` į šį objektą priskyrimo metu. Saugiai tvarko savipriskyrimo atvejį.

Parametrai

<i>other</i>	Perkeliamas objektas (rvalue nuoroda).
--------------	--

Gražina

Nuoroda į šį objektą (`*this`).

Savipriskyrimo apsauga + resursų perkėlimas.

Parametrai

<i>other</i>	Perkeliamas objektas (rvalue).
--------------	--------------------------------

Gražina

Nuoroda į šį objektą.

Apibrėžimas failo [Studentas.cpp](#) eilutėje 104.

4.4.3.8 parinktiAtsitiktinius()

```
void Studentas::parinktiAtsitiktinius ( )
```

Sugeneruoja atsitiktinius namų darbų ir egzamino pažymius.

Naudoja `std::mt19937` generatorių su `std::random_device` sėkla. Pažymiai generuojami intervale [1, 10].

Naudoja `std::mt19937` su `std::random_device` sėkla. Pažymiai – intervale [1, 10].

Apibrėžimas failo [Studentas.cpp](#) eilutėje 312.

4.4.3.9 readNdInteractive()

```
void Studentas::readNdInteractive ( )
```

Interaktyviai įveda namų darbų pažymius iš konsolės.

Interaktyviai įveda namų darbų pažymius.

Prašo vartotojo įvesti pažymius vieną po kito. Įvedimas baigiamas naudojant 0 arba neigiamą skaičių.

Išimtys

<code>std::runtime_error</code>	Jei nepavyksta nuskaityti nė vieno pažymio.
<code>std::runtime_error</code>	Jei neįvestas nė vienas pažymys.

Apibrėžimas failo [Studentas.cpp](#) eilutėje 266.

4.4.3.10 readStudent()

```
std::istream & Studentas::readStudent (
    std::istream & is )
```

Nuskaityto studento duomenis iš srauto.

Nuskaityto studento duomenis iš srauto (vidinė funkcija).

Formatas: Vardas Pavardė ND1 ND2 ... NDn Egzaminas Paskutinis skaičius laikomas egzamino pažymiu.

Parametrai

<i>is</i>	Įvesties srautas.
-----------	-------------------

Gražina

Nuoroda į srautą (grandininiam kvietimui).

Formatas: Vardas Pavardė ND1 ND2 ... NDn Egzaminas Paskutinis skaičius laikomas egzamino pažymiu.

Apibrėžimas failo [Studentas.cpp](#) eilutėje 220.

4.4.3.11 setEgzaminas()

```
void Studentas::setEgzaminas (
    int e ) [inline]
```

Nustato egzamino pažymį.

Parametrai

<i>e</i>	Egzamino pažymys (rekomenduojama 1–10).
----------	---

Apibrėžimas failo [Studentas.h](#) eilutėje 177.

4.4.3.12 setPavarde()

```
void Studentas::setPavarde (
    const std::string & p ) [inline]
```

Nustato studento pavardę.

Parametrai

<i>p</i>	Nauja pavardė.
----------	----------------

Apibrėžimas failo [Studentas.h](#) eilutėje 171.

4.4.3.13 setVardas()

```
void Studentas::setVardas (
    const std::string & v ) [inline]
```

Nustato studento vardą.

Parametrai

<i>v</i>	Naujas vardas.
----------	----------------

Apibrėžimas failo [Studentas.h](#) eilutėje 165.

4.4.3.14 spausdinti()

```
void Studentas::spausdinti (
    std::ostream & os ) const [override], [virtual]
```

Išveda studento duomenis į nurodytą srautą.

Išveda studento duomenis į srautą.

Realizuoja grynąją virtualią funkciją iš [Zmogus](#).

Parametrai

<i>os</i>	Išvesties srautas.
-----------	--------------------

Realizuoja [Zmogus::spausdinti\(\)](#).

Parametrai

<i>os</i>	Išvesties srautas.
-----------	--------------------

Realizuoja [Zmogus](#).

Apibrėžimas failo [Studentas.cpp](#) eilutėje 133.

4.4.3.15 suskaiciuotiGalutini()

```
void Studentas::suskaiciuotiGalutini ( ) [override], [virtual]
```

Apskaičiuoja galutinį įvertinimą (vidurkio ir medianos variantus).

Apskaičiuoja galutinį įvertinimą.

Realizuoja grynąją virtualią funkciją iš [Zmogus](#).

Formulė:

- $galrezVid = 0.4 * vidurkis + 0.6 * egzaminas$
 - $galrezMed = 0.4 * mediana + 0.6 * egzaminas$
- Realizuoja [Zmogus::suskaiciuotiGalutini\(\)](#).

Formulė:

- $galrezVid = 0.4 * vidurkis + 0.6 * egzaminas$
- $galrezMed = 0.4 * mediana + 0.6 * egzaminas$

Mediana skaičiuojama iš surūšiuoto `nd` vektoriaus kopijos.

Realizuoja [Zmogus](#).

Apibrėžimas failo [Studentas.cpp](#) eilutėje 162.

4.4.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija

4.4.4.1 comparePagalGalMed()

```
bool comparePagalGalMed (
    const Studentas & a,
    const Studentas & b ) [related]
```

Lyginimo funkcija rikiavimui pagal galutinį medianos balą.

Parametrai

<i>a</i>	Pirmas studentas.
<i>b</i>	Antras studentas.

Gražina

```
true jei a.galrezMed < b.galrezMed.
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 352.

4.4.4.2 comparePagalGalVid()

```
bool comparePagalGalVid (
    const Studentas & a,
    const Studentas & b ) [related]
```

Lyginimo funkcija rikiavimui pagal galutinį vidurkio balą.

Parametrai

<i>a</i>	Pirmas studentas.
<i>b</i>	Antras studentas.

Gražina

```
true jei a.galrezVid < b.galrezVid.
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 346.

4.4.4.3 comparePagalPavarde()

```
bool comparePagalPavarde (  
    const Studentas & a,  
    const Studentas & b ) [related]
```

Lyginimo funkcija rikiavimui pagal pavardę.

Parametrai

<i>a</i>	Pirmas studentas.
<i>b</i>	Antras studentas.

Gražina

```
true jei a.pavarde < b.pavarde.
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 340.

4.4.4.4 comparePagalVarda()

```
bool comparePagalVarda (  
    const Studentas & a,  
    const Studentas & b ) [related]
```

Lyginimo funkcija rikiavimui pagal vardą.

Parametrai

<i>a</i>	Pirmas studentas.
<i>b</i>	Antras studentas.

Gražina

```
true jei a.vardas < b.vardas.
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 334.

4.4.4.5 operator<<()

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s ) [related]
```

Išvesties operatorius srautui.

Delegacija į `Studentas::spausdinti()`.

Parametrai

<i>os</i>	Išvesties srautas.
<i>s</i>	Studento objektas.

Gražina

Nuoroda į srautą.

Apibrėžimas failo `Studentas.cpp` eilutėje 196.

4.4.4.6 operator>>()

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s ) [related]
```

Išvesties operatorius srautui.

Nuskaityto studento duomenis ir iškart apskaičiuoja galutinį įvertinimą.

Parametrai

<i>is</i>	Išvesties srautas.
<i>s</i>	Studento objektas, į kurį rašoma.

Gražina

Nuoroda į srautą.

Apibrėžimas failo `Studentas.cpp` eilutėje 207.

4.4.5 Atributų Dokumentacija

4.4.5.1 egzaminas

```
int Studentas::egzaminas [private]
```

Apibrėžimas failo `Studentas.h` eilutėje 49.

4.4.5.2 galrezMed

```
float Studentas::galrezMed [private]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 53.

4.4.5.3 galrezVid

```
float Studentas::galrezVid [private]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 52.

4.4.5.4 nd

```
std::vector<int> Studentas::nd [private]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 50.

4.4.5.5 vidurkis

```
float Studentas::vidurkis [private]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 51.

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [Studentas.h](#)
- [Studentas.cpp](#)

4.5 TimerResults Struktūra

Saugo kiekvienos programos fazės laiko matavimus (sekundėmis).

```
#include <laikai.h>
```

Vieši Atributai

- double [generavimas](#) = 0.0
- double [skaitymas](#) = 0.0
- double [rusiavimas](#) = 0.0
- double [skirstymas](#) = 0.0
- double [isvedimas](#) = 0.0

4.5.1 Smulkus aprašymas

Saugo kiekvienos programos fazės laiko matavimus (sekundėmis).

Apibrėžimas failo [laikai.h](#) eilutėje 20.

4.5.2 Atributų Dokumentacija

4.5.2.1 generavimas

```
double TimerResults::generavimas = 0.0
```

Apibrėžimas failo [laikai.h](#) eilutėje 22.

4.5.2.2 isvedimas

```
double TimerResults::isvedimas = 0.0
```

Apibrėžimas failo [laikai.h](#) eilutėje 26.

4.5.2.3 rusiavimas

```
double TimerResults::rusiavimas = 0.0
```

Apibrėžimas failo [laikai.h](#) eilutėje 24.

4.5.2.4 skaitymas

```
double TimerResults::skaitymas = 0.0
```

Apibrėžimas failo [laikai.h](#) eilutėje 23.

4.5.2.5 skirstymas

```
double TimerResults::skirstymas = 0.0
```

Apibrėžimas failo [laikai.h](#) eilutėje 25.

Dokumentacija šiai struktūrai sugeneruota iš šio failo:

- [laikai.h](#)

4.6 Tyrimolrasas Struktūra

Vieši Atributai

- `std::string failas`
- `std::size_t irasuKiekis = 0`
- `double skaitymas = 0.0`
- `double rusiavimas = 0.0`
- `double skirstymas = 0.0`

4.6.1 Smulkus aprašymas

Apibrėžimas failo [tyrimas.cpp](#) eilutėje 16.

4.6.2 Atributų Dokumentacija

4.6.2.1 failas

```
std::string TyrimoIrasas::failas
```

Apibrėžimas failo [tyrimas.cpp](#) eilutėje 18.

4.6.2.2 irasuKiekis

```
std::size_t TyrimoIrasas::irasuKiekis = 0
```

Apibrėžimas failo [tyrimas.cpp](#) eilutėje 19.

4.6.2.3 rusiavimas

```
double TyrimoIrasas::rusiavimas = 0.0
```

Apibrėžimas failo [tyrimas.cpp](#) eilutėje 21.

4.6.2.4 skaitymas

```
double TyrimoIrasas::skaitymas = 0.0
```

Apibrėžimas failo [tyrimas.cpp](#) eilutėje 20.

4.6.2.5 skirstymas

```
double TyrimoIrasas::skirstymas = 0.0
```

Apibrėžimas failo [tyrimas.cpp](#) eilutėje 22.

Dokumentacija šiai struktūrai sugeneruota iš šio failo:

- [tyrimas.cpp](#)

4.7 Vector< T, Allocator > Klasė Šablonas

Dinaminio masyvo konteineris – std::vector analogas.

```
#include <Vector.h>
```

Vieši Tipai

- using value_type = T
- using allocator_type = Allocator
- using size_type = std::size_t
- using difference_type = std::ptrdiff_t
- using reference = value_type &
- using const_reference = const value_type &
- using pointer = typename std::allocator_traits< Allocator >::pointer
- using const_pointer = typename std::allocator_traits< Allocator >::const_pointer
- using iterator = pointer
- using const_iterator = const_pointer
- using reverse_iterator = std::reverse_iterator< iterator >
- using const_reverse_iterator = std::reverse_iterator< const_iterator >

Vieši Metodai

- Vector () noexcept(noexcept(Allocator()))
Numatytasis konstruktorius – sukuria tuščią vektorių.
- Vector (const Allocator &alloc) noexcept
Konstruktorius su paskirstytoju.
- Vector (size_type count, const T &value, const Allocator &alloc=Allocator())
Konstruktorius su skaičiumi ir reikšme.
- Vector (size_type count, const Allocator &alloc=Allocator())
Konstruktorius su skaičiumi (value-initialized).
- template<typename InputIt, typename = std::enable_if_t<std::is_base_of_v< std::input_iterator_tag, typename std::iterator_traits<↔ InputIt>::iterator_category>>>
Vector (InputIt first, InputIt last, const Allocator &alloc=Allocator())
Diapazoninis konstruktorius (InputIterator pora).
- Vector (const Vector &other)
Kopijavimo konstruktorius.
- Vector (const Vector &other, const Allocator &alloc)
Kopijavimo konstruktorius su paskirstytoju.
- Vector (Vector &&other) noexcept
Perkėlimo konstruktorius.
- Vector (Vector &&other, const Allocator &alloc) noexcept
Perkėlimo konstruktorius su paskirstytoju.
- Vector (std::initializer_list< T > init, const Allocator &alloc=Allocator())
Inicializatorių sąrašo konstruktorius.
- ~Vector ()
Destruktorius – atlaisvina visus resursus.
- Vector & operator= (const Vector &other)
Kopijavimo priskyrimo operatorius.
- Vector & operator= (Vector &&other) noexcept

- Perkėlimo priskyrimo operatorius.*

 - `Vector & operator= (std::initializer_list< T > init)`

InicIALIZatorių sąrašo priskyrimo operatorius.

 - `void assign (size_type count, const T &value)`

Priskiria count kopijų value reikšmės.

 - `template<typename InputIt, typename = std::enable_if_t<std::is_base_of_v< std::input_iterator_tag, typename std::iterator_traits<↔
InputIt>::iterator_category>>>`

`void assign (InputIt first, InputIt last)`

Priskiria diapazoną [first, last).

 - `void assign (std::initializer_list< T > init)`

Priskiria inicializatorių sąrašą.

 - `allocator_type get_allocator () const noexcept`

Grąžina naudojamą paskirstytoją.

 - `reference at (size_type pos)`

Prieiga su ribų tikrinimo (meta std::out_of_range jei pos >= size).

 - `const_reference at (size_type pos) const`

Const prieiga su ribų tikrinimo.

 - `reference operator[] (size_type pos)`

Indeksavimo operatorius (be ribų tikrinimo).

 - `const_reference operator[] (size_type pos) const`

Const indeksavimo operatorius.

 - `reference front ()`

Pirmas elementas.

 - `const_reference front () const`
 - `reference back ()`

Paskutinis elementas.

 - `const_reference back () const`
 - `T * data () noexcept`

Rodyklė į vidinį masyvą.

 - `const T * data () const noexcept`
 - `iterator begin () noexcept`
 - `const_iterator begin () const noexcept`
 - `const_iterator cbegin () const noexcept`
 - `iterator end () noexcept`
 - `const_iterator end () const noexcept`
 - `const_iterator cend () const noexcept`
 - `reverse_iterator rbegin () noexcept`
 - `const_reverse_iterator rbegin () const noexcept`
 - `const_reverse_iterator crbegin () const noexcept`
 - `reverse_iterator rend () noexcept`
 - `const_reverse_iterator rend () const noexcept`
 - `const_reverse_iterator crend () const noexcept`
 - `bool empty () const noexcept`

Ar vektorius tuščias?

 - `size_type size () const noexcept`

Elementų skaičius.

 - `size_type max_size () const noexcept`

Maksimalus galimas dydis.

 - `size_type capacity () const noexcept`

Dabartinė talpa.

 - `void reserve (size_type newCap)`

Rezervuoja atmintį bent newCap elementams (nekeičia size).

- `void shrink_to_fit ()`
Sumažina talpos perteklių ($cap \rightarrow size$).
- `void clear () noexcept`
Pašalina visus elementus (talpa nesikeičia).
- `iterator insert (const_iterator pos, const T &value)`
Įterpia vieną kopiją prieš pos.
- `iterator insert (const_iterator pos, T &&value)`
Įterpia vieną elementą (move) prieš pos.
- `iterator insert (const_iterator pos, size_type count, const T &value)`
Įterpia count kopijų value prieš pos.
- `template<typename InputIt, typename = std::enable_if_t<std::is_base_of_v< std::input_iterator_tag, typename std::iterator_traits<InputIt>::iterator_category>>>`
`iterator insert (const_iterator pos, InputIt first, InputIt last)`
Įterpia diapazoną [first, last) prieš pos.
- `iterator insert (const_iterator pos, std::initializer_list< T > init)`
Įterpia inicializatorių sąrašą prieš pos.
- `template<typename... Args>`
`iterator emplace (const_iterator pos, Args &&... args)`
Sukuria elementą vietoje prieš pos.
- `iterator erase (const_iterator pos)`
Pašalina elementą pos pozicijoje.
- `iterator erase (const_iterator first, const_iterator last)`
Pašalina diapazoną [first, last).
- `void push_back (const T &value)`
Prideda kopiją į galą.
- `void push_back (T &&value)`
Prideda (move) elementą į galą.
- `template<typename... Args>`
`reference emplace_back (Args &&... args)`
Sukuria elementą vietoje gale.
- `void pop_back ()`
Pašalina paskutinį elementą.
- `void resize (size_type count)`
Pakeičia dydį į count (nauji elementai – value-initialized).
- `void resize (size_type count, const T &value)`
Pakeičia dydį į count, naujus elementus užpildo value.
- `void swap (Vector &other) noexcept`
Sukeičia du vektorius vietomis ($O(1)$).

Privatūs Tipai

- `using AllocTraits = std::allocator_traits< Allocator >`

Privatūs Metodai

- `void destroyAll () noexcept`
- `void deallocate () noexcept`
- `void reallocate (size_type newCap)`
- `size_type growCap () const noexcept`
- `void ensureSpace (size_type count)`
- `void shiftRight (size_type idx, size_type count)`
Pastumia elementus iš [idx, size_) dešinėn per count pozicijų.

Privatūs Atributai

- [Allocator alloc_](#)
- [pointer data_](#)
- [size_type size_](#)
- [size_type cap_](#)

Susiję Funkcijos

Atkreipkite dėmesį, čia ne metodai

- `template<typename T, typename A >`
`bool operator== (const Vector< T, A > &l, const Vector< T, A > &r)`
- `template<typename T, typename A >`
`bool operator!= (const Vector< T, A > &l, const Vector< T, A > &r)`
- `template<typename T, typename A >`
`bool operator< (const Vector< T, A > &l, const Vector< T, A > &r)`
- `template<typename T, typename A >`
`bool operator<= (const Vector< T, A > &l, const Vector< T, A > &r)`
- `template<typename T, typename A >`
`bool operator> (const Vector< T, A > &l, const Vector< T, A > &r)`
- `template<typename T, typename A >`
`bool operator>= (const Vector< T, A > &l, const Vector< T, A > &r)`
- `template<typename T, typename A >`
`void swap (Vector< T, A > &l, Vector< T, A > &r) noexcept`
ADL-friendly swap.
- `template<typename T, typename A, typename U >`
`Vector< T, A >::size_type erase (Vector< T, A > &c, const U &value)`
Pašalina visus elementus, lygius value.
- `template<typename T, typename A, typename Pred >`
`Vector< T, A >::size_type erase_if (Vector< T, A > &c, Pred pred)`
Pašalina visus elementus, tenkinančius pred.

4.7.1 Smulkus aprašymas

```
template<typename T, typename Allocator = std::allocator<T>>
class Vector< T, Allocator >
```

Dinaminio masyvo konteineris – `std::vector` analogas.

Template Parameters

<i>T</i>	Saugomų elementų tipas.
<i>Allocator</i>	Atminties paskirstytojas (numatytasis: <code>std::allocator<T></code>).

Apibrėžimas failo [Vector.h](#) eilutėje 29.

4.7.2 Tipo Aprašymo Dokumentacija

4.7.2.1 allocator_type

```
template<typename T , typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::allocator_type = Allocator
```

Apibrėžimas failo [Vector.h](#) eilutėje 34.

4.7.2.2 AllocTraits

```
template<typename T , typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::AllocTraits = std::allocator_traits<Allocator> [private]
```

Apibrėžimas failo [Vector.h](#) eilutėje 47.

4.7.2.3 const_iterator

```
template<typename T , typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_iterator = const_pointer
```

Apibrėžimas failo [Vector.h](#) eilutėje 42.

4.7.2.4 const_pointer

```
template<typename T , typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_pointer = typename std::allocator_traits<Allocator>↔
::const_pointer
```

Apibrėžimas failo [Vector.h](#) eilutėje 40.

4.7.2.5 const_reference

```
template<typename T , typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_reference = const value_type&
```

Apibrėžimas failo [Vector.h](#) eilutėje 38.

4.7.2.6 const_reverse_iterator

```
template<typename T , typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

Apibrėžimas failo [Vector.h](#) eilutėje 44.

4.7.2.7 difference_type

```
template<typename T , typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::difference_type = std::ptrdiff_t
```

Apibrėžimas failo [Vector.h](#) eilutėje 36.

4.7.2.8 iterator

```
template<typename T , typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::iterator = pointer
```

Apibrėžimas failo [Vector.h](#) eilutėje 41.

4.7.2.9 pointer

```
template<typename T , typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::pointer = typename std::allocator_traits<Allocator>::pointer
```

Apibrėžimas failo [Vector.h](#) eilutėje 39.

4.7.2.10 reference

```
template<typename T , typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::reference = value_type&
```

Apibrėžimas failo [Vector.h](#) eilutėje 37.

4.7.2.11 reverse_iterator

```
template<typename T , typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::reverse_iterator = std::reverse_iterator<iterator>
```

Apibrėžimas failo [Vector.h](#) eilutėje 43.

4.7.2.12 size_type

```
template<typename T , typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::size_type = std::size_t
```

Apibrėžimas failo [Vector.h](#) eilutėje 35.

4.7.2.13 value_type

```
template<typename T , typename Allocator = std::allocator<T>>  
using Vector< T, Allocator >::value_type = T
```

Apibrėžimas failo [Vector.h](#) eilutėje 33.

4.7.3 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.7.3.1 Vector() [1/10]

```
template<typename T , typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector ( ) [inline], [noexcept]
```

Numatytasis konstruktorius – sukuria tuščią vektorių.

Apibrėžimas failo [Vector.h](#) eilutėje 111.

4.7.3.2 Vector() [2/10]

```
template<typename T , typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Allocator & alloc ) [inline], [explicit], [noexcept]
```

Konstruktorius su paskirstytoju.

Apibrėžimas failo [Vector.h](#) eilutėje 116.

4.7.3.3 Vector() [3/10]

```
template<typename T , typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    size_type count,
    const T & value,
    const Allocator & alloc = Allocator() ) [inline]
```

Konstruktorius su skaičiumi ir reikšme.

Parametrai

<i>count</i>	Elementų skaičius.
<i>value</i>	Pradinė reikšmė.

Apibrėžimas failo [Vector.h](#) eilutėje 123.

4.7.3.4 Vector() [4/10]

```
template<typename T , typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    size_type count,
    const Allocator & alloc = Allocator() ) [inline], [explicit]
```

Konstruktorius su skaičiumi (value-initialized).

Parametrai

<i>count</i>	Elementų skaičius.
--------------	--------------------

Apibrėžimas failo [Vector.h](#) eilutėje 141.

4.7.3.5 Vector() [5/10]

```
template<typename T , typename Allocator = std::allocator<T>>
template<typename InputIt , typename = std::enable_if_t<std::is_base_of_v< std::input_iterator_tag, typename std::iterator_traits<InputIt>::iterator_category>>>
Vector< T, Allocator >::Vector (
    InputIt first,
    InputIt last,
    const Allocator & alloc = Allocator() ) [inline]
```

Diapazoninis konstruktorius (InputIterator pora).

Apibrėžimas failo [Vector.h](#) eilutėje 147.

4.7.3.6 Vector() [6/10]

```
template<typename T , typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Vector< T, Allocator > & other ) [inline]
```

Kopijavimo konstruktorius.

Apibrėžimas failo [Vector.h](#) eilutėje 158.

4.7.3.7 Vector() [7/10]

```
template<typename T , typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Vector< T, Allocator > & other,
    const Allocator & alloc ) [inline]
```

Kopijavimo konstruktorius su paskirstytoju.

Apibrėžimas failo [Vector.h](#) eilutėje 175.

4.7.3.8 Vector() [8/10]

```
template<typename T , typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    Vector< T, Allocator > && other ) [inline], [noexcept]
```

Perkėlimo konstruktorius.

Apibrėžimas failo [Vector.h](#) eilutėje 192.

4.7.3.9 Vector() [9/10]

```
template<typename T , typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    Vector< T, Allocator > && other,
    const Allocator & alloc ) [inline], [noexcept]
```

Perkėlimo konstruktorius su paskirstytoju.

Apibrėžimas failo [Vector.h](#) eilutėje 202.

4.7.3.10 Vector() [10/10]

```
template<typename T , typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    std::initializer_list< T > init,
    const Allocator & alloc = Allocator() ) [inline]
```

Inicializatorių sąrašo konstruktorius.

Apibrėžimas failo [Vector.h](#) eilutėje 212.

4.7.3.11 ~Vector()

```
template<typename T , typename Allocator = std::allocator<T>>
Vector< T, Allocator >::~~Vector ( ) [inline]
```

Destruktorius – atlaisvina visus resursus.

Apibrėžimas failo [Vector.h](#) eilutėje 226.

4.7.4 Metodų Dokumentacija**4.7.4.1 assign()** [1/3]

```
template<typename T , typename Allocator = std::allocator<T>>
template<typename InputIt , typename = std::enable_if_t<std::is_base_of_v< std::input_iterator_tag, typename std::iterator_traits<InputIt>::iterator_category>>>
void Vector< T, Allocator >::assign (
    InputIt first,
    InputIt last ) [inline]
```

Priskiria diapazoną [first, last).

Apibrėžimas failo [Vector.h](#) eilutėje 317.

4.7.4.2 assign() [2/3]

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::assign (
    size_type count,
    const T & value ) [inline]
```

Priskiria count kopijų value reikšmės.

Apibrėžimas failo [Vector.h](#) eilutėje 303.

4.7.4.3 assign() [3/3]

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::assign (
    std::initializer_list< T > init ) [inline]
```

Priskiria inicializatorių sąrašą.

Apibrėžimas failo [Vector.h](#) eilutėje 329.

4.7.4.4 at() [1/2]

```
template<typename T , typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::at (
    size_type pos ) [inline]
```

Prieiga su ribų tikrinimo (meta `std::out_of_range` jei `pos >= size`).

Apibrėžimas failo [Vector.h](#) eilutėje 356.

4.7.4.5 at() [2/2]

```
template<typename T , typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::at (
    size_type pos ) const [inline]
```

Const prieiga su ribų tikrinimo.

Apibrėžimas failo [Vector.h](#) eilutėje 368.

4.7.4.6 back() [1/2]

```
template<typename T , typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::back ( ) [inline]
```

Paskutinis elementas.

Apibrėžimas failo [Vector.h](#) eilutėje 387.

4.7.4.7 back() [2/2]

```
template<typename T , typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::back ( ) const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 388.

4.7.4.8 begin() [1/2]

```
template<typename T , typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::begin ( ) const [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 397.

4.7.4.9 begin() [2/2]

```
template<typename T , typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::begin ( ) [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 396.

4.7.4.10 capacity()

```
template<typename T , typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::capacity ( ) const [inline], [noexcept]
```

Dabartinė talpa.

Apibrėžimas failo [Vector.h](#) eilutėje 421.

4.7.4.11 cbegin()

```
template<typename T , typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::cbegin ( ) const [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 398.

4.7.4.12 cend()

```
template<typename T , typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::cend ( ) const [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 402.

4.7.4.13 clear()

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::clear ( ) [inline], [noexcept]
```

Pašalina visus elementus (talpa nesikeičia).

Apibrėžimas failo [Vector.h](#) eilutėje 458.

4.7.4.14 crbegin()

```
template<typename T , typename Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::crbegin ( ) const [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 406.

4.7.4.15 crend()

```
template<typename T , typename Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::crend ( ) const [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 410.

4.7.4.16 data() [1/2]

```
template<typename T , typename Allocator = std::allocator<T>>
const T * Vector< T, Allocator >::data ( ) const [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 392.

4.7.4.17 data() [2/2]

```
template<typename T , typename Allocator = std::allocator<T>>
T * Vector< T, Allocator >::data ( ) [inline], [noexcept]
```

Rodyklė į vidinį masyvą.

Apibrėžimas failo [Vector.h](#) eilutėje 391.

4.7.4.18 deallocate()

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::deallocate ( ) [inline], [private], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 66.

4.7.4.19 destroyAll()

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::destroyAll ( ) [inline], [private], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 57.

4.7.4.20 emplace()

```
template<typename T , typename Allocator = std::allocator<T>>
template<typename... Args>
iterator Vector< T, Allocator >::emplace (
    const_iterator pos,
    Args &&... args ) [inline]
```

Sukuria elementą vietoje prieš pos.

Apibrėžimas failo [Vector.h](#) eilutėje 568.

4.7.4.21 emplace_back()

```
template<typename T , typename Allocator = std::allocator<T>>
template<typename... Args>
reference Vector< T, Allocator >::emplace_back (
    Args &&... args ) [inline]
```

Sukuria elementą vietoje gale.

Gražina

Nuoroda į sukurtą elementą.

Apibrėžimas failo [Vector.h](#) eilutėje 656.

4.7.4.22 empty()

```
template<typename T , typename Allocator = std::allocator<T>>
bool Vector< T, Allocator >::empty ( ) const [inline], [noexcept]
```

Ar vektorius tuščias?

Apibrėžimas failo [Vector.h](#) eilutėje 415.

4.7.4.23 end() [1/2]

```
template<typename T , typename Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::end ( ) const [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 401.

4.7.4.24 end() [2/2]

```
template<typename T , typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::end ( ) [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 400.

4.7.4.25 ensureSpace()

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::ensureSpace (
    size_type count ) [inline], [private]
```

Apibrėžimas failo [Vector.h](#) eilutėje 745.

4.7.4.26 erase() [1/2]

```
template<typename T , typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    const_iterator first,
    const_iterator last ) [inline]
```

Pašalina diapazoną [first, last).

Gražina

Iteratorius į pirmą elementą po pašalintų.

Apibrėžimas failo [Vector.h](#) eilutėje 600.

4.7.4.27 erase() [2/2]

```
template<typename T , typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    const_iterator pos ) [inline]
```

Pašalina elementą pos pozicijoje.

Gražina

Iteratorius po pašalinto elemento.

Apibrėžimas failo [Vector.h](#) eilutėje 584.

4.7.4.28 front() [1/2]

```
template<typename T , typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::front ( ) [inline]
```

Pirmas elementas.

Apibrėžimas failo [Vector.h](#) eilutėje 383.

4.7.4.29 front() [2/2]

```
template<typename T , typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::front ( ) const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 384.

4.7.4.30 get_allocator()

```
template<typename T , typename Allocator = std::allocator<T>>
allocator_type Vector< T, Allocator >::get_allocator ( ) const [inline], [noexcept]
```

Grąžina naudojamą paskirstytoją.

Apibrėžimas failo [Vector.h](#) eilutėje 344.

4.7.4.31 growCap()

```
template<typename T , typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::growCap ( ) const [inline], [private], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 100.

4.7.4.32 insert() [1/5]

```
template<typename T , typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    const T & value ) [inline]
```

Įterpia vieną kopiją prieš pos.

Gražina

Iteratorius į įterptą elementą.

Apibrėžimas failo [Vector.h](#) eilutėje 470.

4.7.4.33 insert() [2/5]

```
template<typename T , typename Allocator = std::allocator<T>>
template<typename InputIt , typename = std::enable_if_t<std::is_base_of_v< std::input_iterator_tag, typename std::iterator_traits<InputIt>::iterator_category>>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    InputIt first,
    InputIt last ) [inline]
```

Įterpia diapazoną [first, last) prieš pos.

Apibrėžimas failo [Vector.h](#) eilutėje 526.

4.7.4.34 insert() [3/5]

```
template<typename T , typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    size_type count,
    const T & value ) [inline]
```

Įterpia count kopijų value prieš pos.

Apibrėžimas failo [Vector.h](#) eilutėje 496.

4.7.4.35 insert() [4/5]

```
template<typename T , typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    std::initializer_list< T > init ) [inline]
```

Įterpia inicializatorių sąrašą prieš pos.

Apibrėžimas failo [Vector.h](#) eilutėje 557.

4.7.4.36 insert() [5/5]

```
template<typename T , typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    T && value ) [inline]
```

Įterpia vieną elementą (move) prieš pos.

Apibrėžimas failo [Vector.h](#) eilutėje 483.

4.7.4.37 max_size()

```
template<typename T , typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::max_size ( ) const [inline], [noexcept]
```

Maksimalus galimas dydis.

Apibrėžimas failo [Vector.h](#) eilutėje 419.

4.7.4.38 operator=() [1/3]

```
template<typename T , typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    const Vector< T, Allocator > & other ) [inline]
```

Kopijavimo priskyrimo operatorius.

Apibrėžimas failo [Vector.h](#) eilutėje 237.

4.7.4.39 operator=() [2/3]

```
template<typename T , typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    std::initializer_list< T > init ) [inline]
```

Inicializatorių sąrašo priskyrimo operatorius.

Apibrėžimas failo [Vector.h](#) eilutėje 287.

4.7.4.40 operator=() [3/3]

```
template<typename T , typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    Vector< T, Allocator > && other ) [inline], [noexcept]
```

Perkėlimo priskyrimo operatorius.

Apibrėžimas failo [Vector.h](#) eilutėje 265.

4.7.4.41 operator[]() [1/2]

```
template<typename T , typename Allocator = std::allocator<T>>
reference Vector< T, Allocator >::operator[] (
    size_type pos ) [inline]
```

Indeksavimo operatorius (be ribų tikrinimo).

Apibrėžimas failo [Vector.h](#) eilutėje 378.

4.7.4.42 operator[]() [2/2]

```
template<typename T , typename Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::operator[] (
    size_type pos ) const [inline]
```

Const indeksavimo operatorius.

Apibrėžimas failo [Vector.h](#) eilutėje 380.

4.7.4.43 pop_back()

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::pop_back ( ) [inline]
```

Pašalina paskutinį elementą.

Apibrėžimas failo [Vector.h](#) eilutėje 670.

4.7.4.44 push_back() [1/2]

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::push_back (
    const T & value ) [inline]
```

Prideda kopiją į galą.

Apibrėžimas failo [Vector.h](#) eilutėje 628.

4.7.4.45 push_back() [2/2]

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::push_back (
    T && value ) [inline]
```

Prideda (move) elementą į galą.

Apibrėžimas failo [Vector.h](#) eilutėje 641.

4.7.4.46 rbegin() [1/2]

```
template<typename T , typename Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::rbegin ( ) const [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 405.

4.7.4.47 rbegin() [2/2]

```
template<typename T , typename Allocator = std::allocator<T>>
reverse_iterator Vector< T, Allocator >::rbegin ( ) [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 404.

4.7.4.48 reallocate()

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::reallocate (
    size_type newCap ) [inline], [private]
```

Apibrėžimas failo [Vector.h](#) eilutėje 76.

4.7.4.49 rend() [1/2]

```
template<typename T , typename Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::rend ( ) const [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 409.

4.7.4.50 `rend()` [2/2]

```
template<typename T , typename Allocator = std::allocator<T>>
reverse_iterator Vector< T, Allocator >::rend ( ) [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 408.

4.7.4.51 `reserve()`

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::reserve (
    size_type newCap ) [inline]
```

Rezervuoja atmintį bent newCap elementams (nekeičia size).

Apibrėžimas failo [Vector.h](#) eilutėje 426.

4.7.4.52 `resize()` [1/2]

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::resize (
    size_type count ) [inline]
```

Pakeičia dydį į count (nauji elementai – value-initialized).

Apibrėžimas failo [Vector.h](#) eilutėje 682.

4.7.4.53 `resize()` [2/2]

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::resize (
    size_type count,
    const T & value ) [inline]
```

Pakeičia dydį į count, naujus elementus užpildo value.

Apibrėžimas failo [Vector.h](#) eilutėje 706.

4.7.4.54 `shiftRight()`

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::shiftRight (
    size_type idx,
    size_type count ) [inline], [private]
```

Pastumia elementus iš [idx, size_) dešinėn per count pozicijų.

Elementai į nepatalpintą (uninitialized) sritį – construct, į jau egzistuojančią – move-assign. Rezultate pozicijos [idx, idx+count) – „inicializuotos, bet perkeltos" arba neinicializuotos – tvarkomos [insert\(\)](#) metodo.

Išakstinė sąlyga

size_ + count <= cap_ (ensureSpace jau iškviesta)

Apibrėžimas failo [Vector.h](#) eilutėje 763.

4.7.4.55 shrink_to_fit()

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::shrink_to_fit ( ) [inline]
```

Sumažina talpos perteklių (cap $\rightarrow$ size).

Apibrėžimas failo [Vector.h](#) eilutėje 438.

4.7.4.56 size()

```
template<typename T , typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::size ( ) const [inline], [noexcept]
```

Elementų skaičius.

Apibrėžimas failo [Vector.h](#) eilutėje 417.

4.7.4.57 swap()

```
template<typename T , typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::swap (
    Vector< T, Allocator > & other ) [inline], [noexcept]
```

Sukeičia du vektorius vietomis ($O(1)$).

Apibrėžimas failo [Vector.h](#) eilutėje 732.

4.7.5 Draugiškų Ir Susijusių Funkcijų Dokumentacija

4.7.5.1 erase()

```
template<typename T , typename A , typename U >
Vector< T, A >::size_type erase (
    Vector< T, A > & c,
    const U & value ) [related]
```

Pašalina visus elementus, lygius value.

Gražina

Pašalintų elementų skaičius.

Apibrėžimas failo [Vector.h](#) eilutėje 839.

4.7.5.2 erase_if()

```
template<typename T , typename A , typename Pred >
Vector< T, A >::size_type erase_if (
    Vector< T, A > & c,
    Pred pred ) [related]
```

Pašalina visus elementus, tenkinančius pred.

Gražina

Pašalintų elementų skaičius.

Apibrėžimas failo [Vector.h](#) eilutėje 853.

4.7.5.3 operator"!="()

```
template<typename T , typename A >
bool operator!= (
    const Vector< T, A > & l,
    const Vector< T, A > & r ) [related]
```

Apibrėžimas failo [Vector.h](#) eilutėje 804.

4.7.5.4 operator<()

```
template<typename T , typename A >
bool operator< (
    const Vector< T, A > & l,
    const Vector< T, A > & r ) [related]
```

Apibrėžimas failo [Vector.h](#) eilutėje 808.

4.7.5.5 operator<=()

```
template<typename T , typename A >
bool operator<= (
    const Vector< T, A > & l,
    const Vector< T, A > & r ) [related]
```

Apibrėžimas failo [Vector.h](#) eilutėje 812.

4.7.5.6 operator==(())

```
template<typename T , typename A >
bool operator== (
    const Vector< T, A > & l,
    const Vector< T, A > & r ) [related]
```

Apibrėžimas failo [Vector.h](#) eilutėje 786.

4.7.5.7 operator>()

```
template<typename T , typename A >
bool operator> (
    const Vector< T, A > & l,
    const Vector< T, A > & r ) [related]
```

Apibrėžimas failo [Vector.h](#) eilutėje 816.

4.7.5.8 operator>=()

```
template<typename T , typename A >
bool operator>= (
    const Vector< T, A > & l,
    const Vector< T, A > & r ) [related]
```

Apibrėžimas failo [Vector.h](#) eilutėje 820.

4.7.5.9 swap()

```
template<typename T , typename A >
void swap (
    Vector< T, A > & l,
    Vector< T, A > & r ) [related]
```

ADL-friendly swap.

Apibrėžimas failo [Vector.h](#) eilutėje 829.

4.7.6 Atributų Dokumentacija

4.7.6.1 alloc_

```
template<typename T , typename Allocator = std::allocator<T>>
Allocator Vector< T, Allocator >::alloc_ [private]
```

Apibrėžimas failo [Vector.h](#) eilutėje 49.

4.7.6.2 cap_

```
template<typename T , typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::cap_ [private]
```

Apibrėžimas failo [Vector.h](#) eilutėje 52.

4.7.6.3 data_

```
template<typename T , typename Allocator = std::allocator<T>>
pointer Vector< T, Allocator >::data_ [private]
```

Apibrėžimas failo [Vector.h](#) eilutėje 50.

4.7.6.4 size_

```
template<typename T , typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::size_ [private]
```

Apibrėžimas failo [Vector.h](#) eilutėje 51.

Dokumentacija šiai klasei sugeneruota iš šio failo:

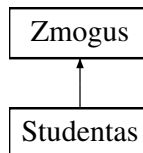
- [Vector.h](#)

4.8 Zmogus Klasė

Abstrakti bazinė klasė, apibrėžianti žmogaus sąsają.

```
#include <Zmogus.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

Rule of Five

- [Zmogus](#) ()=default
Numatytasis konstruktorius.
- [Zmogus](#) (const std::string &v, const std::string &p)
Konstruktorius su parametrais.
- [Zmogus](#) (const [Zmogus](#) &)=default
Kopijavimo konstruktorius.
- [Zmogus](#) & operator= (const [Zmogus](#) &)=default
Kopijavimo priskyrimo operatorius.
- [Zmogus](#) ([Zmogus](#) &&)=default
Perkėlimo konstruktorius.
- [Zmogus](#) & operator= ([Zmogus](#) &&)=default
Perkėlimo priskyrimo operatorius.
- virtual ~[Zmogus](#) ()=default
Virtualus destruktorius.
- virtual std::string [getVardas](#) () const
- virtual std::string [getPavarde](#) () const
- void [setVardas](#) (const std::string &v)
- void [setPavarde](#) (const std::string &p)

Grynosios virtualios funkcijos

Išvestinės klasės privalo šias funkcijas realizuoti.

- virtual void [suskaiciuotiGalutini](#) ()=0
Apskaičiuoja galutinį įvertinimą.
- virtual void [spausdinti](#) (std::ostream &os) const =0
Išspausdina objekto duomenis į srautą.

Apsaugoti Atributai

- `std::string vardas`
- `std::string pavarde`

4.8.1 Smulkus aprašymas

Abstrakti bazinė klasė, apibrėžianti žmogaus sąsają.

Saugo vardą ir pavardę, bei deklaruoja grynąsias virtualias funkcijas, kurias privalo realizuoti išvestinės klasės. Realizuoja Rule of Five naudojant `= default` direktyvą.

Apibrėžimas failo [Zmogus.h](#) eilutėje 27.

4.8.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.8.2.1 Zmogus() [1/4]

```
Zmogus::Zmogus ( ) [default]
```

Numatytasis konstruktorius.

Inicializuoja tuščius laukus.

4.8.2.2 Zmogus() [2/4]

```
Zmogus::Zmogus (
    const std::string & v,
    const std::string & p )
```

Konstruktorius su parametrais.

Parametrai

<i>v</i>	Vardas.
<i>p</i>	Pavardė.

Apibrėžimas failo [Zmogus.cpp](#) eilutėje 3.

4.8.2.3 Zmogus() [3/4]

```
Zmogus::Zmogus (
    const Zmogus & ) [default]
```

Kopijavimo konstruktorius.

4.8.2.4 Zmogus() [4/4]

```
Zmogus::Zmogus (
    Zmogus && ) [default]
```

Perkėlimo konstruktorius.

4.8.2.5 ~Zmogus()

```
virtual Zmogus::~Zmogus ( ) [virtual], [default]
```

Virtualus destruktorius.

Užtikrina teisingą išvestinių klasių sunaikinimą per bazinės klasės rodyklę.

4.8.3 Metodų Dokumentacija

4.8.3.1 getPavarde()

```
virtual std::string Zmogus::getPavarde ( ) const [inline], [virtual]
```

Apibrėžimas failo [Zmogus.h](#) eilutėje 70.

4.8.3.2 getVardas()

```
virtual std::string Zmogus::getVardas ( ) const [inline], [virtual]
```

Apibrėžimas failo [Zmogus.h](#) eilutėje 69.

4.8.3.3 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & ) [default]
```

Kopijavimo priskyrimo operatorius.

4.8.3.4 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && ) [default]
```

Perkėlimo priskyrimo operatorius.

4.8.3.5 setPavarde()

```
void Zmogus::setPavarde (
    const std::string & p ) [inline]
```

Apibrėžimas failo [Zmogus.h](#) eilutėje 73.

4.8.3.6 setVardas()

```
void Zmogus::setVardas (
    const std::string & v ) [inline]
```

Apibrėžimas failo [Zmogus.h](#) eilutėje 72.

4.8.3.7 spausdinti()

```
virtual void Zmogus::spausdinti (
    std::ostream & os ) const [pure virtual]
```

Išspausdina objekto duomenis į srautą.

Parametrai

<i>os</i>	Išvesties srautas.
-----------	--------------------

Realizuota [Studentas](#).

4.8.3.8 suskaiciuotiGalutini()

```
virtual void Zmogus::suskaiciuotiGalutini ( ) [pure virtual]
```

Apskaičiuoja galutinį įvertinimą.

Konkretus skaičiavimo algoritmas priklauso nuo išvestinės klasės.

Realizuota [Studentas](#).

4.8.4 Atributų Dokumentacija

4.8.4.1 pavarde

```
std::string Zmogus::pavarde [protected]
```

Apibrėžimas failo [Zmogus.h](#) eilutėje 31.

4.8.4.2 vardas

```
std::string Zmogus::vardas [protected]
```

Apibrėžimas failo [Zmogus.h](#) eilutėje 30.

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [Zmogus.h](#)
- [Zmogus.cpp](#)

skyrius 5

Failo Dokumentacija

5.1 Generavimas.cpp Failo Nuoroda

```
#include "Generavimas.h"
#include "laikai.h"
#include "spausdinam.h"
#include <algorithm>
#include <chrono>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <random>
#include <stdexcept>
#include <string>
```

Funkcijos

- static bool [galimasPavadinimas](#) (const std::string &pav)
• std::string [failoPavadinimas](#) (const std::string &bazinisPavadinimas)
Sugeneruoja unikali failo pavadinimą.
- [SkirstymoStrategija pasirinktiStrategija](#) ()
Interaktyviai prašo vartotojo pasirinkti skirstymo strategiją.
- [Skirstymorezultatas skirstymasStrategija1](#) (const [StudContainer](#) &studis)
Skirsto studentus pagal 1-ą strategiją.
- [Skirstymorezultatas skirstymasStrategija2](#) ([StudContainer](#) &studis)
Skirsto studentus pagal 2-ą strategiją.
- [Skirstymorezultatas skirstymasStrategija3](#) ([StudContainer](#) &studis)
Skirsto studentus pagal 3-ą strategiją (efektyviausia).
- [Skirstymorezultatas skirstymasGrupes](#) ([StudContainer](#) &studis, [SkirstymoStrategija](#) strategija, bool irasyti↔
IFailus, bool paprasytiRikiavimo)
Bendro naudojimo skirstymo funkcija.
- void [generuotiFaila](#) ()
Generuoja studentų duomenų failą.

5.1.1 Funkcijos Dokumentacija

5.1.1.1 failoPavadinimas()

```
std::string failoPavadinimas (
    const std::string & bazinisPavadinimas )
```

Sugeneruoja unikalų failo pavadinimą.

Jei failas su nurodytu pavadinimu jau egzistuoja, automatiškai pridedamas skaitinis sufiksas (pvz. _2, _3).

Parametrai

<i>bazinisPavadinimas</i>	Bazinis failo pavadinimas be plėtinio.
---------------------------	--

Gražina

Unikalus failo pavadinimas su .txt plėtiniu.

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 22.

5.1.1.2 galimasPavadinimas()

```
static bool galimasPavadinimas (
    const std::string & pav ) [static]
```

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 16.

5.1.1.3 generuotiFaila()

```
void generuotiFaila ( )
```

Generuoja studentų duomenų failą.

Interaktyviai prašo vartotojo įvesti failo pavadinimą ir studentų kiekį. Studentų vardai, pavardės ir pažymiai generuojami automatiškai.

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 174.

5.1.1.4 pasirinktiStrategija()

```
SkirstymoStrategija pasirinktiStrategija ( )
```

Interaktyviai prašo vartotojo pasirinkti skirstymo strategiją.

Išveda meniu į konsolę ir nuskaito vartotojo pasirinkimą.

Gražina

Pasirinkta `SkirstymoStrategija`.

Išimtys

<code>std::runtime_error</code>	Jei vartotojo įvestis neteisinga.
---------------------------------	-----------------------------------

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 36.

5.1.1.5 skirstymasGrupes()

```
Skirstymorezultatas skirstymasGrupes (
    StudContainer & studis,
    SkirstymoStrategija strategija,
    bool irasytiIFailus = true,
    bool paprasytiRikiavimo = true )
```

Bendro naudojimo skirstymo funkcija.

Išsaugo laiko matavimus į globalų `timers` objektą. Pagal parametrus gali rašyti į failus ir prašyti rikiavimo pasirinkimo.

Parametrai

<code>studis</code>	Studentų konteineris.
<code>strategija</code>	Naudojama skirstymo strategija.
<code>irasytiIFailus</code>	Jei <code>true</code> , rezultatai išsaugomi į failus.
<code>paprasytiRikiavimo</code>	Jei <code>true</code> , vartotojas gali pasirinkti rikiavimą.

Gražina

`Skirstymorezultatas` su vargsiukais ir protais.

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 112.

5.1.1.6 skirstymasStrategija1()

```
Skirstymorezultatas skirstymasStrategija1 (
    const StudContainer & studis )
```

Skirsto studentus pagal 1-ą strategiją.

Eina per visus studentus ir juos kopijuoja į atskirus konteinerius. Originalus konteineris nekeičiamas.

Parametrai

<code>studis</code>	Studentų konteineris (const – nekeičiamas).
---------------------	---

Gražina

`Skirstymorezultatas` su dviem užpildytais konteineriais.

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 54.

5.1.1.7 skirstymasStrategija2()

```
Skirstymorezultatas skirstymasStrategija2 (
    StudContainer & studis )
```

Skirsto studentus pagal 2-ą strategiją.

Vargsiukai ištraukiami iš originalaus konteinerio (erase). Likę studentai (protai) perkeliama į rezultatą.

Parametrai

<code>studis</code>	Studentų konteineris (keičiamas – iš jo šalinami vargsiukai).
---------------------	---

Gražina

```
Skirstymorezultatas.
```

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 73.

5.1.1.8 skirstymasStrategija3()

```
Skirstymorezultatas skirstymasStrategija3 (
    StudContainer & studis )
```

Skirsto studentus pagal 3-ą strategiją (efektyviausia).

Naudoja `std::partition` algoritmą, kuris perskirsto elementus vietoje, tuomet siunčia į du atskirus konteinerius.

Parametrai

<code>studis</code>	Studentų konteineris (keičiamas).
---------------------	-----------------------------------

Gražina

```
Skirstymorezultatas.
```

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 94.

5.2 Generavimas.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "Generavimas.h"
00002
00003 #include "laikai.h"
00004 #include "spausdinam.h"
00005
00006 #include <algorithm>
00007 #include <chrono>
00008 #include <fstream>
00009 #include <iomanip>
00010 #include <iostream>
```

```

00011 #include <random>
00012 #include <stdexcept>
00013 #include <string>
00014
00015
00016 static bool galimasPavadinimas(const std::string& pav)
00017 {
00018     std::ifstream f(pav);
00019     return f.is_open();
00020 }
00021
00022 std::string failoPavadinimas(const std::string& bazinisPavadinimas)
00023 {
00024     std::string pavadinimas = bazinisPavadinimas + ".txt";
00025     int count = 1;
00026
00027     while (galimasPavadinimas(pavadinimas))
00028     {
00029         count++;
00030         pavadinimas = bazinisPavadinimas + "_" + std::to_string(count) + ".txt";
00031     }
00032
00033     return pavadinimas;
00034 }
00035
00036 SkirstymoStrategija pasirinktiStrategija()
00037 {
00038     std::cout << "\nPasirinkite studentu skirstymo strategija:\n"
00039               << "1 - Du nauji konteineriai (vargsiukai ir protai)\n"
00040               << "2 - Tik vargsiuku konteineris; protus palikti bendrame\n"
00041               << "3 - Efektyvi strategija su std::partition\n";
00042
00043     int p = 0;
00044     std::cin >> p;
00045
00046     if (!std::cin || p < 1 || p > 3)
00047     {
00048         throw std::runtime_error("Neteisingas strategijos pasirinkimas.");
00049     }
00050
00051     return static_cast<SkirstymoStrategija>(p);
00052 }
00053
00054 Skirstymorezultatas skirstymasStrategijal(const StudContainer& studis)
00055 {
00056     Skirstymorezultatas rezultatas;
00057
00058     for (const auto& s : studis)
00059     {
00060         if (s.getgalrezVid() < 5.0f)
00061         {
00062             rezultatas.vargsiukai.push_back(s);
00063         }
00064         else
00065         {
00066             rezultatas.protai.push_back(s);
00067         }
00068     }
00069
00070     return rezultatas;
00071 }
00072
00073 Skirstymorezultatas skirstymasStrategija2(StudContainer& studis)
00074 {
00075     Skirstymorezultatas rezultatas;
00076
00077     for (auto i = studis.begin(); i != studis.end(); )
00078     {
00079         if (i->getgalrezVid() < 5.0f)
00080         {
00081             rezultatas.vargsiukai.push_back(*i);
00082             i = studis.erase(i);
00083         }
00084         else
00085         {
00086             i++;
00087         }
00088     }
00089
00090     rezultatas.protai = std::move(studis);
00091     return rezultatas;
00092 }
00093
00094 Skirstymorezultatas skirstymasStrategija3(StudContainer& studis)
00095 {
00096     Skirstymorezultatas rezultatas;
00097

```

```

00098     auto riba = std::partition(studis.begin(), studis.end(),
00099                               [](const Studentas& s)
00100                               {
00101                                   return s.getgalrezVid() >= 5.0f;
00102                               });
00103
00104     rezultatas.vargsiukai.assign(riba, studis.end());
00105     rezultatas.protai.assign(studis.begin(), riba);
00106
00107     studis.clear();
00108
00109     return rezultatas;
00110 }
00111
00112 Skirstymorezultatas skirstymasGrupes(
00113     StudContainer& studis,
00114     SkirstymoStrategija strategija,
00115     bool irasytiIFailus,
00116     bool paprasytiRikiavimo)
00117 {
00118     auto start = std::chrono::high_resolution_clock::now();
00119
00120     Skirstymorezultatas rezultatas;
00121
00122     switch (strategija)
00123     {
00124     case SkirstymoStrategija::Pirma:
00125     {
00126         rezultatas = skirstymasStrategijal(studis);
00127         break;
00128     }
00129     case SkirstymoStrategija::Antra:
00130     {
00131         rezultatas = skirstymasStrategija2(studis);
00132         break;
00133     }
00134     case SkirstymoStrategija::Trecia:
00135     {
00136         rezultatas = skirstymasStrategija3(studis);
00137         break;
00138     }
00139     }
00140
00141     auto end = std::chrono::high_resolution_clock::now();
00142     timers.skirstymas = std::chrono::duration<double>(end - start).count();
00143
00144     if (!irasytiIFailus)
00145     {
00146         timers.isvedimas = 0.0;
00147         return rezultatas;
00148     }
00149
00150     if (paprasytiRikiavimo)
00151     {
00152         rikiuotiStudentus(rezultatas.vargsiukai, "vargsiukus");
00153         rikiuotiStudentus(rezultatas.protai, "protus");
00154     }
00155
00156     auto startPrint = std::chrono::high_resolution_clock::now();
00157
00158     const std::string sufiksas =
00159         "_" + aktyvausKonteinerioTrumpasPavadinimas() +
00160         "_S" + std::to_string(static_cast<int>(strategija));
00161
00162     spausdinam(rezultatas.vargsiukai, failoPavadinimas("Vargsiukai" + sufiksas));
00163     spausdinam(rezultatas.protai, failoPavadinimas("protai" + sufiksas));
00164
00165     auto endPrint = std::chrono::high_resolution_clock::now();
00166     timers.isvedimas = std::chrono::duration<double>(endPrint - startPrint).count();
00167
00168     std::cout << "Studentai suskirstyti. Vargsiuku: " << rezultatas.vargsiukai.size()
00169               << ", protu: " << rezultatas.protai.size() << std::endl;
00170
00171     return rezultatas;
00172 }
00173
00174 void generuotiFaila()
00175 {
00176     std::cout << "Iveskite failo pavadinima: ";
00177     std::string pav;
00178     std::cin >> pav;
00179
00180     std::ofstream write(pav);
00181     if (!write.is_open())
00182     {
00183         std::cerr << "Nepavyko sukurti failo." << std::endl;
00184         return;

```

```

00185     }
00186
00187     std::mt19937 gen(std::random_device{}());
00188     std::uniform_int_distribution<int> dist(1, 10);
00189
00190     std::cout << "Iveskite studentu kieki: ";
00191     int kiekis = 0;
00192     std::cin >> kiekis;
00193
00194     auto start = std::chrono::high_resolution_clock::now();
00195
00196     const int ndKiek = dist(gen);
00197
00198     write << std::left << std::setw(15) << "Vardas" << std::setw(15) << "Pavarde";
00199     for (int j = 0; j < ndKiek; ++j)
00200     {
00201         write << std::setw(8) << ("ND" + std::to_string(j + 1));
00202     }
00203     write << std::setw(8) << "Egz." << '\n';
00204
00205     for (int i = 0; i < kiekis; ++i)
00206     {
00207         write << std::left
00208             << std::setw(15) << ("Vardas" + std::to_string(i + 1))
00209             << std::setw(15) << ("Pavarde" + std::to_string(i + 1));
00210
00211         for (int j = 0; j < ndKiek; ++j)
00212         {
00213             write << std::setw(8) << dist(gen);
00214         }
00215
00216         write << std::setw(8) << dist(gen) << '\n'; // egzaminas
00217     }
00218
00219     auto end = std::chrono::high_resolution_clock::now();
00220     timers.generavimas = std::chrono::duration<double>(end - start).count();
00221
00222     std::cout << "Failas \" << pav << "\" sugeneruotas per "
00223         << timers.generavimas << " s." << std::endl;
00224 }

```

5.3 Generavimas.h Failo Nuoroda

Studentų failų generavimo ir skirstymo į grupes funkcijų deklaracijos.

```

#include "konteineris.h"
#include <string>

```

Klasės

- struct [Skirstymorezultatas](#)
Saugo skirstymo į grupes rezultatus.

Išvardinimai

- enum class [SkirstymoStrategija](#) { [Pirma](#) = 1 , [Antra](#) = 2 , [Trecia](#) = 3 }
Nurodo, kokių algoritmu skirstyti studentus į grupes.

Funkcijos

- void [generuotiFaila](#) ()
Generuoja studentų duomenų failą.
- std::string [failoPavadinimas](#) (const std::string &bazinisPavadinimas)
Sugeneruoja unikalų failo pavadinimą.
- [SkirstymoStrategija](#) pasirinktiStrategija ()
Interaktyviai prašo vartotojo pasirinkti skirstymo strategiją.
- [Skirstymorezultatas](#) skirstymasStrategija1 (const [StudContainer](#) &studis)
Skirsto studentus pagal 1-ą strategiją.
- [Skirstymorezultatas](#) skirstymasStrategija2 ([StudContainer](#) &studis)
Skirsto studentus pagal 2-ą strategiją.
- [Skirstymorezultatas](#) skirstymasStrategija3 ([StudContainer](#) &studis)
Skirsto studentus pagal 3-ą strategiją (efektyviausia).
- [Skirstymorezultatas](#) skirstymasGrupes ([StudContainer](#) &studis, [SkirstymoStrategija](#) strategija, bool irasyti↔ IFailus=true, bool paprasytiRikiavimo=true)
Bendro naudojimo skirstymo funkcija.

5.3.1 Smulkus aprašymas

Studentų failų generavimo ir skirstymo į grupes funkcijų deklaracijos.

Apibrėžia [SkirstymoStrategija](#) enumeraciją, [Skirstymorezultatas](#) struktūrą ir visas su studentų generavimu bei grupavimu susijusias funkcijas.

Autorius

[Studentas](#)

Versija

3.0

Apibrėžimas faile [Generavimas.h](#).

5.3.2 Išvardinimo Tipo Dokumentacija

5.3.2.1 SkirstymoStrategija

```
enum class SkirstymoStrategija [strong]
```

Nurodo, koku algoritmu skirstyti studentus į grupes.

Reikšmė	Aprašymas
Pirma	Du nauji konteineriai (vargsiukai ir protai)
Antra	Vargsiukai ištraukiami iš bendro konteinerio; protai lieka
Trecia	Efektyvus <code>std::partition</code> algoritmas

Išvardinimų reikšmės

Pirma	
Antra	
Trecia	

Apibrėžimas failo [Generavimas.h](#) eilutėje 29.

5.3.3 Funkcijos Dokumentacija

5.3.3.1 failoPavadinimas()

```
std::string failoPavadinimas (
    const std::string & bazinisPavadinimas )
```

Sugeneruoja unikalų failo pavadinimą.

Jei failas su nurodytu pavadinimu jau egzistuoja, automatiškai pridedamas skaitinis sufiksas (pvz. _2, _3).

Parametrai

<i>bazinisPavadinimas</i>	Bazinis failo pavadinimas be plėtinio.
---------------------------	--

Gražina

Unikalus failo pavadinimas su .txt plėtiniu.

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 22.

5.3.3.2 generuotiFaila()

```
void generuotiFaila ( )
```

Generuoja studentų duomenų failą.

Interaktyviai prašo vartotojo įvesti failo pavadinimą ir studentų kiekį. Studentų vardai, pavardės ir pažymiai generuojami automatiškai.

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 174.

5.3.3.3 pasirinktiStrategija()

```
SkirstymoStrategija pasirinktiStrategija ( )
```

Interaktyviai prašo vartotojo pasirinkti skirstymo strategiją.

Išveda meniu į konsolę ir nuskaito vartotojo pasirinkimą.

Gražina

Pasirinkta SkirstymoStrategija.

Išimtys

<code>std::runtime_error</code>	Jei vartotojo įvestis neteisinga.
---------------------------------	-----------------------------------

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 36.

5.3.3.4 skirstymasGrupes()

```
Skirstymorezultatas skirstymasGrupes (
    StudContainer & studis,
    SkirstymoStrategija strategija,
    bool irasytiIFailus = true,
    bool paprasytiRikiavimo = true )
```

Bendro naudojimo skirstymo funkcija.

Išsaugo laiko matavimus į globalų `timers` objektą. Pagal parametrus gali rašyti į failus ir prašyti rikiavimo pasirinkimo.

Parametrai

<code>studis</code>	Studentų konteineris.
<code>strategija</code>	Naudojama skirstymo strategija.
<code>irasytiIFailus</code>	Jei <code>true</code> , rezultatai išsaugomi į failus.
<code>paprasytiRikiavimo</code>	Jei <code>true</code> , vartotojas gali pasirinkti rikiavimą.

Gražina

`Skirstymorezultatas` su vargsiukais ir protais.

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 112.

5.3.3.5 skirstymasStrategija1()

```
Skirstymorezultatas skirstymasStrategija1 (
    const StudContainer & studis )
```

Skirsto studentus pagal 1-ą strategiją.

Eina per visus studentus ir juos kopijuoja į atskirus konteinerius. Originalus konteineris nekeičiamas.

Parametrai

<code>studis</code>	Studentų konteineris (const – nekeičiamas).
---------------------	---

Gražina

`Skirstymorezultatas` su dviem užpildytais konteineriais.

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 54.

5.3.3.6 skirstymasStrategija2()

```
Skirstymorezultatas skirstymasStrategija2 (
    StudContainer & studis )
```

Skirsto studentus pagal 2-ą strategiją.

Vargsiukai ištraukiami iš originalaus konteinerio (erase). Likę studentai (protai) perkeliama į rezultatą.

Parametrai

<i>studis</i>	Studentų konteineris (keičiamas – iš jo šalinami vargsiukai).
---------------	---

Gražina

Skirstymorezultatas.

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 73.

5.3.3.7 skirstymasStrategija3()

```
Skirstymorezultatas skirstymasStrategija3 (
    StudContainer & studis )
```

Skirsto studentus pagal 3-ą strategiją (efektyviausia).

Naudoja `std::partition` algoritmą, kuris perskirsto elementus vietoje, tuomet siunčia į du atskirus konteinerius.

Parametrai

<i>studis</i>	Studentų konteineris (keičiamas).
---------------	-----------------------------------

Gražina

Skirstymorezultatas.

Apibrėžimas failo [Generavimas.cpp](#) eilutėje 94.

5.4 Generavimas.h

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file Generavimas.h
00003  * @brief Studentų failų generavimo ir skirstymo į grupes funkcijų deklaracijos.
00004  *
00005  * Apibrėžia `SkirstymoStrategija` enumeraciją, `Skirstymorezultatas` struktūrą
00006  * ir visas su studentų generavimu bei grupavimu susijusias funkcijas.
00007  *
00008  * @author Studentas
00009  * @version 3.0
00010  */
```

```

00011
00012 #pragma once
00013
00014 #include "konteineris.h"
00015
00016 #include <string>
00017
00018 /**
00019  * @enum SkirstymoStrategija
00020  * @brief Nurodo, koku algoritmu skirstyti studentus į grupes.
00021  *
00022  * | Reikšmė | Aprašymas |
00023  * |-----|-----|
00024  * | Pirma   | Du nauji konteineriai (vargsiukai ir protai) |
00025  * | Antra   | Vargsiukai ištraukiami iš bendro konteinerio; protai lieka |
00026  * | Trecia  | Efektyvus `std::partition` algoritmas |
00027  */
00028
00029 enum class SkirstymoStrategija
00030 {
00031     Pirma = 1,
00032     Antra = 2,
00033     Trecia = 3
00034 };
00035
00036 /**
00037  * @struct Skirstymorezultatas
00038  * @brief Saugo skirstymo į grupes rezultatus.
00039  */
00040
00041 struct Skirstymorezultatas
00042 {
00043     StudContainer vargsiukai;
00044     StudContainer protai;
00045 };
00046
00047 /**
00048  * @brief Generuoja studentų duomenų failą.
00049  *
00050  * Interaktyviai prašo vartotojo įvesti failo pavadinimą ir studentų kiekį.
00051  * Studentų vardai, pavardės ir pažymiai generuojami automatiškai.
00052  */
00053 void generuotiFaila();
00054
00055 /**
00056  * @brief Sugeneruoja unikalų failo pavadinimą.
00057  *
00058  * Jei failas su nurodytu pavadinimu jau egzistuoja, automatiškai
00059  * pridedamas skaitinis sufiksas (pvz. `_2`, `_3`).
00060  *
00061  * @param bazinisPavadinimas Bazinis failo pavadinimas be plėtinio.
00062  * @return Unikalus failo pavadinimas su `.txt` plėtiniu.
00063  */
00064
00065 std::string failoPavadinimas(const std::string& bazinisPavadinimas);
00066
00067 /**
00068  * @brief Interaktyviai prašo vartotojo pasirinkti skirstymo strategiją.
00069  *
00070  * Išveda meniu į konsolę ir nuskaito vartotojo pasirinkimą.
00071  *
00072  * @return Pasirinkta `SkirstymoStrategija`.
00073  * @throws std::runtime_error Jei vartotojo įvestis neteisinga.
00074  */
00075
00076 SkirstymoStrategija pasirinktiStrategija();
00077
00078 /**
00079  * @brief Skirsto studentus pagal 1-ą strategiją.
00080  *
00081  * Eina per visus studentus ir juos kopijuoja į atskirus konteinerius.
00082  * Originalus konteineris nekeičiamas.
00083  *
00084  * @param studis Studentų konteineris (const - nekeičiamas).
00085  * @return `Skirstymorezultatas` su dviem užpildytais konteineriais.
00086  */
00087
00088 Skirstymorezultatas skirstymasStrategijal(const StudContainer& studis);
00089
00090 /**
00091  * @brief Skirsto studentus pagal 2-ą strategiją.
00092  *
00093  * Vargsiukai ištraukiami iš originalaus konteinerio (erase).
00094  * Likę studentai (protai) perkeliama į rezultatą.
00095  *
00096  * @param studis Studentų konteineris (keičiamas - iš jo šalinami vargsiukai).
00097  * @return `Skirstymorezultatas`.

```

```

00098  */
00099
00100 Skirstymorezultatas skirstymasStrategija2(StudContainer& studis);
00101
00102 /**
00103  * @brief Skirsto studentus pagal 3-ą strategiją (efektyviausia).
00104  *
00105  * Naudoja `std::partition` algoritmą, kuris perskirsto elementus
00106  * vietoje, tuomet siunčia į du atskirus konteinerius.
00107  *
00108  * @param studis Studentų konteineris (keičiamas).
00109  * @return `Skirstymorezultatas`.
00110  */
00111
00112 Skirstymorezultatas skirstymasStrategija3(StudContainer& studis);
00113
00114 /**
00115  * @brief Bendro naudojimo skirstymo funkcija.
00116  *
00117  * Išsaugo laiko matavimus į globalų `timers` objektą.
00118  * Pagal parametrus gali rašyti į failus ir prašyti rikiavimo pasirinkimo.
00119  *
00120  * @param studis Studentų konteineris.
00121  * @param strategija Naudojama skirstymo strategija.
00122  * @param irasytiIFailus Jei `true`, rezultatai išsaugomi į failus.
00123  * @param paprasytiRikiavimo Jei `true`, vartotojas gali pasirinkti rikiavimą.
00124  * @return `Skirstymorezultatas` su vargsiukais ir protais.
00125  */
00126
00127 Skirstymorezultatas skirstymasGrupes(
00128     StudContainer& studis,
00129     SkirstymoStrategija strategija,
00130     bool irasytiIFailus = true,
00131     bool paprasytiRikiavimo = true);

```

5.5 konteineris.h Failo Nuoroda

```

#include "Studentas.h"
#include <algorithm>
#include <deque>
#include <list>
#include <string>
#include <vector>
#include "Vector.h"

```

Tipų apibrėžimai

- `template<typename T>`
using `StudentuKonteineris` = `Vector< T >`
- using `StudContainer` = `StudentuKonteineris< Studentas >`

Funkcijos

- `std::string aktyvausKonteinerioPavadinimas ()`
- `std::string aktyvausKonteinerioTrumpasPavadinimas ()`
- `template<typename T, typename Alloc, typename Compare >`
void `konteinerioSort` (`std::list< T, Alloc > &konteineris`, `Compare comp`)
- `template<typename Container, typename Compare >`
void `konteinerioSort` (`Container &konteineris`, `Compare comp`)

5.5.1 Tipų apibrėžimų Dokumentacija

5.5.1.1 StudContainer

```
using StudContainer = StudentuKonteineris<Studentas>
```

Apibrėžimas failo [konteineris.h](#) eilutėje 72.

5.5.1.2 StudentuKonteineris

```
template<typename T >  
using StudentuKonteineris = Vector<T>
```

Apibrėžimas failo [konteineris.h](#) eilutėje 44.

5.5.2 Funkcijos Dokumentacija

5.5.2.1 aktyvausKonteinerioPavadinimas()

```
std::string aktyvausKonteinerioPavadinimas ( ) [inline]
```

Apibrėžimas failo [konteineris.h](#) eilutėje 46.

5.5.2.2 aktyvausKonteinerioTrumpasPavadinimas()

```
std::string aktyvausKonteinerioTrumpasPavadinimas ( ) [inline]
```

Apibrėžimas failo [konteineris.h](#) eilutėje 51.

5.5.2.3 konteinerioSort() [1/2]

```
template<typename Container , typename Compare >  
void konteinerioSort (   
    Container & konteineris,  
    Compare comp )
```

Apibrėžimas failo [konteineris.h](#) eilutėje 81.

5.5.2.4 konteinerioSort() [2/2]

```
template<typename T , typename Alloc , typename Compare >  
void konteinerioSort (   
    std::list< T, Alloc > & konteineris,  
    Compare comp )
```

Apibrėžimas failo [konteineris.h](#) eilutėje 75.

5.6 konteineris.h

Eiti į šio failo dokumentaciją.

```

00001 #pragma once
00002
00003 #include "Studentas.h"
00004
00005 #include <algorithm>
00006 #include <deque>
00007 #include <list>
00008 #include <string>
00009 #include <vector>
00010
00011 #if defined(USE_LIST)
00012 template <typename T>
00013 using StudentuKonteineris = std::list<T>;
00014
00015 inline std::string aktyvausKonteinerioPavadinimas()
00016 {
00017     return "std::list";
00018 }
00019
00020 inline std::string aktyvausKonteinerioTrumpasPavadinimas()
00021 {
00022     return "list";
00023 }
00024
00025 #elif defined(USE_DEQUE)
00026 template <typename T>
00027 using StudentuKonteineris = std::deque<T>;
00028
00029 inline std::string aktyvausKonteinerioPavadinimas()
00030 {
00031     return "std::deque";
00032 }
00033
00034 inline std::string aktyvausKonteinerioTrumpasPavadinimas()
00035 {
00036     return "deque";
00037 }
00038
00039 #elif defined(USE_MYVECTOR)
00040
00041 #include "Vector.h"
00042
00043 template <typename T>
00044 using StudentuKonteineris = Vector<T>;
00045
00046 inline std::string aktyvausKonteinerioPavadinimas()
00047 {
00048     return "Vector<T> (nuosava realizacija)";
00049 }
00050
00051 inline std::string aktyvausKonteinerioTrumpasPavadinimas()
00052 {
00053     return "myvector";
00054 }
00055
00056 #else
00057 template <typename T>
00058 using StudentuKonteineris = std::vector<T>;
00059
00060 inline std::string aktyvausKonteinerioPavadinimas()
00061 {
00062     return "std::vector";
00063 }
00064
00065 inline std::string aktyvausKonteinerioTrumpasPavadinimas()
00066 {
00067     return "vector";
00068 }
00069 #endif
00070
00071 // StudContainer dabar laiko objektus ( yay :) )
00072 using StudContainer = StudentuKonteineris<Studentas>;
00073
00074 template <typename T, typename Alloc, typename Compare>
00075 void konteinerioSort(std::list<T, Alloc>& konteineris, Compare comp)
00076 {
00077     konteineris.sort(comp);
00078 }
00079
00080 template <typename Container, typename Compare>
00081 void konteinerioSort(Container& konteineris, Compare comp)
00082 {

```

```
00083     std::sort(konteineris.begin(), konteineris.end(), comp);
00084 }
```

5.7 laikai.cpp Failo Nuoroda

```
#include "laikai.h"
```

Funkcijos

- void [nunulintiLaikus\(\)](#)
Nulinina visus laiko matavimus.

Kintamieji

- [TimerResults timers](#)
Globalus laiko matavimų objektas.

5.7.1 Funkcijos Dokumentacija

5.7.1.1 nunulintiLaikus()

```
void nunulintiLaikus ( )
```

Nulinina visus laiko matavimus.

Iškviečiama kiekvienos naujos iteracijos pradžioje.

Apibrėžimas failo [laikai.cpp](#) eilutėje 5.

5.7.2 Kintamojo Dokumentacija

5.7.2.1 timers

```
TimerResults timers
```

Globalus laiko matavimų objektas.

Apibrėžimas failo [laikai.cpp](#) eilutėje 3.

5.8 laikai.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "laikai.h"
00002
00003 TimerResults timers;
00004
00005 void nunulintiLaikus()
00006 {
00007     timers = TimerResults{};
00008 }
```

5.9 laikai.h Failo Nuoroda

Laiko matavimų struktūra ir pagalbinės funkcijos.

Klasės

- struct [TimerResults](#)
Saugo kiekvienos programos fazės laiko matavimus (sekundėmis).

Funkcijos

- void [nunulintiLaikus](#) ()
Nulinina visus laiko matavimus.

Kintamieji

- [TimerResults timers](#)
Globalus laiko matavimų objektas.

5.9.1 Smulkus aprašymas

Laiko matavimų struktūra ir pagalbinės funkcijos.

Apibrėžia globalų `timers` objektą, kuriame kaupiami kiekvienos programos fazės trukmės matavimai.

Autorius

[Studentas](#)

Versija

3.0

Apibrėžimas faile [laikai.h](#).

5.9.2 Funkcijos Dokumentacija

5.9.2.1 `nunulintiLaikus()`

```
void nunulintiLaikus ( )
```

Nulinina visus laiko matavimus.

Iškviečiama kiekvienos naujos iteracijos pradžioje.

Apibrėžimas failo [laikai.cpp](#) eilutėje 5.

5.9.3 Kintamojo Dokumentacija

5.9.3.1 timers

`TimerResults` timers [extern]

Globalus laiko matavimų objektas.

Apibrėžimas failo `laikai.cpp` eilutėje 3.

5.10 laikai.h

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file laikai.h
00003  * @brief Laiko matavimų struktūra ir pagalbinės funkcijos.
00004  *
00005  * Apibrėžia globalų 'timers' objektą, kuriame kaupiami kiekvienos
00006  * programos fazės trukmės matavimai.
00007  *
00008  * @author Studentas
00009  * @version 3.0
00010  */
00011
00012
00013 #pragma once
00014
00015 /**
00016  * @struct TimerResults
00017  * @brief Saugo kiekvienos programos fazės laiko matavimus (sekundėmis).
00018  */
00019
00020 struct TimerResults
00021 {
00022     double generavimas = 0.0;
00023     double skaitymas = 0.0;
00024     double rusiavimas = 0.0;
00025     double skirstymas = 0.0;
00026     double isvedimas = 0.0;
00027 };
00028
00029 /** @brief Globalus laiko matavimų objektas. */
00030 extern TimerResults timers;
00031
00032 /**
00033  * @brief Nulinina visus laiko matavimus.
00034  *
00035  * Iškviečiama kiekvienos naujos iteracijos pradžioje.
00036  */
00037
00038 void nunulintiLaikus();
```

5.11 main.cpp Failo Nuoroda

```
#include "Generavimas.h"
#include "konteineris.h"
#include "laikai.h"
#include "menu.h"
#include "skaitymas.h"
#include "spausdinam.h"
#include <chrono>
#include <iomanip>
#include <iostream>
#include <limits>
```


Funkcijos

- int `main` ()

5.11.1 Funkcijos Dokumentacija

5.11.1.1 main()

```
int main ( )
```

Apibrėžimas failo `main.cpp` eilutėje 122.

5.12 main.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "Generavimas.h"
00002 #include "konteineris.h"
00003 #include "laikai.h"
00004 #include "menu.h"
00005 #include "skaitymas.h"
00006 #include "spausdinam.h"
00007
00008 #include <chrono>
00009 #include <iomanip>
00010 #include <iostream>
00011 #include <limits>
00012
00013
00014 namespace
00015 {
00016
00017 int gautiApdorojimoPasirinkima()
00018 {
00019     std::cout << "\nKa daryti su gautais studentais?\n"
00020               << "1 - Spausdinti visus studentus\n"
00021               << "2 - Suskirstyti i vargsiukus ir protus\n"
00022               << "3 - Abu veiksmi\n";
00023
00024     while (true)
00025     {
00026         int p = 0;
00027         std::cin >> p;
00028
00029         if (!std::cin || p < 1 || p > 3)
00030         {
00031             std::cerr << "Neteisinga ivestis. Bandykite is naujo.\n";
00032             std::cin.clear();
00033             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00034             continue;
00035         }
00036
00037         return p;
00038     }
00039 }
00040
00041 void pradetiSpausdinti(const StudContainer& studis)
00042 {
00043     std::cout << "I konsle ar i faila?\n1 - Konsle\n2 - Faila\n";
00044
00045     int failas = 0;
00046     while (true)
00047     {
00048         std::cin >> failas;
00049
00050         if (!std::cin)
00051         {
00052             std::cerr << "Neteisinga ivestis. Bandykite is naujo.\n";
00053             std::cin.clear();
00054             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00055             continue;
00056         }
00057
00058         break;
00059     }
00060 }
```

```

00059     }
00060
00061     if (failas == 1)
00062     {
00063         // Naudojame get'eries - tiesioginis prieigos prie laukų nėra
00064         std::cout << std::left
00065             << std::setw(20) << "Vardas"
00066             << std::setw(20) << "Pavarde"
00067             << std::setw(20) << "Galutinis (Vid.)"
00068             << std::setw(15) << "Galutinis (Med.)" << '\n';
00069         std::cout << std::string(75, '-') << '\n';
00070
00071         for (const auto& s : studis)
00072         {
00073             std::cout << std::left
00074                 << std::setw(20) << s.getVardas()
00075                 << std::setw(20) << s.getPavarde()
00076                 << std::setw(20) << std::fixed << std::setprecision(2) << s.getgalrezVid()
00077                 << std::setw(15) << std::fixed << std::setprecision(2) << s.getgalrezMed()
00078                 << '\n';
00079         }
00080     }
00081     else if (failas == 2)
00082     {
00083         std::cout << "Iveskite pavadinima: ";
00084         std::string pav;
00085
00086         while (true)
00087         {
00088             std::cin >> pav;
00089
00090             if (!std::cin)
00091             {
00092                 std::cerr << "Neteisinga ivestis. Bandykite is naujo.\n";
00093                 std::cin.clear();
00094                 std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00095                 continue;
00096             }
00097
00098             break;
00099         }
00100
00101         spausdinam(studis, pav);
00102     }
00103     else
00104     {
00105         std::cout << "Neteisingas pasirinkimas.\n";
00106     }
00107 }
00108
00109 void laikorezultatas(double progTrukme)
00110 {
00111     std::cout << "\n--- Laiko rezultatai ---\n"
00112         << "Failo skaitymas: " << timers.skaitymas << " s\n"
00113         << "Studentu rikiavimas: " << timers.rusiavimas << " s\n"
00114         << "Studentu skirstymas: " << timers.skirstymas << " s\n"
00115         << "Failu isvedimas: " << timers.isvedimas << " s\n"
00116         << "Visos programos trukme: " << progTrukme << " s\n";
00117 }
00118
00119 }
00120
00121
00122 int main()
00123 {
00124     auto progStart = std::chrono::high_resolution_clock::now();
00125
00126     try
00127     {
00128         while (true)
00129         {
00130             nunulintiLaikus();
00131
00132             std::cout << "\nAktyvus konteineris: " << aktyvusKonteinerioPavadinimas() << '\n';
00133
00134             StudContainer studis;
00135
00136             if (!vykdytiMeniu(studis))
00137             {
00138                 break;
00139             }
00140
00141             if (studis.empty())
00142             {
00143                 std::cout << "Nera ivestu studentu.\n";
00144                 continue;
00145             }

```

```

00146
00147 //kviečia s.suskaiciuotiGalutinius() kiekvienam
00148 suskaiciuotiGalutinius(studis);
00149
00150 const int apdorojimas = gautiApdorojimoPasirinkima();
00151
00152 if (apdorojimas == 1 || apdorojimas == 3)
00153 {
00154     rikiuotiStudentus(studis, "studentus");
00155     pradetiSpausdinti(studis);
00156 }
00157
00158 if (apdorojimas == 2 || apdorojimas == 3)
00159 {
00160     const SkirstymoStrategija strategija = pasirinktiStrategija();
00161     skirstymasGrupes(studis, strategija, true, true);
00162 }
00163
00164 char gr = 'n';
00165 std::cout << "Grizti i pradzia? (y/n): ";
00166 std::cin >> gr;
00167
00168 if (!std::cin)
00169 {
00170     std::cin.clear();
00171     std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00172 }
00173
00174 if (gr != 'y' && gr != 'Y')
00175 {
00176     break;
00177 }
00178 }
00179 }
00180 catch (const std::exception& e)
00181 {
00182     std::cerr << "Klaida: " << e.what() << std::endl;
00183     return 1;
00184 }
00185
00186 auto progEnd = std::chrono::high_resolution_clock::now();
00187 laikorezultatas(std::chrono::duration<double>(progEnd - progStart).count());
00188
00189 return 0;
00190 }

```

5.13 meniu.cpp Failo Nuoroda

```

#include "menu.h"
#include "Generavimas.h"
#include "skaitymas.h"
#include "tyrimas.h"
#include <iostream>
#include <stdexcept>
#include <string>

```

Funkcijos

- bool vykdytiMenu (StudContainer &studis)
Paleidžia pagrindinį interaktyvų programos meniu.

5.13.1 Funkcijos Dokumentacija

5.13.1.1 vykdytiMenu()

```

bool vykdytiMenu (
    StudContainer & studis )

```

Paleidžia pagrindinį interaktyvų programos meniu.

Leidžia vartotojui pasirinkti vieną iš šių veiksmų:

- įvesti studentą rankiniu būdu;
- nuskaityti studentus iš failo;
- pridėti atsitiktinius studentus;
- generuoti duomenų failą;
- atlikti konteinerių greیتaveikos tyrimą;
- paleisti klasės unit testus;
- tikrinti [Zmogus](#) abstraktumą;
- baigti darbą.

Parametrai

<code>studis</code>	Studentų konteineris, į kurį pridedami / keičiami duomenys.
---------------------	---

Gražina

`true` jei reikia tęsti programą (grįžti į pagrindinį ciklą), `false` jei vartotojas pasirinko baigti darbą.

Išimtys

<code>std::runtime_error</code>	Klaidos įvesties atveju.
---------------------------------	--------------------------

Apibrėžimas failo [menu.cpp](#) eilutėje 11.

5.14 menu.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "menu.h"
00002
00003 #include "Generavimas.h"
00004 #include "skaitymas.h"
00005 #include "tyrimas.h"
00006
00007 #include <iostream>
00008 #include <stdexcept>
00009 #include <string>
00010
00011 bool vykdytiMenu(StudContainer& studis)
00012 {
00013     while (true)
00014     {
00015         std::cout << "\nPasirinkite programos eiga:\n"
00016             << "1 - Įvesti studentą ranka\n"
00017             << "2 - Skaityti iš failo ir iskart apdoroti\n"
00018             << "3 - Pridėti studentą su atsitiktiniais pazymiais (su vardu)\n"
00019             << "4 - Pridėti sugeneruotą studentą (auto vardas/pavarde)\n"
00020             << "5 - Sugeneruoti studentų failą\n"
00021             << "6 - Atlikti konteineriu ir strategiju tyrimą\n"
00022             << "7 - Baigti darbą\n";
00023
00024         int pasirinkimas = 0;
```

```

00025         std::cin » pasirinkimas;
00026
00027         if (!std::cin)
00028         {
00029             throw std::runtime_error("Bloga ivestis.");
00030         }
00031
00032         switch (pasirinkimas)
00033         {
00034             case 1:
00035             {
00036                 Studentas s;
00037                 skaitomRanka(s);
00038                 studis.push_back(s);
00039                 break;
00040             }
00041
00042             case 2:
00043             {
00044                 if (failoSkaitymas(studis))
00045                 {
00046                     return true;
00047                 }
00048                 break;
00049             }
00050
00051             case 3:
00052             {
00053                 Studentas s;
00054                 std::string v, p;
00055                 std::cout << "Iveskite varda: ";
00056                 std::cin » v;
00057                 s.setVardas(v);
00058                 std::cout << "Iveskite pavarde: ";
00059                 std::cin » p;
00060                 s.setPavarde(p);
00061                 s.parinktiAtsitiktinius();
00062                 studis.push_back(s);
00063                 break;
00064             }
00065
00066             case 4:
00067             {
00068                 static int nr = 1;
00069                 Studentas s;
00070                 s.setVardas("Vardas" + std::to_string(nr));
00071                 s.setPavarde("Pavarde" + std::to_string(nr));
00072                 nr++;
00073                 s.parinktiAtsitiktinius();
00074                 studis.push_back(s);
00075                 break;
00076             }
00077
00078             case 5:
00079             {
00080                 generuotiFaila();
00081                 break;
00082             }
00083
00084             case 6:
00085             {
00086                 vykdytiTyrima();
00087                 break;
00088             }
00089             case 7:
00090             {
00091                 return false;
00092             }
00093
00094             default:
00095                 std::cout << "Neteisingas pasirinkimas.\n";
00096                 break;
00097         }
00098     }
00099 }

```

5.15 meniu.h Failo Nuoroda

Pagrindinio programos meniu funkcijos deklaracija.

```
#include "konteineris.h"
```

Funkcijos

- bool `vykdytiMenu` (`StudContainer` & `studis`)
Paleidžia pagrindinį interaktyvų programos meniu.

5.15.1 Smulkus aprašymas

Pagrindinio programos meniu funkcijos deklaracija.

Autorius

`Studentas`

Versija

3.0

Apibrėžimas faile `menu.h`.

5.15.2 Funkcijos Dokumentacija

5.15.2.1 `vykdytiMenu()`

```
bool vykdytiMenu (
    StudContainer & studis )
```

Paleidžia pagrindinį interaktyvų programos meniu.

Leidžia vartotojui pasirinkti vieną iš šių veiksmų:

- įvesti studentą rankiniu būdu;
- nuskaityti studentus iš failo;
- pridėti atsitiktinius studentus;
- generuoti duomenų failą;
- atlikti konteinerių greیتaveikos tyrimą;
- paleisti klasės unit testus;
- tikrinti `Zmogus` abstraktumą;
- baigti darbą.

Parametrai

<code>studis</code>	Studentų konteineris, į kurį pridedami / keičiami duomenys.
---------------------	---

Gražina

`true` jei reikia tęsti programą (grįžti į pagrindinį ciklą), `false` jei vartotojas pasirinko baigti darbą.

Išimtys

<code>std::runtime_error</code>	Klaidos įvesties atveju.
---------------------------------	--------------------------

Apibrėžimas failo [menu.cpp](#) eilutėje 11.

5.16 meniu.h

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file meniu.h
00003  * @brief Pagrindinio programos meniu funkcijos deklaracija.
00004  *
00005  * @author Studentas
00006  * @version 3.0
00007  */
00008
00009 #pragma once
00010
00011 #include "konteineris.h"
00012
00013 /**
00014  * @brief Paleidžia pagrindinį interaktyvų programos meniu.
00015  *
00016  * Leidžia vartotojui pasirinkti vieną iš šių veiksmų:
00017  * - įvesti studentą rankiniu būdu;
00018  * - nuskaityti studentus iš failo;
00019  * - pridėti atsitiktinius studentus;
00020  * - generuoti duomenų failą;
00021  * - atlikti konteinerių greیتaveikos tyrimą;
00022  * - paleisti klasės unit testus;
00023  * - tikrinti Zmogus abstraktumą;
00024  * - baigti darbą.
00025  *
00026  * @param studis Studentų konteineris, į kurį pridedami / keičiami duomenys.
00027  * @return `true` jei reikia tęsti programą (grįžti į pagrindinį ciklą),
00028  *         `false` jei vartotojas pasirinko baigti darbą.
00029  * @throws std::runtime_error Klaidos įvesties atveju.
00030  */
00031
00032 bool vykdytiMenu(StudContainer& studis);
```

5.17 skaitymas.cpp Failo Nuoroda

```
#include "skaitymas.h"
#include "laikai.h"
#include <chrono>
#include <fstream>
#include <iostream>
#include <limits>
#include <sstream>
#include <stdexcept>
#include <string>
```

Funkcijos

- void `skaitomRanka` (`Studentas` &`s`)
Interaktyviai nuskaityti vieno studento duomenis iš konsolės.
- bool `failoSkaitymas` (`StudContainer` &`studis`)
Interaktyviai prašo failo pavadinimo ir nuskaityti studentus.
- bool `failoSkaitymas` (`StudContainer` &`studis`, const std::string &`pav`)
Nuskaityti studentus iš nurodyto failo.
- void `suskaiciuotiGalutinius` (`StudContainer` &`studis`)
Apskaiciuoja galutinius balus visiems studentams konteineryje.

5.17.1 Funkcijos Dokumentacija

5.17.1.1 failoSkaitymas() [1/2]

```
bool failoSkaitymas (
    StudContainer & studis )
```

Interaktyviai prašo failo pavadinimo ir nuskaityti studentus.

Pirmoji perkrovos versija – prašo vartotojo įvesti failo pavadinimą ir perduoda jį antrajai versijai.

Parametrai

<code>studis</code>	Konteineris, į kurį pridedami nuskaityti studentai.
---------------------	---

Gražina

`true` jei failas sėkmingai nuskaitytas, `false` klaidos atveju.

Apibrėžimas failo `skaitymas.cpp` eilutėje 43.

5.17.1.2 failoSkaitymas() [2/2]

```
bool failoSkaitymas (
    StudContainer & studis,
    const std::string & pav )
```

Nuskaityti studentus iš nurodyto failo.

Antrinė perkrovos versija su ekspllicitiniu failo pavadinimu. Failo formatas:

Vardas Pavardė ND1 ND2 ... NDn Egzaminas

Pirma eilutė (antraštė) ignoruojama. Matuoja nuskaitymo laiką ir išsaugo į globalų `timers` objektą.

Parametrai

<code>studis</code>	Konteineris, į kurį pridedami nuskaityti studentai.
<code>pav</code>	Failo pavadinimas.

Gražina

`true` jei failas sėkmingai nuskaitytas, `false` klaidos atveju.

Apibrėžimas failo [skaitymas.cpp](#) eilutėje 52.

5.17.1.3 skaitomRanka()

```
void skaitomRanka (
    Studentas & s )
```

Interaktyviai nuskaityti vieno studento duomenis iš konsolės.

Prašo vartotojo įvesti vardą, pavardę ir (pagal pasirinkimą) namų darbų pažymius bei egzamino rezultatą rankiniu būdu arba sugeneruoja juos atsitiktinai.

Parametrai

<code>s</code>	Studento objektas, į kurį įrašomi duomenys.
----------------	---

Apibrėžimas failo [skaitymas.cpp](#) eilutėje 12.

5.17.1.4 suskaiciuotiGalutinius()

```
void suskaiciuotiGalutinius (
    StudContainer & studis )
```

Apskaičiuoja galutinius balus visiems studentams konteineryje.

Iškviečia `Studentas::suskaiciuotiGalutini()` kiekvienam studentui.

Parametrai

<code>studis</code>	Studentų konteineris.
---------------------	-----------------------

Apibrėžimas failo [skaitymas.cpp](#) eilutėje 99.

5.18 skaitymas.cpp

[Eiti į šio failo dokumentaciją.](#)

```
00001 #include "skaitymas.h"
00002 #include "laikai.h"
00003
00004 #include <chrono>
00005 #include <fstream>
00006 #include <iostream>
00007 #include <limits>
00008 #include <sstream>
00009 #include <stdexcept>
00010 #include <string>
00011
00012 void skaitomRanka (Studentas& s)
00013 {
```

```

00014     std::string v, p;
00015
00016     std::cout << "Iveskite varda: ";
00017     std::cin >> v;
00018     s.setVardas(v);
00019
00020     std::cout << "Iveskite pavarde: ";
00021     std::cin >> p;
00022     s.setPavarde(p);
00023
00024     char c = 'y';
00025     std::cout << "Ar naudoti atsitiktinai parinktus n.d. rezultatus? (y/n): ";
00026     std::cin >> c;
00027
00028     if (c == 'n' || c == 'N')
00029     {
00030         s.readNdInteractive();
00031
00032         int egz = 0;
00033         std::cout << "Iveskite egzamino rezultatasultata (1-10): ";
00034         std::cin >> egz;
00035         s.setEgzaminas(egz);
00036     }
00037     else
00038     {
00039         s.parinktiAtsitiktinius();
00040     }
00041 }
00042
00043 bool failoSkaitymas(StudContainer& studis)
00044 {
00045     std::cout << "Iveskite failo pavadinima: ";
00046     std::string pav;
00047     std::cin >> pav;
00048     return failoSkaitymas(studis, pav);
00049 }
00050
00051
00052 bool failoSkaitymas(StudContainer& studis, const std::string& pav)
00053 {
00054     auto start = std::chrono::high_resolution_clock::now();
00055
00056     try
00057     {
00058         std::ifstream read(pav);
00059
00060         if (!read.is_open())
00061         {
00062             throw std::runtime_error("Neatidarem failo: " + pav);
00063         }
00064
00065         std::string eilute;
00066         std::getline(read, eilute);
00067
00068         while (std::getline(read, eilute))
00069         {
00070             if (eilute.empty())
00071             {
00072                 continue;
00073             }
00074
00075             // Studentas konstruktorius kviečia readStudent()
00076             std::stringstream ss(eilute);
00077             Studentas s(ss);
00078
00079             if (s.getND().empty())
00080             {
00081                 continue;
00082             }
00083
00084             studis.push_back(std::move(s));
00085         }
00086
00087         auto end = std::chrono::high_resolution_clock::now();
00088         timers.skaitymas = std::chrono::duration<double>(end - start).count();
00089
00090         return true;
00091     }
00092     catch (const std::exception& e)
00093     {
00094         std::cerr << "Klaida skaitant faila: " << e.what() << std::endl;
00095         return false;
00096     }
00097 }
00098
00099 void suskaiciuotiGalutinius(StudContainer& studis)
00100 {

```

```
00101     for (auto& s : studis)
00102     {
00103         s.suskaiciuotiGalutini();
00104     }
00105 }
```

5.19 skaitymas.h Failo Nuoroda

Studentų duomenų skaitymo funkcijų deklaracijos.

```
#include "konteineris.h"
#include <string>
```

Funkcijos

- void [skaitomRanka](#) ([Studentas](#) &s)
Interaktyviai nuskaito vieno studento duomenis iš konsolės.
- bool [failoSkaitymas](#) ([StudContainer](#) &studis)
Interaktyviai prašo failo pavadinimo ir nuskaito studentus.
- bool [failoSkaitymas](#) ([StudContainer](#) &studis, const std::string &pav)
Nuskaito studentus iš nurodyto failo.
- void [suskaiciuotiGalutinius](#) ([StudContainer](#) &studis)
Apskaiciuoja galutinius balus visiems studentams konteineryje.

5.19.1 Smulkus aprašymas

Studentų duomenų skaitymo funkcijų deklaracijos.

Pateikia funkcijas duomenų nuskaitymui rankiniu būdu arba iš failo, bei galutinių balų apskaičiavimui visiems studentams konteineryje.

Autorius

[Studentas](#)

Versija

3.0

Apibrėžimas faile [skaitymas.h](#).

5.19.2 Funkcijos Dokumentacija

5.19.2.1 failoSkaitymas() [1/2]

```
bool failoSkaitymas (
    StudContainer & studis )
```

Interaktyviai prašo failo pavadinimo ir nuskaito studentus.

Pirmoji perkrovos versija – prašo vartotojo įvesti failo pavadinimą ir perduoda jį antrajai versijai.

Parametrai

<i>studis</i>	Konteineris, į kurį pridedami nuskaityti studentai.
---------------	---

Gražina

`true` jei failas sėkmingai nuskaitytas, `false` klaidos atveju.

Apibrėžimas failo [skaitymas.cpp](#) eilutėje 43.

5.19.2.2 failoSkaitymas() [2/2]

```
bool failoSkaitymas (
    StudContainer & studis,
    const std::string & pav )
```

Nuskaityti studentus iš nurodyto failo.

Antrinė perkrovos versija su eksplisitiniu failo pavadinimu. Failo formatas:
Vardas Pavardė ND1 ND2 ... NDn Egzaminas

Pirma eilutė (antraštė) ignoruojama. Matuoja nuskaitymo laiką ir išsaugo į globalų `timers` objektą.

Parametrai

<i>studis</i>	Konteineris, į kurį pridedami nuskaityti studentai.
<i>pav</i>	Failo pavadinimas.

Gražina

`true` jei failas sėkmingai nuskaitytas, `false` klaidos atveju.

Apibrėžimas failo [skaitymas.cpp](#) eilutėje 52.

5.19.2.3 skaitomRanka()

```
void skaitomRanka (
    Studentas & s )
```

Interaktyviai nuskaityti vieno studento duomenis iš konsolės.

Prašo vartotojo įvesti vardą, pavardę ir (pagal pasirinkimą) namų darbų pažymius bei egzamino rezultatą rankiniu būdu arba sugeneruoja juos atsitiktinai.

Parametrai

<i>s</i>	Studento objektas, į kurį įrašomi duomenys.
----------	---

Apibrėžimas failo [skaitymas.cpp](#) eilutėje 12.

5.19.2.4 suskaiciuotiGalutinius()

```
void suskaiciuotiGalutinius (
    StudContainer & studis )
```

Apskaičiuoja galutinius balus visiems studentams konteineriye.

Iškviečia `Studentas::suskaiciuotiGalutini()` kiekvienam studentui.

Parametrai

<code>studis</code>	Studentų konteineris.
---------------------	-----------------------

Apibrėžimas failo `skaitymas.cpp` eilutėje 99.

5.20 skaitymas.h

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file skaitymas.h
00003  * @brief Studentų duomenų skaitymo funkcijų deklaracijos.
00004  *
00005  * Pateikia funkcijas duomenų nuskaitymui rankiniu būdu arba iš failo,
00006  * bei galutinių balų apskaičiavimui visiems studentams konteineriye.
00007  *
00008  * @author Studentas
00009  * @version 3.0
00010  */
00011
00012 #pragma once
00013
00014 #include "konteineris.h"
00015
00016 #include <string>
00017
00018 /**
00019  * @brief Interaktyviai nuskaityti vieno studento duomenis iš konsolės.
00020  *
00021  * Prašo vartotojo įvesti vardą, pavardę ir (pagal pasirinkimą)
00022  * namų darbų pažymius bei egzamino rezultatą rankiniu būdu arba
00023  * sugeneruoja juos atsitiktinai.
00024  *
00025  * @param s Studento objektas, į kurį įrašomi duomenys.
00026  */
00027
00028 void skaitomRanka(Studentas& s);
00029
00030 /**
00031  * @brief Interaktyviai prašo failo pavadinimo ir nuskaityti studentus.
00032  *
00033  * Pirmoji perkrovos versija - prašo vartotojo įvesti failo pavadinimą
00034  * ir perduoda jį antrajai versijai.
00035  *
00036  * @param studis Konteineris, į kurį pridedami nuskaityti studentai.
00037  * @return 'true' jei failas sėkmingai nuskaitytas, 'false' klaidos atveju.
00038  */
00039
00040 bool failoSkaitymas(StudContainer& studis);
00041
00042 /**
00043  * @brief Nuskaityti studentus iš nurodyto failo.
00044  *
00045  * Antrąją perkrovos versiją su eksplisitinu failo pavadinimu.
00046  * Failo formatas:
00047  * ```
00048  * Vardas Pavardė ND1 ND2 ... NDn Egzaminas
00049  * ```
00050  * Pirmą eilutę (antraštę) ignoruojama.
00051  * Matuoja nuskaitymo laiką ir išsaugo į globalų `timers` objektą.
00052  *
00053  * @param studis Konteineris, į kurį pridedami nuskaityti studentai.
00054  * @param pav Failo pavadinimas.
```

```

00055 * @return `true` jei failas sėkmingai nuskaitytas, `false` klaidos atveju.
00056 */
00057
00058 bool failoSkaitymas(StudContainer& studis, const std::string& pav);
00059
00060 /**
00061 * @brief Apskaičiuoja galutinius balus visiems studentams konteineryje.
00062 *
00063 * Iškviečia `Studentas::suskaiciuotiGalutini()` kiekvienam studentui.
00064 *
00065 * @param studis Studentų konteineris.
00066 */
00067
00068 void suskaiciuotiGalutinius(StudContainer& studis);

```

5.21 spartos_analize.cpp Failo Nuoroda

Spartos analizė: `std::vector` vs `Vector<T>`

```

#include "Vector.h"
#include <chrono>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

```

Klasės

- struct [PushBackResult](#)
- struct [ReallocResult](#)

Tipų apibrėžimai

- using [Clock](#) = `std::chrono::high_resolution_clock`
- using [Seconds](#) = `std::chrono::duration< double >`

Funkcijos

- double [elapsed](#) (const `Clock::time_point` &start)
- [PushBackResult](#) [benchPushBack](#) (std::size_t n)
- void [pushBackBenchmark](#) ()
- [ReallocResult](#) [countReallocations](#) (std::size_t n)
- void [reallocBenchmark](#) ()
- void [otherOpsBenchmark](#) ()
- void [correctnessCheck](#) ()
- int [main](#) ()

5.21.1 Smulkus aprašymas

Spartos analizė: `std::vector` vs `Vector<T>`

Kompiliavimas: `g++ -std=c++17 -O2 -I./src spartos_analize.cpp -o spartos_analize`

Paleidimas: `./spartos_analize`

Apibrėžimas faile [spartos_analize.cpp](#).

5.21.2 Tipų apibrėžimų Dokumentacija

5.21.2.1 Clock

```
using Clock = std::chrono::high_resolution_clock
```

Apibrėžimas failo [spartos_analize.cpp](#) eilutėje 20.

5.21.2.2 Seconds

```
using Seconds = std::chrono::duration<double>
```

Apibrėžimas failo [spartos_analize.cpp](#) eilutėje 21.

5.21.3 Funkcijos Dokumentacija

5.21.3.1 benchPushBack()

```
PushBackResult benchPushBack (
    std::size_t n )
```

Apibrėžimas failo [spartos_analize.cpp](#) eilutėje 37.

5.21.3.2 correctnessCheck()

```
void correctnessCheck ( )
```

Apibrėžimas failo [spartos_analize.cpp](#) eilutėje 245.

5.21.3.3 countReallocations()

```
ReallocResult countReallocations (
    std::size_t n )
```

Apibrėžimas failo [spartos_analize.cpp](#) eilutėje 99.

5.21.3.4 elapsed()

```
double elapsed (
    const Clock::time_point & start ) [inline]
```

Apibrėžimas failo [spartos_analize.cpp](#) eilutėje 23.

5.21.3.5 main()

```
int main ( )
```

Apibrėžimas failo [spartos_analyze.cpp](#) eilutėje 293.

5.21.3.6 otherOpsBenchmark()

```
void otherOpsBenchmark ( )
```

Apibrėžimas failo [spartos_analyze.cpp](#) eilutėje 164.

5.21.3.7 pushBackBenchmark()

```
void pushBackBenchmark ( )
```

Apibrėžimas failo [spartos_analyze.cpp](#) eilutėje 63.

5.21.3.8 reallocBenchmark()

```
void reallocBenchmark ( )
```

Apibrėžimas failo [spartos_analyze.cpp](#) eilutėje 139.

5.22 spartos_analyze.cpp

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file spartos_analyze.cpp
00003  * @brief Spartos analizė: std::vector vs Vector<T>
00004  *
00005  * Kompiliavimas:
00006  * g++ -std=c++17 -O2 -I../src spartos_analyze.cpp -o spartos_analyze
00007  *
00008  * Paleidimas:
00009  * ./spartos_analyze
00010  */
00011
00012 #include "Vector.h"
00013
00014 #include <chrono>
00015 #include <iomanip>
00016 #include <iostream>
00017 #include <string>
00018 #include <vector>
00019
00020 using Clock = std::chrono::high_resolution_clock;
00021 using Seconds = std::chrono::duration<double>;
00022
00023 inline double elapsed(const Clock::time_point& start)
00024 {
00025     return Seconds(Clock::now() - start).count();
00026 }
00027
00028 //1. push_back spartos testas
00029
00030 struct PushBackResult
00031 {
00032     std::size_t n;
00033     double stdTime;
00034     double myTime;
00035 };
```



```

00036
00037 PushBackResult benchPushBack(std::size_t n)
00038 {
00039     PushBackResult r;
00040     r.n = n;
00041
00042     // std::vector
00043     {
00044         auto t = Clock::now();
00045         std::vector<int> v;
00046         for (int i = 1; i <= static_cast<int>(n); ++i)
00047             v.push_back(i);
00048         r.stdTime = elapsed(t);
00049     }
00050
00051     // Vector<T>
00052     {
00053         auto t = Clock::now();
00054         Vector<int> v;
00055         for (int i = 1; i <= static_cast<int>(n); ++i)
00056             v.push_back(i);
00057         r.myTime = elapsed(t);
00058     }
00059
00060     return r;
00061 }
00062
00063 void pushBackBenchmark()
00064 {
00065     const std::size_t sizes[] = {
00066         10'000, 100'000, 1'000'000, 10'000'000, 100'000'000
00067     };
00068
00069     std::cout << "\n=== 1. push_back() spartos palyginimas ===\n\n";
00070     std::cout << std::left
00071         << std::setw(15) << "Elementu sk."
00072         << std::setw(18) << "std::vector (s)"
00073         << std::setw(18) << "Vector<T> (s)"
00074         << "Santykis (my/std)\n";
00075     std::cout << std::string(70, '-') << '\n';
00076
00077     for (auto n : sizes)
00078     {
00079         auto res = benchPushBack(n);
00080         const double ratio = res.myTime / res.stdTime;
00081
00082         std::cout << std::left
00083             << std::setw(15) << n
00084             << std::setw(18) << std::fixed << std::setprecision(6) << res.stdTime
00085             << std::setw(18) << std::fixed << std::setprecision(6) << res.myTime
00086             << std::fixed << std::setprecision(3) << ratio << '\n';
00087     }
00088 }
00089
00090 //2. Perskirstymų skaičius
00091
00092 struct ReallocResult
00093 {
00094     std::size_t n;
00095     int stdReallocations;
00096     int myReallocations;
00097 };
00098
00099 ReallocResult countReallocations(std::size_t n)
00100 {
00101     ReallocResult r;
00102     r.n = n;
00103
00104     // std::vector
00105     {
00106         std::vector<int> v;
00107         r.stdReallocations = 0;
00108         std::size_t prevCap = v.capacity();
00109         for (int i = 1; i <= static_cast<int>(n); ++i)
00110         {
00111             v.push_back(i);
00112             if (v.capacity() != prevCap)
00113             {
00114                 prevCap = v.capacity();
00115                 ++r.stdReallocations;
00116             }
00117         }
00118     }
00119
00120     // Vector<T>
00121     {
00122         Vector<int> v;

```

```

00123         r.myReallocations = 0;
00124         std::size_t prevCap = v.capacity();
00125         for (int i = 1; i <= static_cast<int>(n); ++i)
00126         {
00127             v.push_back(i);
00128             if (v.capacity() != prevCap)
00129             {
00130                 prevCap = v.capacity();
00131                 ++r.myReallocations;
00132             }
00133         }
00134     }
00135 }
00136 return r;
00137 }
00138
00139 void reallocBenchmark()
00140 {
00141     std::cout << "\n=== 2. Atminties perskirstymu skaičius ===\n\n";
00142     std::cout << std::left
00143         << std::setw(15) << "Elementu sk."
00144         << std::setw(22) << "std::vector perskirst."
00145         << "Vector<T> perskirst.\n";
00146     std::cout << std::string(60, '-') << '\n';
00147
00148     const std::size_t sizes[] = {
00149         100'000, 1'000'000, 10'000'000, 100'000'000
00150     };
00151
00152     for (auto n : sizes)
00153     {
00154         auto res = countReallocations(n);
00155         std::cout << std::left
00156             << std::setw(15) << n
00157             << std::setw(22) << res.stdReallocations
00158             << res.myReallocations << '\n';
00159     }
00160 }
00161
00162 // 3. Kitos operacijos (insert, erase, reserve)
00163
00164 void otherOpsBenchmark()
00165 {
00166     const int N = 1'000'000;
00167
00168     std::cout << "\n=== 3. Kitos operacijos (N=" << N << ") ===\n\n";
00169
00170     // --- reserve ---
00171     double stdT, myT;
00172     {
00173         auto t = Clock::now();
00174         std::vector<int> v;
00175         v.reserve(N);
00176         for (int i = 0; i < N; ++i)
00177             v.push_back(i);
00178         stdT = elapsed(t);
00179     }
00180     {
00181         auto t = Clock::now();
00182         Vector<int> v;
00183         v.reserve(N);
00184         for (int i = 0; i < N; ++i)
00185             v.push_back(i);
00186         myT = elapsed(t);
00187     }
00188     std::cout << std::left
00189         << std::setw(30) << "reserve(N) + push_back:"
00190         << "std=" << std::fixed << std::setprecision(6) << stdT
00191         << "s my=" << std::fixed << std::setprecision(6) << myT << "s\n";
00192
00193     // --- erase (priekis) ---
00194     const int kiekis = 10000;
00195     {
00196         std::vector<int> v(kiekis);
00197         for (int i = 0; i < v.size(); ++i)
00198         {
00199             v[i] = i;
00200         }
00201         auto t = Clock::now();
00202         while (!v.empty())
00203             v.erase(v.begin());
00204         stdT = elapsed(t);
00205     }
00206     {
00207         Vector<int> v(kiekis);
00208         for (int i = 0; i < v.size(); ++i)
00209     {

```

```

00210         v[i] = i;
00211     }
00212     auto t = Clock::now();
00213     while (!v.empty())
00214         v.erase(v.begin());
00215     myT = elapsed(t);
00216 }
00217 std::cout << std::left
00218     << std::setw(30) << "erase(begin()) x" + std::to_string(kiekis) + ":"
00219     << "std=" << std::fixed << std::setprecision(6) << stdT
00220     << "s my=" << std::fixed << std::setprecision(6) << myT << "s\n";
00221
00222 // --- shrink_to_fit ---
00223 {
00224     std::vector<int> v(N);
00225     v.resize(10);
00226     auto t = Clock::now();
00227     v.shrink_to_fit();
00228     stdT = elapsed(t);
00229 }
00230 {
00231     Vector<int> v(N);
00232     v.resize(10);
00233     auto t = Clock::now();
00234     v.shrink_to_fit();
00235     myT = elapsed(t);
00236 }
00237 std::cout << std::left
00238     << std::setw(30) << "shrink_to_fit (N->10):"
00239     << "std=" << std::fixed << std::setprecision(6) << stdT
00240     << "s my=" << std::fixed << std::setprecision(6) << myT << "s\n";
00241 }
00242
00243 //4. Paprastas korektiškumo patikrinimas
00244
00245 void correctnessCheck()
00246 {
00247     std::cout << "\n== 4. Greitas korektiškumo patikrinimas ==\n";
00248
00249     bool ok = true;
00250
00251     // push_back + at
00252     Vector<int> v;
00253     for (int i = 0; i < 1000; ++i)
00254         v.push_back(i * 2);
00255     for (int i = 0; i < 1000; ++i)
00256         if (v.at(i) != i * 2) { ok = false; break; }
00257     std::cout << (ok ? " [OK] " : " [FAIL] ")
00258         << "push_back + at\n";
00259
00260     // insert viduryje
00261     Vector<int> v2 = {1, 3};
00262     v2.insert(v2.begin() + 1, 2);
00263     ok = (v2[0] == 1 && v2[1] == 2 && v2[2] == 3);
00264     std::cout << (ok ? " [OK] " : " [FAIL] ")
00265         << "insert(begin+1, value)\n";
00266
00267     // erase
00268     Vector<int> v3 = {10, 20, 30, 40};
00269     v3.erase(v3.begin() + 1);
00270     ok = (v3.size() == 3 && v3[1] == 30);
00271     std::cout << (ok ? " [OK] " : " [FAIL] ")
00272         << "erase single\n";
00273
00274     // swap
00275     Vector<int> a = {1, 2}, b = {9, 8, 7};
00276     a.swap(b);
00277     ok = (a.size() == 3 && b.size() == 2);
00278     std::cout << (ok ? " [OK] " : " [FAIL] ")
00279         << "swap\n";
00280
00281     // erase(value) non-member
00282     Vector<int> v4 = {1, 2, 3, 2, 4};
00283     erase(v4, 2);
00284     ok = (v4.size() == 3 && v4[1] == 3);
00285     std::cout << (ok ? " [OK] " : " [FAIL] ")
00286         << "non-member erase(value)\n";
00287
00288     std::cout << '\n';
00289 }
00290
00291
00292
00293 int main()
00294 {
00295     std::cout << "=====\n";
00296     std::cout << "          Vector<T> vs std::vector - Spartos analyze v3.0\n";

```

```

00297     std::cout << "=====\n";
00298
00299     correctnessCheck();
00300     pushBackBenchmark();
00301     reallocBenchmark();
00302     otherOpsBenchmark();
00303
00304     std::cout << "\n[Baigta]\n";
00305     return 0;
00306 }

```

5.23 spausdinam.cpp Failo Nuoroda

```

#include "spausdinam.h"
#include "laikai.h"
#include <chrono>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <stdexcept>

```

Funkcijos

- void `surikiuotiStudentus` (`StudContainer` &`studis`, int pasirinkimas)
Rikiuoja studentus pagal nurodytą kriterijų (int kodas).
- void `surikiuotiPagalGalutiniVid` (`StudContainer` &`studis`)
Rikiuoja studentus pagal galutinį vidurkio balą (didėjančia tvarka).
- void `rikiuotiStudentus` (`StudContainer` &`studis`, const std::string &`target`)
Interaktyviai prašo rikiavimo kriterijaus ir rikiuoja studentus.
- void `spausdinam` (const `StudContainer` &`studis`, const std::string &`pav`)
Išveda studentų sąrašą į nurodytą failą.

5.23.1 Funkcijos Dokumentacija

5.23.1.1 rikiuotiStudentus()

```

void rikiuotiStudentus (
    StudContainer & studis,
    const std::string & target )

```

Interaktyviai prašo rikiavimo kriterijaus ir rikiuoja studentus.

Išveda meniu į konsolę, leidžia pasirinkti rikiavimą pagal: vardą, pavardę, galutinį vidurkio arba medianos balą.

Parametrai

<i>studis</i>	Rikiuojamas konteineris.
<i>target</i>	Rodomas tekste vardas (pvz. "studentus", "vargsiukus").

Išimtys

<code>std::runtime_error</code>	Jei vartotojo įvestis neteisinga.
---------------------------------	-----------------------------------

Apibrėžimas failo [spausdinam.cpp](#) eilutėje 55.

5.23.1.2 spausdinam()

```
void spausdinam (
    const StudContainer & studis,
    const std::string & pav )
```

Išveda studentų sąrašą į nurodytą failą.

Rašo lentelę su kolonėlėmis: Vardas, Pavardė, Galutinis(Vid.), Galutinis(Med.).

Parametrai

<code>studis</code>	Studentų konteineris.
<code>pav</code>	Išvesties failo pavadinimas.

Apibrėžimas failo [spausdinam.cpp](#) eilutėje 76.

5.23.1.3 surikiuotiPagalGalutiniVid()

```
void surikiuotiPagalGalutiniVid (
    StudContainer & studis )
```

Rikiuoja studentus pagal galutinį vidurkio balą (didėjančia tvarka).

Patogiausia funkcija tyrimui - nenaudoja interaktyvaus meniu.

Parametrai

<code>studis</code>	Rikiuojamas konteineris.
---------------------	--------------------------

Apibrėžimas failo [spausdinam.cpp](#) eilutėje 50.

5.23.1.4 surikiuotiStudentus()

```
void surikiuotiStudentus (
    StudContainer & studis,
    int pasirinkimas )
```

Rikiuoja studentus pagal nurodytą kriterijų (int kodas).

Parametrai

<i>studis</i>	Rikiuojamas konteineris.
<i>pasirinkimas</i>	Rikiavimo kriterijus: 1–vardas, 2–pavardė, 3–Vid., 4–Med.

Išimtys

<i>std::runtime_error</i>	Jei pasirinkimas nėra 1–4.
---------------------------	----------------------------

Apibrėžimas failo [spausdinam.cpp](#) eilutėje 10.

5.24 spausdinam.cpp

Eiti į šio failo dokumentaciją.

```

00001 #include "spausdinam.h"
00002 #include "laikai.h"
00003
00004 #include <chrono>
00005 #include <fstream>
00006 #include <iomanip>
00007 #include <iostream>
00008 #include <stdexcept>
00009
00010 void surikiuotiStudentus(StudContainer& studis, int pasirinkimas)
00011 {
00012     if (studis.empty())
00013     {
00014         timers.rusiavimas = 0.0;
00015         return;
00016     }
00017
00018     auto start = std::chrono::high_resolution_clock::now();
00019
00020     switch (pasirinkimas)
00021     {
00022     case 1:
00023         konteinerioSort(studis,
00024             [](const Studentas& a, const Studentas& b)
00025             { return a.getVardas() < b.getVardas(); });
00026         break;
00027     case 2:
00028         konteinerioSort(studis,
00029             [](const Studentas& a, const Studentas& b)
00030             { return a.getPavarde() < b.getPavarde(); });
00031         break;
00032     case 3:
00033         konteinerioSort(studis,
00034             [](const Studentas& a, const Studentas& b)
00035             { return a.getgalrezVid() < b.getgalrezVid(); });
00036         break;
00037     case 4:
00038         konteinerioSort(studis,
00039             [](const Studentas& a, const Studentas& b)
00040             { return a.getgalrezMed() < b.getgalrezMed(); });
00041         break;
00042     default:
00043         throw std::runtime_error("Neteisingas rusiavimo pasirinkimas.");
00044     }
00045
00046     auto end = std::chrono::high_resolution_clock::now();
00047     timers.rusiavimas = std::chrono::duration<double>(end - start).count();
00048 }
00049
00050 void surikiuotiPagalGalutiniVid(StudContainer& studis)
00051 {
00052     surikiuotiStudentus(studis, 3);
00053 }
00054
00055 void rikiuotiStudentus(StudContainer& studis, const std::string& target)
00056 {
00057     if (studis.empty())
00058     {

```

```

00059         return;
00060     }
00061
00062     std::cout << "\nPagal ka surikiuoti " << target << "?\n";
00063     std::cout << "1 - Varda\n2 - Pavarde\n3 - Galutinis (Vid.)\n4 - Galutinis (Med.)\n";
00064
00065     int p = 0;
00066     std::cin >> p;
00067
00068     if (!std::cin || p < 1 || p > 4)
00069     {
00070         throw std::runtime_error("Neteisingas rusiavimo pasirinkimas.");
00071     }
00072
00073     surikiuotiStudentus(studis, p);
00074 }
00075
00076 void spausdinam(const StudContainer& studis, const std::string& pav)
00077 {
00078     std::ofstream write(pav);
00079
00080     if (!write.is_open())
00081     {
00082         std::cerr << "Nepavyko atidaryti failo: " << pav << std::endl;
00083         return;
00084     }
00085
00086     write << std::left
00087         << std::setw(20) << "Vardas"
00088         << std::setw(20) << "Pavarde"
00089         << std::setw(20) << "Galutinis(Vid.)"
00090         << std::setw(15) << "Galutinis(Med.)" << '\n';
00091     write << std::string(75, '-') << '\n';
00092
00093     for (const auto& s : studis)
00094     {
00095         write << std::left
00096             << std::setw(20) << s.getVardas()
00097             << std::setw(20) << s.getPavarde()
00098             << std::setw(20) << std::fixed << std::setprecision(2) << s.getgalrezVid()
00099             << std::setw(15) << std::fixed << std::setprecision(2) << s.getgalrezVid() << '\n';
00100     }
00101 }

```

5.25 spausdinam.h Failo Nuoroda

Studentų duomenų rikiavimo ir išvedimo funkcijų deklaracijos.

```

#include "konteineris.h"
#include <string>

```

Funkcijos

- void `spausdinam` (const `StudContainer` &studis, const std::string &pav)
Išveda studentų sąrašą į nurodytą failą.
- void `rikiuotiStudentus` (`StudContainer` &studis, const std::string &target)
Interaktyviai prašo rikiavimo kriterijaus ir rikiuoja studentus.
- void `surikiuotiStudentus` (`StudContainer` &studis, int pasirinkimas)
Rikiuoja studentus pagal nurodytą kriterijų (int kodas).
- void `surikiuotiPagalGalutiniVid` (`StudContainer` &studis)
Rikiuoja studentus pagal galutinį vidurkio balą (didėjančia tvarka).

5.25.1 Smulkus aprašymas

Studentų duomenų rikiavimo ir išvedimo funkcijų deklaracijos.

Pateikia funkcijas studentų rikiavimui pagal įvairius kriterijus ir duomenų išvedimui į failą.

Autorius

Studentas

Versija

3.0

Apibrėžimas faile [spausdinam.h](#).

5.25.2 Funkcijos Dokumentacija

5.25.2.1 rikiuotiStudentus()

```
void rikiuotiStudentus (
    StudContainer & studis,
    const std::string & target )
```

Interaktyviai prašo rikiavimo kriterijaus ir rikiuoja studentus.

Išveda meniu į konsolę, leidžia pasirinkti rikiavimą pagal: vardą, pavardę, galutinį vidurkio arba medianos balą.

Parametrai

<i>studis</i>	Rikiuojamas konteineris.
<i>target</i>	Rodomas tekste vardas (pvz. "studentus", "vargsiukus").

Išimtys

<i>std::runtime_error</i>	Jei vartotojo įvestis neteisinga.
---------------------------	-----------------------------------

Apibrėžimas failo [spausdinam.cpp](#) eilutėje 55.

5.25.2.2 spausdinam()

```
void spausdinam (
    const StudContainer & studis,
    const std::string & pav )
```

Išveda studentų sąrašą į nurodytą failą.

Rašo lentelę su kolonėlėmis: Vardas, Pavardė, Galutinis(Vid.), Galutinis(Med.).

Parametrai

<i>studis</i>	Studentų konteineris.
<i>pav</i>	Išvesties failo pavadinimas.

Apibrėžimas failo [spausdinam.cpp](#) eilutėje 76.

5.25.2.3 surikiuotiPagalGalutiniVid()

```
void surikiuotiPagalGalutiniVid (
    StudContainer & studis )
```

Rikiuoja studentus pagal galutinį vidurkio balą (didėjančia tvarka).

Patogiausia funkcija tyrimui - nenaudoja interaktyvaus meniu.

Parametrai

<i>studis</i>	Rikiuojamas konteineris.
---------------	--------------------------

Apibrėžimas failo [spausdinam.cpp](#) eilutėje 50.

5.25.2.4 surikiuotiStudentus()

```
void surikiuotiStudentus (
    StudContainer & studis,
    int pasirinkimas )
```

Rikiuoja studentus pagal nurodytą kriterijų (int kodas).

Parametrai

<i>studis</i>	Rikiuojamas konteineris.
<i>pasirinkimas</i>	Rikiavimo kriterijus: 1–vardas, 2–pavardė, 3–Vid., 4–Med.

Išimtys

<i>std::runtime_error</i>	Jei pasirinkimas nėra 1–4.
---------------------------	----------------------------

Apibrėžimas failo [spausdinam.cpp](#) eilutėje 10.

5.26 spausdinam.h

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file spausdinam.h
```

```

00003  * @brief Studentų duomenų rikiavimo ir išvedimo funkcijų deklaracijos.
00004  *
00005  * Pateikia funkcijas studentų rikiavimui pagal įvairius kriterijus
00006  * ir duomenų išvedimui į failą.
00007  *
00008  * @author Studentas
00009  * @version 3.0
00010  */
00011
00012 #pragma once
00013
00014 #include "konteineris.h"
00015
00016 #include <string>
00017
00018 /**
00019  * @brief Išveda studentų sąrašą į nurodytą failą.
00020  *
00021  * Rašo lentelę su kolonėlėmis: Vardas, Pavardė, Galutinis(Vid.), Galutinis(Med.).
00022  *
00023  * @param studis Studentų konteineris.
00024  * @param pav Išvesties failo pavadinimas.
00025  */
00026
00027 void spausdinam(const StudContainer& studis, const std::string& pav);
00028
00029 /**
00030  * @brief Interaktyviai prašo rikiavimo kriterijaus ir rikiuoja studentus.
00031  *
00032  * Išveda meniu į konsolę, leidžia pasirinkti rikiavimą pagal:
00033  * vardą, pavardę, galutinį vidurkio arba medianos balą.
00034  *
00035  * @param studis Rikiuojamas konteineris.
00036  * @param target Rodomas tekste vardas (pvz. `"studentus"`, `"vargsiukus"`).
00037  * @throws std::runtime_error Jei vartotojo įvestis neteisinga.
00038  */
00039
00040 void rikiuotiStudentus(StudContainer& studis, const std::string& target);
00041
00042 /**
00043  * @brief Rikiuoja studentus pagal nurodytą kriterijų (int kodas).
00044  *
00045  * @param studis Rikiuojamas konteineris.
00046  * @param pasirinkimas Rikiavimo kriterijus: 1-vardas, 2-pavardė, 3-Vid., 4-Med.
00047  * @throws std::runtime_error Jei `pasirinkimas` nėra 1-4.
00048  */
00049
00050 void surikiuotiStudentus(StudContainer& studis, int pasirinkimas);
00051
00052 /**
00053  * @brief Rikiuoja studentus pagal galutinį vidurkio balą (didėjančia tvarka).
00054  *
00055  * Patogiausia funkcija tyrimui - nenaudoja interaktyvaus meniu.
00056  *
00057  * @param studis Rikiuojamas konteineris.
00058  */
00059
00060 void surikiuotiPagalGalutiniVid(StudContainer& studis);

```

5.27 Studentas.cpp Failo Nuoroda

[Studentas](#) klasės metodų realizacija.

```

#include "Studentas.h"
#include <iomanip>
#include <algorithm>
#include <iostream>
#include <random>
#include <stdexcept>
#include <utility>
#include <sstream>

```

5.27.1 Smulkus aprašymas

[Studentas](#) klasės metodų realizacija.

Realizuojami visi Rule of Five metodai, įvesties/išvesties operatoriai, galutinio balo skaičiavimas ir pagalbinės lyginimo funkcijos.

Autorius

[Studentas](#)

Versija

3.0

Apibrėžimas faile [Studentas.cpp](#).

5.27.2 Funkcijos Dokumentacija

5.27.2.1 comparePagalGalMed()

```
bool comparePagalGalMed (  
    const Studentas & a,  
    const Studentas & b ) [related]
```

Rikiavimas pagal galutinį medianos balą (didėjančiai).

Apibrėžimas failo [Studentas.cpp](#) eilutėje 352.

5.27.2.2 comparePagalGalVid()

```
bool comparePagalGalVid (  
    const Studentas & a,  
    const Studentas & b ) [related]
```

Rikiavimas pagal galutinį vidurkio balą (didėjančiai).

Apibrėžimas failo [Studentas.cpp](#) eilutėje 346.

5.27.2.3 comparePagalPavarde()

```
bool comparePagalPavarde (  
    const Studentas & a,  
    const Studentas & b ) [related]
```

Rikiavimas pagal pavardę (didėjančiai).

Apibrėžimas failo [Studentas.cpp](#) eilutėje 340.

5.27.2.4 comparePagalVarda()

```
bool comparePagalVarda (
    const Studentas & a,
    const Studentas & b ) [related]
```

Rikiavimas pagal vardą (didėjančiai).

Apibrėžimas failo [Studentas.cpp](#) eilutėje 334.

5.27.2.5 operator<<()

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s ) [related]
```

Išvesties operatorius.

Delegacija į [Studentas::spausdinti\(\)](#) (virtualus dispatch).

Apibrėžimas failo [Studentas.cpp](#) eilutėje 196.

5.27.2.6 operator>>()

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s ) [related]
```

Įvesties operatorius.

Nuskaityto duomenis ir apskaičiuoja galutinį balą.

Apibrėžimas failo [Studentas.cpp](#) eilutėje 207.

5.28 Studentas.cpp

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file Studentas.cpp
00003  * @brief Studentas klasės metodų realizacija.
00004  *
00005  * Realizuojami visi Rule of Five metodai, įvesties/išvesties operatoriai,
00006  * galutinio balo skaičiavimas ir pagalbinės lyginimo funkcijos.
00007  *
00008  * @author Studentas
00009  * @version 3.0
00010  */
00011
00012 #include "Studentas.h"
00013
00014 #include <iomanip>
00015 #include <algorithm>
00016 #include <iostream>
00017 #include <random>
00018 #include <stdexcept>
00019 #include <utility>
00020 #include <sstream>
00021
00022
00023 /** @brief Numatytasis konstruktorius. Inicializuoja skaičinius laukus į 0. */
```

```

00024 Studentas::Studentas() : egzaminas(0), vidurkis(0.0f), galrezVid(0.0f), galrezMed(0.0f)
00025 {
00026 }
00027
00028 /**
00029  * @brief Srautinis konstruktorius.
00030  * @details Nuskaito eilutę iš srauto per `readStudent()`.
00031  * @param is Įvesties srautas (pvz. `std::istringstream`).
00032  */
00033
00034 Studentas::Studentas(std::istream& is): egzaminas(0), vidurkis(0.0f), galrezVid(0.0f), galrezMed(0.0f)
00035 {
00036     readStudent(is);
00037 }
00038
00039 /**
00040  * @brief Kopijavimo konstruktorius. Sukuria gilia kopiją.
00041  * @param other Priskiriamas objektas.
00042  */
00043
00044 Studentas::Studentas(const Studentas& other): Zmogus(other),
00045     egzaminas(other.egzaminas),
00046     nd(other.nd),
00047     vidurkis(other.vidurkis),
00048     galrezVid(other.galrezVid),
00049     galrezMed(other.galrezMed)
00050 {
00051 }
00052
00053 /**
00054  * @brief Kopijavimo priskyrimo operatorius.
00055  * @details Naudoja savipriskyrimo apsaugą ('if (this == &other)').
00056  * @param other Priskiriamas objektas.
00057  * @return Nuoroda į šį objektą.
00058  */
00059
00060 Studentas& Studentas::operator=(const Studentas& other)
00061 {
00062     if (this == &other)
00063     {
00064         return *this;
00065     }
00066
00067     Zmogus::operator=(other); //vardas, pavarde
00068     egzaminas = other.egzaminas;
00069     nd = other.nd;
00070     vidurkis = other.vidurkis;
00071     galrezVid = other.galrezVid;
00072     galrezMed = other.galrezMed;
00073
00074     return *this;
00075 }
00076
00077 /**
00078  * @brief Perkėlimo konstruktorius.
00079  * @details Po perkėlimo šaltinio skaičiai nulinami,
00080  * `nd` vektorius perkeliamas.
00081  * @param other Perkeliamas objektas (rvalue).
00082  */
00083
00084 Studentas::Studentas(Studentas&& other): Zmogus(std::move(other)),
00085     egzaminas(other.egzaminas),
00086     nd(std::move(other.nd)),
00087     vidurkis(other.vidurkis),
00088     galrezVid(other.galrezVid),
00089     galrezMed(other.galrezMed)
00090 {
00091     other.egzaminas = 0;
00092     other.vidurkis = 0.0f;
00093     other.galrezVid = 0.0f;
00094     other.galrezMed = 0.0f;
00095 }
00096
00097 /**
00098  * @brief Perkėlimo priskyrimo operatorius.
00099  * @details Savipriskyrimo apsauga + resursų perkėlimas.
00100  * @param other Perkeliamas objektas (rvalue).
00101  * @return Nuoroda į šį objektą.
00102  */
00103
00104 Studentas& Studentas::operator=(Studentas&& other)
00105 {
00106     if (this == &other)
00107     {
00108         return *this;
00109     }
00110 }

```

```

00111     Zmogus::operator=(std::move(other));
00112     egzaminas = other egzaminas;
00113     nd = std::move(other.nd);
00114     vidurkis = other.vidurkis;
00115     galrezVid = other.galrezVid;
00116     galrezMed = other.galrezMed;
00117
00118     other.egzaminas = 0;
00119     other.vidurkis = 0.0f;
00120     other.galrezVid = 0.0f;
00121     other.galrezMed = 0.0f;
00122
00123     return *this;
00124 }
00125
00126
00127 /**
00128  * @brief Išveda studento duomenis į srautą.
00129  * @details Realizuoja `Zmogus::spausdinti()`.
00130  * @param os Išvesties srautas.
00131  */
00132
00133 void Studentas::spausdinti(std::ostream& os) const
00134 {
00135     os << std::left << std::setw(15) << vardas
00136         << std::setw(15) << pavarde
00137         << "Egz: " << std::setw(4) << egzaminas << "ND: [";
00138
00139     for (std::size_t i = 0; i < nd.size(); ++i)
00140     {
00141         if (i > 0) os << ' ';
00142         os << nd[i];
00143     }
00144
00145     os << ']'
00146         << std::fixed << std::setprecision(2)
00147         << " Gal.Vid: " << galrezVid
00148         << " Gal.Med: " << galrezMed;
00149 }
00150
00151 /**
00152  * @brief Apskaičiuoja galutinį įvertinimą.
00153  * @details Realizuoja `Zmogus::suskaiciuotiGalutini()`.
00154  *
00155  * Formulė:
00156  * - `galrezVid = 0.4 * vidurkis + 0.6 * egzaminas`
00157  * - `galrezMed = 0.4 * mediana + 0.6 * egzaminas`
00158  *
00159  * Mediana skaičiuojama iš surūšiuoto `nd` vektoriaus kopijos.
00160  */
00161
00162 void Studentas::suskaiciuotiGalutini()
00163 {
00164     std::vector<int> surikiuoti = nd;
00165     std::sort(surikiuoti.begin(), surikiuoti.end());
00166
00167     const int n = static_cast<int>(surikiuoti.size());
00168
00169     if (n == 0)
00170     {
00171         galrezVid = 0.0f;
00172         galrezMed = 0.0f;
00173         return;
00174     }
00175
00176     float med = 0.0f;
00177     if (n % 2 == 1)
00178     {
00179         med = static_cast<float>(surikiuoti[n / 2]);
00180     }
00181     else
00182     {
00183         med = (static_cast<float>(surikiuoti[n / 2]) +
00184             static_cast<float>(surikiuoti[n / 2 - 1])) / 2.0f;
00185     }
00186
00187     galrezVid = 0.4f * vidurkis + 0.6f * static_cast<float>(egzaminas);
00188     galrezMed = 0.4f * med + 0.6f * static_cast<float>(egzaminas);
00189 }
00190
00191 /**
00192  * @brief Išvesties operatorius.
00193  * @details Delegacija į `Studentas::spausdinti()` (virtualus dispatch).
00194  */
00195
00196 std::ostream& operator<<(std::ostream& os, const Studentas& s)
00197 {

```

```

00198     s.spausdinti(os);
00199     return os;
00200 }
00201
00202 /**
00203  * @brief Įvesties operatorius.
00204  * @details Nuskaityto duomenis ir apskaičiuoja galutinį balą.
00205  */
00206
00207 std::istream& operator>(std::istream& is, Studentas& s)
00208 {
00209     s.readStudent(is);
00210     s.suskaiciuotiGalutini();
00211     return is;
00212 }
00213
00214 /**
00215  * @brief Nuskaityto studento duomenis iš srauto (vidinė funkcija).
00216  * @details Formatas: `Vardas Pavardė ND1 ND2 ... NDn Egzaminas`
00217  * Paskutinis skaičius laikomas egzamino pažymiu.
00218  */
00219
00220 std::istream& Studentas::readStudent(std::istream& is)
00221 {
00222     std::string line;
00223     if (!std::getline(is, line) || line.empty())
00224     {
00225         return is;
00226     }
00227     std::istringstream ss(line);
00228
00229     ss >> vardas >> pavarde;
00230
00231     nd.clear();
00232     vidurkis = 0.0f;
00233     egzaminas = 0;
00234
00235     int x = 0;
00236
00237     while (ss >> x)
00238     {
00239         nd.push_back(x);
00240     }
00241
00242     if (!nd.empty())
00243     {
00244         egzaminas = nd.back();
00245         nd.pop_back();
00246
00247         for (const int p : nd)
00248         {
00249             vidurkis += static_cast<float>(p);
00250         }
00251
00252         if (!nd.empty())
00253         {
00254             vidurkis /= static_cast<float>(nd.size());
00255         }
00256     }
00257
00258     return is;
00259 }
00260
00261 /**
00262  * @brief Interaktyviai įveda namų darbų pažymius.
00263  * @throws std::runtime_error Jei neišvestas nė vienas pažymys.
00264  */
00265
00266 void Studentas::readNdInteractive()
00267 {
00268     std::cout << "Iveskite namu darbu pazymius (0 - pabaiga):" << std::endl;
00269
00270     nd.clear();
00271     vidurkis = 0.0f;
00272
00273     while (true)
00274     {
00275         int x = 0;
00276         std::cin >> x;
00277
00278         if (!std::cin)
00279         {
00280             throw std::runtime_error("Kazkas ne taip su cin");
00281         }
00282
00283         if (x <= 0)
00284         {

```

```

00285         break;
00286     }
00287
00288     if (x > 10)
00289     {
00290         std::cout << "Netinkamas skaicius. Iveskite tarp 1 ir 10." << std::endl;
00291         continue;
00292     }
00293
00294     nd.push_back(x);
00295     vidurkis += static_cast<float>(x);
00296 }
00297
00298 if (nd.empty())
00299 {
00300     throw std::runtime_error("Nepavyko nuskaityti ne vieno pazymio!");
00301 }
00302
00303 vidurkis /= static_cast<float>(nd.size());
00304 }
00305
00306 /**
00307  * @brief Sugeneruoja atsitiktinius namų darbų ir egzamino pažymius.
00308  * @details Naudoja `std::mt19937` su `std::random_device` sėkla.
00309  * Pažymiai - intervale [1, 10].
00310  */
00311
00312 void Studentas::parinktiAtsitiktinius()
00313 {
00314     std::mt19937 gen(std::random_device{}());
00315     std::uniform_int_distribution<int> dist(1, 10);
00316
00317     const int kiekND = dist(gen);
00318
00319     nd.clear();
00320     vidurkis = 0.0f;
00321
00322     for (int i = 0; i < kiekND; i++)
00323     {
00324         const int balas = dist(gen);
00325         nd.push_back(balas);
00326         vidurkis += static_cast<float>(balas);
00327     }
00328
00329     vidurkis /= static_cast<float>(nd.size());
00330     egzaminas = dist(gen);
00331 }
00332
00333 /** @brief Rikiavimas pagal vardą (didėjančiai). */
00334 bool comparePagalVarda(const Studentas& a, const Studentas& b)
00335 {
00336     return a.getVardas() < b.getVardas();
00337 }
00338
00339 /** @brief Rikiavimas pagal pavardę (didėjančiai). */
00340 bool comparePagalPavarde(const Studentas& a, const Studentas& b)
00341 {
00342     return a.getPavarde() < b.getPavarde();
00343 }
00344
00345 /** @brief Rikiavimas pagal galutinį vidurkio balą (didėjančiai). */
00346 bool comparePagalGalVid(const Studentas& a, const Studentas& b)
00347 {
00348     return a.getgalrezVid() < b.getgalrezVid();
00349 }
00350
00351 /** @brief Rikiavimas pagal galutinį medianos balą (didėjančiai). */
00352 bool comparePagalGalMed(const Studentas& a, const Studentas& b)
00353 {
00354     return a.getgalrezMed() < b.getgalrezMed();
00355 }

```

5.29 Studentas.h Failo Nuoroda

Studento klasės apibrėžimas.

```

#include "Zmogus.h"
#include <iostream>
#include <string>
#include <vector>

```


Klasės

- class [Studentas](#)

Konkreči klasė, paveldinti iš [Zmogus](#).

5.29.1 Smulkus aprašymas

Studento klasės apibrėžimas.

Realizuoja [Zmogus](#) abstrakčią klasę konkretaus studento atveju. Saugo namų darbų pažymius, egzamino pažymį ir skaičiuoja galutinius įvertinimus pagal vidurkį bei medianą.

Autorius

[Studentas](#)

Versija

3.0

Apibrėžimas faile [Studentas.h](#).

5.30 Studentas.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 /**
00002  * @file Studentas.h
00003  * @brief Studento klasės apibrėžimas.
00004  *
00005  * Realizuoja `Zmogus` abstrakčią klasę konkretaus studento atveju.
00006  * Saugo namų darbų pažymius, egzamino pažymį ir skaičiuoja galutinius
00007  * įvertinimus pagal vidurkį bei medianą.
00008  *
00009  * @author Studentas
00010  * @version 3.0
00011  */
00012
00013 #pragma once
00014 #include "Zmogus.h"
00015
00016 #include <iostream>
00017 #include <string>
00018 #include <vector>
00019
00020
00021 /**
00022  * @class Studentas
00023  * @brief Konkreti klasė, paveldinti iš Zmogus.
00024  *
00025  * Saugo ir apdoroja studento namų darbų pažymius, egzamino rezultata,
00026  * skaičiuoja galutinius įvertinimus (pagal vidurkį ir medianą).
00027  *
00028  * ### Rule of Five
00029  * Klasė realizuoja visus penkis specialiuosius metodus:
00030  * - Numatytąjį konstruktorių
00031  * - Kopijavimo konstruktorių
00032  * - Kopijavimo priskyrimo operatorių
00033  * - Perkėlimo konstruktorių
00034  * - Perkėlimo priskyrimo operatorių
00035  * - Destruktorių (paveldi iš `Zmogus`)
00036  *
00037  * ### Galutinio balo formulė
00038  * @f[
00039  * \text{galrezVid} = 0.4 \cdot \text{vidurkis} + 0.6 \cdot \text{egzaminas}
00040  * @f]
00041  * @f[
```

```

00042 * \text{galrezMed} = 0.4 \cdot \text{mediana} + 0.6 \cdot \text{egzaminas}
00043 * @f]
00044 */
00045
00046 class Studentas : public Zmogus
00047 {
00048 private:
00049     int egzaminas;
00050     std::vector<int> nd;
00051     float vidurkis;
00052     float galrezVid;
00053     float galrezMed;
00054
00055 public:
00056     /** @name Rule of Five
00057      * @{
00058      */
00059
00060     /**
00061      * @brief Numatytasis konstruktorius.
00062      *
00063      * Inicializuoja visus skaičinius laukus į 0, o vektorių - į tuščią.
00064      */
00065     Studentas(); // Default konstruktorius
00066
00067     /**
00068      * @brief Srautinis konstruktorius.
00069      *
00070      * Nuskaito studento duomenis iš nurodyto įvesties srauto
00071      * (pvz. failo eilutės pateiktos kaip `std::istream`).
00072      *
00073      * @param is Įvesties srautas.
00074      */
00075     explicit Studentas(std::istream& is); // stream konstruktorius
00076
00077     /**
00078      * @brief Kopijavimo konstruktorius.
00079      *
00080      * Sukuria gilia kopiją kito `Studentas` objekto.
00081      *
00082      * @param other Kopijuojamas objektas.
00083      */
00084     Studentas(const Studentas& other); // Kopijavimo konstruktorius
00085
00086     /**
00087      * @brief Kopijavimo priskyrimo operatorius.
00088      *
00089      * Nukopijuoja visus duomenis iš `other` į šį objektą.
00090      * Saugiai tvarko savipriskyrimo atvejį.
00091      *
00092      * @param other Priskiriamas objektas.
00093      * @return Nuoroda į šį objektą (`*this`).
00094      */
00095     Studentas& operator=(const Studentas& other); // Kopijavimo priskyrimo operatorius
00096
00097     /**
00098      * @brief Perkėlimo konstruktorius.
00099      *
00100      * Perkelia resursus iš `other` į šį objektą.
00101      * Po perkėlimo `other` lieka tuščios būsenos.
00102      *
00103      * @param other Perkeliamas objektas (rvalue nuoroda).
00104      */
00105     Studentas(Studentas&& other); // Perkėlimo konstruktorius
00106
00107     /**
00108      * @brief Perkėlimo priskyrimo operatorius.
00109      *
00110      * Perkelia resursus iš `other` į šį objektą priskyrimo metu.
00111      * Saugiai tvarko savipriskyrimo atvejį.
00112      *
00113      * @param other Perkeliamas objektas (rvalue nuoroda).
00114      * @return Nuoroda į šį objektą (`*this`).
00115      */
00116     Studentas& operator=(Studentas&& other); // Perkėlimo priskyrimo operatorius
00117
00118     /**
00119      * @brief Destruktorius.
00120      *
00121      * Paveldi iš `Zmogus`. Automatiškai atlaisvina `nd` vektoriaus atmintį.
00122      */
00123     ~Studentas() override = default; // Destruktorius
00124
00125     /**
00126      * @brief Gražina namų darbų pažymių vektorių (tik skaitymui).
00127      * @return Konstantinė nuoroda į `nd` vektorių.
00128      */

```

```

00129     inline const std::vector<int>& getND() const { return nd; }
00130
00131     /**
00132      * @brief Gražina egzamino pažymį.
00133      * @return Egzamino pažymys (1-10).
00134      */
00135     inline int getEgzaminas() const { return egzaminas; }
00136
00137     /**
00138      * @brief Gražina namų darbų pažymių vidurkį.
00139      * @return Vidurkis kaip `float`.
00140      */
00141     inline float getVidurkis() const { return vidurkis; }
00142
00143     /**
00144      * @brief Gražina galutinį balą apskaičiuotą pagal vidurkį.
00145      * @return Galutinis balas (vidurkio variantas).
00146      */
00147     inline float getgalrezVid() const { return galrezVid; }
00148
00149     /**
00150      * @brief Gražina galutinį balą apskaičiuotą pagal medianą.
00151      * @return Galutinis balas (medianos variantas).
00152      */
00153     inline float getgalrezMed() const { return galrezMed; }
00154
00155     // -----
00156     /** @name Keitimo metodai (setters)
00157      * @{
00158      */
00159
00160     /**@brief rankinio įvedimo atvejui
00161     /**
00162      * @brief Nustato studento vardą.
00163      * @param v Naujas vardas.
00164      */
00165     void setVardas(const std::string& v) { vardas = v; }
00166
00167     /**
00168      * @brief Nustato studento pavardę.
00169      * @param p Nauja pavardė.
00170      */
00171     void setPavarde(const std::string& p) { pavarde = p; }
00172
00173     /**
00174      * @brief Nustato egzamino pažymį.
00175      * @param e Egzamino pažymys (rekomenduojama 1-10).
00176      */
00177     void setEgzaminas(int e) { egzaminas = e; }
00178
00179     /** @} */
00180
00181     // -----
00182     /** @name Funkcijos
00183      * @{
00184      */
00185
00186     /**
00187      * @brief Nuskaito studento duomenis iš srauto.
00188      *
00189      * Formatas: `Vardas Pavardė ND1 ND2 ... NDn Egzaminas`
00190      * Paskutinis skaičius laikomas egzamino pažymiu.
00191      *
00192      * @param is Įvesties srautas.
00193      * @return Nuoroda į srautą (grandininiam kvietimui).
00194      */
00195     std::istream& readStudent(std::istream& is); // skaito vardas/pavardė/nd/egz iš stream
00196
00197     /**
00198      * @brief Interaktyviai įveda namų darbų pažymius iš konsolės.
00199      *
00200      * Prašo vartotojo įvesti pažymius vieną po kito.
00201      * Įvedimas baigiamas naudojant 0 arba neigiamą skaičių.
00202      *
00203      * @throws std::runtime_error Jei nepavyksta nuskaityti nė vieno pažymio.
00204      */
00205     void readNdInteractive();
00206
00207     /**
00208      * @brief Sugeneruoja atsitiktinius namų darbų ir egzamino pažymius.
00209      *
00210      * Naudoja `std::mt19937` generatorių su `std::random_device` sėkla.
00211      * Pažymiai generuojami intervale [1, 10].
00212      */
00213     void parinktiAtsitiktinius();
00214
00215     /**

```

```

00216     * @brief Apskaičiuoja galutinį įvertinimą (vidurkio ir medianos variantus).
00217     *
00218     * Realizuoja grynąją virtualią funkciją iš `Zmogus`.
00219     *
00220     * Formulė:
00221     * - galrezVid = 0.4 * vidurkis + 0.6 * egzaminas
00222     * - galrezMed = 0.4 * mediana + 0.6 * egzaminas
00223     */
00224     void suskaiciuotiGalutini() override;
00225
00226     /**
00227     * @brief Išveda studento duomenis į nurodytą srautą.
00228     *
00229     * Realizuoja grynąją virtualią funkciją iš `Zmogus`.
00230     *
00231     * @param os Išvesties srautas.
00232     */
00233     void spausdinti(std::ostream& os) const override;
00234
00235     /** @} */
00236 };
00237
00238 // -----
00239 /** @name Laisvosios funkcijos
00240 * @relates Studentas
00241 * @{
00242 */
00243
00244 /**
00245 * @brief Išvesties operatorius srautui.
00246 *
00247 * Delegacija į `Studentas::spausdinti()`.
00248 *
00249 * @param os Išvesties srautas.
00250 * @param s Studento objektas.
00251 * @return Nuoroda į srautą.
00252 */
00253 std::ostream& operator<<(std::ostream& os, const Studentas& s);
00254
00255 /**
00256 * @brief Įvesties operatorius srautui.
00257 *
00258 * Nuskaito studento duomenis ir iškart apskaičiuoja galutinį įvertinimą.
00259 *
00260 * @param is Įvesties srautas.
00261 * @param s Studento objektas, į kurį rašoma.
00262 * @return Nuoroda į srautą.
00263 */
00264 std::istream& operator>>(std::istream& is, Studentas& s);
00265
00266 /**
00267 * @brief Lyginimo funkcija rikiavimui pagal vardą.
00268 * @param a Pirmas studentas.
00269 * @param b Antras studentas.
00270 * @return `true` jei `a.vardas < b.vardas`.
00271 */
00272 bool comparePagalVarda (const Studentas& a, const Studentas& b);
00273
00274 /**
00275 * @brief Lyginimo funkcija rikiavimui pagal pavardę.
00276 * @param a Pirmas studentas.
00277 * @param b Antras studentas.
00278 * @return `true` jei `a.pavarde < b.pavarde`.
00279 */
00280 bool comparePagalPavarde (const Studentas& a, const Studentas& b);
00281
00282 /**
00283 * @brief Lyginimo funkcija rikiavimui pagal galutinį vidurkio balą.
00284 * @param a Pirmas studentas.
00285 * @param b Antras studentas.
00286 * @return `true` jei `a.galrezVid < b.galrezVid`.
00287 */
00288 bool comparePagalGalVid (const Studentas& a, const Studentas& b);
00289
00290 /**
00291 * @brief Lyginimo funkcija rikiavimui pagal galutinį medianos balą.
00292 * @param a Pirmas studentas.
00293 * @param b Antras studentas.
00294 * @return `true` jei `a.galrezMed < b.galrezMed`.
00295 */
00296 bool comparePagalGalMed (const Studentas& a, const Studentas& b);
00297 /** @} */

```

5.31 tyrimas.cpp Failo Nuoroda

```
#include "tyrimas.h"
#include "Generavimas.h"
#include "konteineris.h"
#include "laikai.h"
#include "skaitymas.h"
#include "spausdinam.h"
#include <fstream>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>
```

Klasės

- struct [Tyrimolrasas](#)

Funkcijos

- void [vykdytiTyrima](#) ()
Paleidžia interaktyvų konteinerių greitaveikos tyrimą.

5.31.1 Funkcijos Dokumentacija

5.31.1.1 vykdytiTyrima()

```
void vykdytiTyrima ( )
```

Paleidžia interaktyvų konteinerių greitaveikos tyrimą.

Vartotojas gali pasirinkti:

- skirstymo strategiją;
- kartojimų skaičių (rezultatai vidurkinami);
- testuojamą failą arba standartinį failų rinkinį (1k–10M eilučių).

Rezultatai išvedami į konsolę ir išsaugomi kaip Markdown lentelė faile `benchmark_<konteineris>_↵S<strategija>.md`.

Apibrėžimas failo [tyrimas.cpp](#) eilutėje 134.

5.32 tyrimas.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "tyrimas.h"
00002
00003 #include "Generavimas.h"
00004 #include "konteineris.h"
00005 #include "laikai.h"
00006 #include "skaitymas.h"
00007 #include "spausdinam.h"
00008
00009 #include <fstream>
00010 #include <iomanip>
00011 #include <iostream>
00012 #include <string>
00013 #include <vector>
00014
00015
00016 struct TyrimoIrasas
00017 {
00018     std::string failas;
00019     std::size_t irasuKiekis = 0;
00020     double skaitymas = 0.0;
00021     double rusiavimas = 0.0;
00022     double skirstymas = 0.0;
00023 };
00024
00025 namespace
00026 {
00027
00028 bool failasEgzistuoja(const std::string& pav)
00029 {
00030     std::ifstream f(pav);
00031     return f.good();
00032 }
00033
00034 TyrimoIrasas atliktiBandyMuSerija(
00035     const std::string& failas,
00036     SkirstymoStrategija strategija,
00037     int kartojimai)
00038 {
00039     TyrimoIrasas vidurkiai;
00040     vidurkiai.failas = failas;
00041
00042     for (int i = 0; i < kartojimai; ++i)
00043     {
00044         nunulintiLaikus();
00045
00046         StudContainer studis;
00047         if (!failoSkaitymas(studis, failas)) // naudoja Studentas() vid.
00048         {
00049             return {};
00050         }
00051
00052         suskaiciuotiGalutinius(studis);
00053         vidurkiai.irasuKiekis = studis.size();
00054
00055         surikiuotiPagalGalutiniVid(studis);
00056         skirstymasGrupes(studis, strategija, false, false);
00057
00058         vidurkiai.skaitymas += timers.skaitymas;
00059         vidurkiai.rusiavimas += timers.rusiavimas;
00060         vidurkiai.skirstymas += timers.skirstymas;
00061     }
00062
00063     vidurkiai.skaitymas /= kartojimai;
00064     vidurkiai.rusiavimas /= kartojimai;
00065     vidurkiai.skirstymas /= kartojimai;
00066
00067     return vidurkiai;
00068 }
00069
00070 void spausdintiLentele(const std::vector<TyrimoIrasas>& rezultatasultatai)
00071 {
00072     std::cout << '\n'
00073         << std::left
00074         << std::setw(22) << "Failas"
00075         << std::setw(16) << "Irasu kiekis"
00076         << std::setw(16) << "Skaitymas (s)"
00077         << std::setw(18) << "Rusiavimas (s)"
00078         << std::setw(18) << "Skirstymas (s)"
00079         << "Is viso (s)\n"
00080         << std::string(90, '-') << '\n';
00081
00082     for (const auto& r : rezultatasultatai)
```

```

00083     {
00084         const double isViso = r.skaitymas + r.rusiavimas + r.skirstymas;
00085
00086         std::cout << std::left
00087             << std::setw(22) << r.failas
00088             << std::setw(16) << r.irasuKiekis
00089             << std::setw(16) << std::fixed << std::setprecision(6) << r.skaitymas
00090             << std::setw(18) << std::fixed << std::setprecision(6) << r.rusiavimas
00091             << std::setw(18) << std::fixed << std::setprecision(6) << r.skirstymas
00092             << std::fixed << std::setprecision(6) << isViso << '\n';
00093     }
00094 }
00095
00096 void iREADME(const std::vector<TyrimoIrasas>& rezultatasultatai, SkirstymoStrategija strategija)
00097 {
00098     const std::string pav = "benchmark_" + aktyvausKonteinerioTrumpasPavadinimas() + "_S" +
00099         std::to_string(static_cast<int>(strategija)) + ".md";
00100     std::ofstream write(pav);
00101     if (!write.is_open()) { return; }
00102
00103     write << "# " << aktyvausKonteinerioPavadinimas() << " - strategija " << static_cast<int>(strategija)
00104 << "\n\n";
00105     write << "| Failas | Irasu kiekis | Skaitymas (s) | Rusiavimas (s) | Skirstymas (s) | Is viso (s)
00106 |\n";
00107     write << "|---|---|---|---|---|---|\n";
00108     for (const auto& r : rezultatasultatai)
00109     {
00110         const double isViso = r.skaitymas + r.rusiavimas + r.skirstymas;
00111         write << "| " << r.failas
00112             << " | " << r.irasuKiekis
00113             << " | " << std::fixed << std::setprecision(6) << r.skaitymas
00114             << " | " << std::fixed << std::setprecision(6) << r.rusiavimas
00115             << " | " << std::fixed << std::setprecision(6) << r.skirstymas
00116             << " | " << std::fixed << std::setprecision(6) << isViso << " |\n";
00117     }
00118     std::cout << "Rezultatu lentele issaugota: " << pav << '\n';
00119 }
00120
00121 std::vector<std::string> sudarytiNumatytujuFailuSarasa(const std::string& prefiksas)
00122 {
00123     return {
00124         prefiksas + "1000.txt",
00125         prefiksas + "10000.txt",
00126         prefiksas + "100000.txt",
00127         prefiksas + "1000000.txt",
00128         prefiksas + "10000000.txt"
00129     };
00130 }
00131
00132 }
00133
00134 void vykdytiTyrima()
00135 {
00136     std::cout << "\nAktyvus konteineris: " << aktyvausKonteinerioPavadinimas() << '\n';
00137
00138     const SkirstymoStrategija strategija = pasirinktiStrategija();
00139
00140     std::cout << "Kiek kartu kartoti kiekviena testa? ";
00141     int kartojimai = 1;
00142     std::cin >> kartojimai;
00143
00144     if (!std::cin || kartojimai < 1)
00145     {
00146         throw std::runtime_error("Neteisingas kartojimui skaicius.");
00147     }
00148
00149     std::cout << "\n1 - Testuoti viena faila\n"
00150         << "2 - Testuoti numatytuosius failus (1k, 10k, 100k, 1M, 10M)\n";
00151
00152     int pasirinkimas = 0;
00153     std::cin >> pasirinkimas;
00154
00155     if (!std::cin || pasirinkimas < 1 || pasirinkimas > 2)
00156     {
00157         throw std::runtime_error("Neteisingas tyrimo meniu pasirinkimas.");
00158     }
00159
00160     std::vector<std::string> failai;
00161
00162     if (pasirinkimas == 1)
00163     {
00164         std::cout << "Iveskite pilna failo pavadinima: ";
00165         std::string f;
00166         std::cin >> f;

```

```
00167         failai.push_back(f);
00168     }
00169     else
00170     {
00171         std::cout << "Iveskite failu prefiksa be dydzio ir .txt (pvz. studentai): ";
00172         std::string prefiksas;
00173         std::cin >> prefiksas;
00174         failai = sudarytiNumatytujuFailuSarasa(prefiksas);
00175     }
00176
00177     std::vector<TyrimoIrasas> rezultatasultatai;
00178
00179     for (const auto& failas : failai)
00180     {
00181         if (!failasEgzistuoja(failas))
00182         {
00183             std::cout << "Failas nerastas, praleidziamas: " << failas << '\n';
00184             continue;
00185         }
00186
00187         auto rezultatas = atliktiBandyMuSeriJa(failas, strategija, kartoJimai);
00188         if (rezultatas.irasuKiekis != 0)
00189         {
00190             rezultatasultatai.push_back(rezultatas);
00191         }
00192     }
00193
00194     if (rezultatasultatai.empty())
00195     {
00196         std::cout << "Tyrimui tinkamu failu nerasta.\n";
00197         return;
00198     }
00199
00200     spausdintiLentele(rezultatasultatai);
00201     iREADME(rezultatasultatai, strategija);
00202 }
```

5.33 tyrimas.h Failo Nuoroda

Konteinerių ir skirstymo strategijų greitaveikos tyrimo funkcijos.

Funkcijos

- void [vykdytiTyrima\(\)](#)
Paleidžia interaktyvų konteinerių greitaveikos tyrimą.

5.33.1 Smulkus aprašymas

Konteinerių ir skirstymo strategijų greitaveikos tyrimo funkcijos.

Atlieka laiko matavimus skaitant failus, rikiuojant ir skirstant studentus bei išveda rezultatus į konsolę ir Markdown lentelę.

Autorius

[Studentas](#)

Versija

3.0

Apibrėžimas faile [tyrimas.h](#).

5.33.2 Funkcijos Dokumentacija

5.33.2.1 vykdytiTyrima()

```
void vykdytiTyrima ( )
```

Paleidžia interaktyvų konteinerių greitaveikos tyrimą.

Vartotojas gali pasirinkti:

- skirstymo strategiją;
- kartojimų skaičių (rezultatai vidurkinami);
- testuojamą failą arba standartinį failų rinkinį (1k–10M eilučių).

Rezultatai išvedami į konsolę ir išsaugomi kaip Markdown lentelė faile `benchmark_<konteineris>_<S>S<strategija>.md`.

Apibrėžimas failo `tyrimas.cpp` eilutėje 134.

5.34 tyrimas.h

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file tyrimas.h
00003  * @brief Konteinerių ir skirstymo strategijų greitaveikos tyrimo funkcijos.
00004  *
00005  * Atlieka laiko matavimus skaitant failus, rikiuojant ir skirstant studentus
00006  * bei išveda rezultatus į konsolę ir Markdown lentelę.
00007  *
00008  * @author Studentas
00009  * @version 3.0
00010  */
00011
00012 #pragma once
00013
00014 /**
00015  * @brief Paleidžia interaktyvų konteinerių greitaveikos tyrimą.
00016  *
00017  * Vartotojas gali pasirinkti:
00018  * - skirstymo strategiją;
00019  * - kartojimų skaičių (rezultatai vidurkinami);
00020  * - testuojamą failą arba standartinį failų rinkinį (1k-10M eilučių).
00021  *
00022  * Rezultatai išvedami į konsolę ir išsaugomi kaip Markdown lentelė
00023  * faile `benchmark_<konteineris>_S<strategija>.md`.
00024  */
00025
00026 void vykdytiTyrima();
```

5.35 Vector.h Failo Nuoroda

Pilnavertė `std::vector` alternatyva – `Vector<T, Allocator>`

```
#include <algorithm>
#include <cassert>
#include <cstddef>
#include <initializer_list>
#include <iterator>
#include <limits>
#include <memory>
#include <stdexcept>
#include <type_traits>
#include <utility>
```

Klasės

- class [Vector< T, Allocator >](#)

Dinaminio masyvo konteineris – std::vector analogas.

5.35.1 Smulkus aprašymas

Pilnavertė std::vector alternatyva – Vector<T, Allocator>

Apima visus Member types, Member functions ir Non-member functions.

Autorius

v3.0

Apibrėžimas faile [Vector.h](#).

5.36 Vector.h

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file Vector.h
00003  * @brief Pilnavertė std::vector alternatyva - Vector<T, Allocator>
00004  *
00005  * Apima visus Member types, Member functions ir Non-member functions.
00006  *
00007  * @author v3.0
00008  */
00009 #pragma once
00010
00011 #include <algorithm>
00012 #include <cassert>
00013 #include <cstdint>
00014 #include <initializer_list>
00015 #include <iterator>
00016 #include <limits>
00017 #include <memory>
00018 #include <stdexcept>
00019 #include <type_traits>
00020 #include <utility>
00021
00022 /**
00023  * @brief Dinaminio masyvo konteineris - std::vector analogas.
00024  *
00025  * @tparam T          Saugomų elementų tipas.
00026  * @tparam Allocator Atminties paskirstytojas (numatytasis: std::allocator<T>).
00027  */
00028 template <typename T, typename Allocator = std::allocator<T>
00029 class Vector
00030 {
00031 public:
00032     //Member types
00033     using value_type = T;
00034     using allocator_type = Allocator;
00035     using size_type = std::size_t;
00036     using difference_type = std::ptrdiff_t;
00037     using reference = value_type&;
00038     using const_reference = const value_type&;
00039     using pointer = typename std::allocator_traits<Allocator>::pointer;
00040     using const_pointer = typename std::allocator_traits<Allocator>::const_pointer;
00041     using iterator = pointer;
00042     using const_iterator = const_pointer;
00043     using reverse_iterator = std::reverse_iterator<iterator>;
00044     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00045
00046 private:
00047     using AllocTraits = std::allocator_traits<Allocator>;
00048
00049     Allocator alloc_;
```

```

00050     pointer data_;
00051     size_type size_;
00052     size_type cap_;
00053
00054     //Vidiniai pagalbiniai metodai
00055
00056     //Sunaikina visus elementus
00057     void destroyAll() noexcept
00058     {
00059         for (size_type i = 0; i < size_; ++i)
00060         {
00061             AllocTraits::destroy(alloc_, data_ + i);
00062         }
00063     }
00064
00065     //Atpalaiduoja paskirtą atminties bloką
00066     void deallocate() noexcept
00067     {
00068         if (data_)
00069         {
00070             AllocTraits::deallocate(alloc_, data_, cap_);
00071             data_ = nullptr;
00072         }
00073     }
00074
00075     //Perskirsto atmintį į naują bloką (move semantika)
00076     void reallocate(size_type newCap)
00077     {
00078         pointer newData = AllocTraits::allocate(alloc_, newCap);
00079         size_type i = 0;
00080         try
00081         {
00082             for (; i < size_; ++i)
00083                 AllocTraits::construct(alloc_, newData + i,
00084                                         std::move_if_noexcept(data_[i]));
00085         }
00086         catch (...)
00087         {
00088             for (size_type j = 0; j < i; ++j)
00089                 AllocTraits::destroy(alloc_, newData + j);
00090             AllocTraits::deallocate(alloc_, newData, newCap);
00091             throw;
00092         }
00093         destroyAll();
00094         deallocate();
00095         data_ = newData;
00096         cap_ = newCap;
00097     }
00098
00099     //Eksponentinis augimas (x2)
00100     size_type growCap() const noexcept
00101     {
00102         return cap_ == 0 ? 1 : cap_ * 2;
00103     }
00104
00105 public:
00106     //Konstruktoriai
00107
00108     /**
00109      * @brief Numatytasis konstruktorius - sukuria tuščią vektorių.
00110      */
00111     Vector() noexcept(noexcept(Allocator())) : alloc_(0), data_(nullptr), size_(0), cap_(0) {}
00112
00113     /**
00114      * @brief Konstruktorius su paskirstytoju.
00115      */
00116     explicit Vector(const Allocator& alloc) noexcept : alloc_(alloc), data_(nullptr), size_(0),
00117 cap_(0) {}
00118
00119     /**
00120      * @brief Konstruktorius su skaičiumi ir reikšme.
00121      * @param count Elementų skaičius.
00122      * @param value Pradinė reikšmė.
00123      */
00123     Vector(size_type count, const T& value, const Allocator& alloc = Allocator()) : alloc_(alloc),
00124 data_(nullptr), size_(0), cap_(0)
00125     {
00126         if (count > 0)
00127         {
00127             data_ = AllocTraits::allocate(alloc_, count);
00128             cap_ = count;
00129             for (size_type i = 0; i < count; ++i)
00130             {
00131                 AllocTraits::construct(alloc_, data_ + i, value);
00132                 ++size_;
00133             }
00134         }
00135     }

```

```

00135     }
00136
00137     /**
00138      * @brief Konstruktorius su skaičiumi (value-initialized).
00139      * @param count Elementų skaičius.
00140      */
00141     explicit Vector(size_type count, const Allocator& alloc = Allocator()) : Vector(count, T(), alloc)
00142 {}
00143
00144     /**
00145      * @brief Diapazoninis konstruktorius (InputIterator pora).
00146      */
00147     template <typename InputIt, typename = std::enable_if_t<std::is_base_of_v< std::input_iterator_tag, typename std::iterator_traits<InputIt>::value_type>::is_arithmetic>>
00148     Vector(InputIt first, InputIt last, const Allocator& alloc = Allocator()) : alloc_(alloc),
00149     data_(nullptr), size_(0), cap_(0)
00150     {
00151         for (auto it = first; it != last; ++it)
00152             push_back(*it);
00153     }
00154
00155     /**
00156      * @brief Kopijavimo konstruktorius.
00157      */
00158     Vector(const Vector& other) :
00159     alloc_(AllocTraits::select_on_container_copy_construction(other.alloc_)), data_(nullptr), size_(0),
00160     cap_(0)
00161     {
00162         if (other.size_ > 0)
00163         {
00164             data_ = AllocTraits::allocate(alloc_, other.size_);
00165             cap_ = other.size_;
00166             for (size_type i = 0; i < other.size_; ++i)
00167             {
00168                 AllocTraits::construct(alloc_, data_ + i, other.data_[i]);
00169                 ++size_;
00170             }
00171         }
00172     }
00173
00174     /**
00175      * @brief Kopijavimo konstruktorius su paskirstytoju.
00176      */
00177     Vector(const Vector& other, const Allocator& alloc) : alloc_(alloc), data_(nullptr), size_(0),
00178     cap_(0)
00179     {
00180         if (other.size_ > 0)
00181         {
00182             data_ = AllocTraits::allocate(alloc_, other.size_);
00183             cap_ = other.size_;
00184             for (size_type i = 0; i < other.size_; ++i)
00185             {
00186                 AllocTraits::construct(alloc_, data_ + i, other.data_[i]);
00187                 ++size_;
00188             }
00189         }
00190     }
00191
00192     /**
00193      * @brief Perkėlimo konstruktorius.
00194      */
00195     Vector(Vector&& other) noexcept : alloc_(std::move(other.alloc_)), data_(other.data_),
00196     size_(other.size_), cap_(other.cap_)
00197     {
00198         other.data_ = nullptr;
00199         other.size_ = 0;
00200         other.cap_ = 0;
00201     }
00202
00203     /**
00204      * @brief Perkėlimo konstruktorius su paskirstytoju.
00205      */
00206     Vector(Vector&& other, const Allocator& alloc) noexcept : alloc_(alloc), data_(other.data_),
00207     size_(other.size_), cap_(other.cap_)
00208     {
00209         other.data_ = nullptr;
00210         other.size_ = 0;
00211         other.cap_ = 0;
00212     }
00213
00214     /**
00215      * @brief Inicializatorių sąrašo konstruktorius.
00216      */
00217     Vector(std::initializer_list<T> init, const Allocator& alloc = Allocator()) : alloc_(alloc),
00218     data_(nullptr), size_(0), cap_(0)

```

```

00213     {
00214         reserve(init.size());
00215         for (const auto& v : init)
00216         {
00217             push_back(v);
00218         }
00219     }
00220
00221     //Destruktorius
00222
00223     /**
00224     * @brief Destruktorius - atlaisvina visus resursus.
00225     */
00226     ~Vector()
00227     {
00228         destroyAll();
00229         deallocate();
00230     }
00231
00232     //Priskyrimo operatoriai
00233
00234     /**
00235     * @brief Kopijavimo priskyrimo operatorius.
00236     */
00237     Vector& operator=(const Vector& other)
00238     {
00239         if (this == &other)
00240         {
00241             return *this;
00242         }
00243         clear();
00244         if constexpr (AllocTraits::propagate_on_container_copy_assignment::value)
00245         {
00246             if (alloc_ != other.alloc_)
00247             {
00248                 deallocate();
00249                 cap_ = 0;
00250             }
00251             alloc_ = other.alloc_;
00252         }
00253         reserve(other.size_);
00254         for (size_type i = 0; i < other.size_; ++i)
00255         {
00256             AllocTraits::construct(alloc_, data_ + i, other.data_[i]);
00257             ++size_;
00258         }
00259         return *this;
00260     }
00261
00262     /**
00263     * @brief Perkëlimo priskyrimo operatorius.
00264     */
00265     Vector& operator=(Vector&& other) noexcept
00266     {
00267         if (this == &other)
00268         {
00269             return *this;
00270         }
00271         destroyAll();
00272         deallocate();
00273         if constexpr (AllocTraits::propagate_on_container_move_assignment::value)
00274             alloc_ = std::move(other.alloc_);
00275         data_ = other.data_;
00276         size_ = other.size_;
00277         cap_ = other.cap_;
00278         other.data_ = nullptr;
00279         other.size_ = 0;
00280         other.cap_ = 0;
00281         return *this;
00282     }
00283
00284     /**
00285     * @brief Inicializatorių sąrašo priskyrimo operatorius.
00286     */
00287     Vector& operator=(std::initializer_list<T> init)
00288     {
00289         clear();
00290         reserve(init.size());
00291         for (const auto& v : init)
00292         {
00293             push_back(v);
00294         }
00295         return *this;
00296     }
00297
00298     // assign
00299

```

```

00300     /**
00301     * @brief Priskiria count kopijų value reikšmės.
00302     */
00303     void assign(size_type count, const T& value)
00304     {
00305         clear();
00306         reserve(count);
00307         for (size_type i = 0; i < count; ++i)
00308         {
00309             push_back(value);
00310         }
00311     }
00312
00313     /**
00314     * @brief Priskiria diapazoną [first, last).
00315     */
00316     template <typename InputIt, typename = std::enable_if_t<std::is_base_of_v< std::input_iterator_tag, typename std::iterator_traits<InputIt>::value_type>::value_type>
00317     void assign(InputIt first, InputIt last)
00318     {
00319         clear();
00320         for (auto it = first; it != last; ++it)
00321         {
00322             push_back(*it);
00323         }
00324     }
00325
00326     /**
00327     * @brief Priskiria inicializatorių sąrašą.
00328     */
00329     void assign(std::initializer_list<T> init)
00330     {
00331         clear();
00332         reserve(init.size());
00333         for (const auto& v : init)
00334         {
00335             push_back(v);
00336         }
00337     }
00338
00339     // get_allocator
00340
00341     /**
00342     * @brief Gražina naudojamą paskirstytoją.
00343     */
00344     allocator_type get_allocator() const noexcept
00345     {
00346         return alloc_;
00347     }
00348
00349     // -----
00350     // Elementų prieiga
00351     // -----
00352
00353     /**
00354     * @brief Prieiga su ribų tikrinimo (meta std::out_of_range jei pos >= size).
00355     */
00356     reference at(size_type pos)
00357     {
00358         if (pos >= size_)
00359         {
00360             throw std::out_of_range("Vector::at - indeksas uz ribu: " + std::to_string(pos));
00361         }
00362         return data_[pos];
00363     }
00364
00365     /**
00366     * @brief Const prieiga su ribų tikrinimo.
00367     */
00368     const_reference at(size_type pos) const
00369     {
00370         if (pos >= size_)
00371         {
00372             throw std::out_of_range("Vector::at - indeksas uz ribu: " + std::to_string(pos));
00373         }
00374         return data_[pos];
00375     }
00376
00377     /// @brief Indeksavimo operatorius (be ribų tikrinimo).
00378     reference operator[](size_type pos) { return data_[pos]; }
00379     /// @brief Const indeksavimo operatorius.
00380     const_reference operator[](size_type pos) const { return data_[pos]; }
00381
00382     /// @brief Pirmas elementas.
00383     reference front() { return data_[0]; }
00384     const_reference front() const { return data_[0]; }
00385

```

```

00386     /// @brief Paskutinis elementas.
00387     reference back() { return data_[size_ - 1]; }
00388     const_reference back() const { return data_[size_ - 1]; }
00389
00390     /// @brief Rodyklė į vidinį masyvą.
00391     T* data() noexcept { return data_; }
00392     const T* data() const noexcept { return data_; }
00393
00394     //Iteratoriai
00395
00396     iterator begin() noexcept { return data_; }
00397     const_iterator begin() const noexcept { return data_; }
00398     const_iterator cbegin() const noexcept { return data_; }
00399
00400     iterator end() noexcept { return data_ + size_; }
00401     const_iterator end() const noexcept { return data_ + size_; }
00402     const_iterator cend() const noexcept { return data_ + size_; }
00403
00404     reverse_iterator rbegin() noexcept { return reverse_iterator(end()); }
00405     const_reverse_iterator rbegin() const noexcept { return const_reverse_iterator(end()); }
00406     const_reverse_iterator crbegin() const noexcept { return const_reverse_iterator(cend()); }
00407
00408     reverse_iterator rend() noexcept { return reverse_iterator(begin()); }
00409     const_reverse_iterator rend() const noexcept { return const_reverse_iterator(begin()); }
00410     const_reverse_iterator crend() const noexcept { return const_reverse_iterator(cbegin()); }
00411
00412     //Talpa
00413
00414     /// @brief Ar vektorius tuščias?
00415     bool empty() const noexcept { return size_ == 0; }
00416     /// @brief Elementų skaičius.
00417     size_type size() const noexcept { return size_; }
00418     /// @brief Maksimalus galimas dydis.
00419     size_type max_size() const noexcept { return AllocTraits::max_size(alloc_); }
00420     /// @brief Dabartinė talpa.
00421     size_type capacity() const noexcept { return cap_; }
00422
00423     /**
00424      * @brief Rezervuoja atmintį bent newCap elementams (nekeičia size).
00425      */
00426     void reserve(size_type newCap)
00427     {
00428         if (newCap <= cap_)
00429         {
00430             return;
00431         }
00432         reallocate(newCap);
00433     }
00434
00435     /**
00436      * @brief Sumažina talpos perteklių (cap → size).
00437      */
00438     void shrink_to_fit()
00439     {
00440         if (size_ == cap_)
00441         {
00442             return;
00443         }
00444         if (size_ == 0)
00445         {
00446             deallocate();
00447             cap_ = 0;
00448             return;
00449         }
00450         reallocate(size_);
00451     }
00452
00453     //Modifikatoriai
00454
00455     /**
00456      * @brief Pašalina visus elementus (talpa nesikeičia).
00457      */
00458     void clear() noexcept
00459     {
00460         destroyAll();
00461         size_ = 0;
00462     }
00463
00464     // --- insert ---
00465
00466     /**
00467      * @brief Įterpia vieną kopiją prieš pos.
00468      * @return Iteratorius į įterptą elementą.
00469      */
00470     iterator insert(const_iterator pos, const T& value)
00471     {
00472         const size_type idx = static_cast<size_type>(pos - data_);

```

```

00473         ensureSpace(1);
00474         shiftRight(idx, 1);
00475         AllocTraits::construct(alloc_, data_ + idx, value);
00476         size_++;
00477         return data_ + idx;
00478     }
00479
00480     /**
00481     * @brief Įterpia vieną elementą (move) prieš pos.
00482     */
00483     iterator insert(const_iterator pos, T&& value)
00484     {
00485         const size_type idx = static_cast<size_type>(pos - data_);
00486         ensureSpace(1);
00487         shiftRight(idx, 1);
00488         AllocTraits::construct(alloc_, data_ + idx, std::move(value));
00489         ++size_;
00490         return data_ + idx;
00491     }
00492
00493     /**
00494     * @brief Įterpia count kopijų value prieš pos.
00495     */
00496     iterator insert(const_iterator pos, size_type count, const T& value)
00497     {
00498         const size_type idx = static_cast<size_type>(pos - data_);
00499         if (count == 0)
00500         {
00501             return data_ + idx;
00502         }
00503         ensureSpace(count);
00504         const size_type oldSize = size_;
00505         shiftRight(idx, count);
00506         for (size_type i = 0; i < count; ++i)
00507         {
00508             const size_type dst = idx + i;
00509             if (dst < oldSize)
00510             {
00511                 data_[dst] = value;
00512             }
00513             else
00514             {
00515                 AllocTraits::construct(alloc_, data_ + dst, value);
00516             }
00517         }
00518         size_ += count;
00519         return data_ + idx;
00520     }
00521
00522     /**
00523     * @brief Įterpia diapazoną [first, last) prieš pos.
00524     */
00525     template <typename InputIt, typename = std::enable_if_t<std::is_base_of_v< std::input_iterator_tag, typename std::iterator_traits<InputIt>::value_type>::is_trivially_copyable>>
00526     iterator insert(const_iterator pos, InputIt first, InputIt last)
00527     {
00528         const size_type idx = static_cast<size_type>(pos - data_);
00529         Vector tmp(first, last, alloc_);
00530         const size_type count = tmp.size_;
00531         if (count == 0)
00532         {
00533             return data_ + idx;
00534         }
00535         ensureSpace(count);
00536         const size_type oldSize = size_;
00537         shiftRight(idx, count);
00538         for (size_type i = 0; i < count; ++i)
00539         {
00540             const size_type dst = idx + i;
00541             if (dst < oldSize)
00542             {
00543                 data_[dst] = std::move(tmp.data_[i]);
00544             }
00545             else
00546             {
00547                 AllocTraits::construct(alloc_, data_ + dst, std::move(tmp.data_[i]));
00548             }
00549         }
00550         size_ += count;
00551         return data_ + idx;
00552     }
00553
00554     /**
00555     * @brief Įterpia inicializatorių sąrašą prieš pos.
00556     */
00557     iterator insert(const_iterator pos, std::initializer_list<T> init)
00558     {

```



```

00559         return insert(pos, init.begin(), init.end());
00560     }
00561
00562     // --- emplace ---
00563
00564     /**
00565      * @brief Sukuria elementą vietoje prieš pos.
00566      */
00567     template <typename... Args>
00568     iterator emplace(const_iterator pos, Args&&... args)
00569     {
00570         const size_type idx = static_cast<size_type>(pos - data_);
00571         ensureSpace(1);
00572         shiftRight(idx, 1);
00573         AllocTraits::construct(alloc_, data_ + idx, std::forward<Args>(args)...);
00574         size_++;
00575         return data_ + idx;
00576     }
00577
00578     // --- erase ---
00579
00580     /**
00581      * @brief Pašalina elementą pos pozicijoje.
00582      * @return Iteratorius po pašalinto elemento.
00583      */
00584     iterator erase(const_iterator pos)
00585     {
00586         const size_type idx = static_cast<size_type>(pos - data_);
00587         for (size_type i = idx; i + 1 < size_; ++i)
00588         {
00589             data_[i] = std::move(data_[i + 1]);
00590         }
00591         AllocTraits::destroy(alloc_, data_ + size_ - 1);
00592         size_--;
00593         return data_ + idx;
00594     }
00595
00596     /**
00597      * @brief Pašalina diapazoną [first, last).
00598      * @return Iteratorius į pirmą elementą po pašalintų.
00599      */
00600     iterator erase(const_iterator first, const_iterator last)
00601     {
00602         const size_type idxFirst = static_cast<size_type>(first - data_);
00603         const size_type idxLast = static_cast<size_type>(last - data_);
00604         const size_type count = idxLast - idxFirst;
00605         if (count == 0)
00606         {
00607             return data_ + idxFirst;
00608         }
00609         // Perkeliame elementus kairėn
00610         for (size_type i = idxFirst; i + count < size_; ++i)
00611         {
00612             data_[i] = std::move(data_[i + count]);
00613         }
00614         // Sunaikinami galo elementai
00615         for (size_type i = size_ - count; i < size_; ++i)
00616         {
00617             AllocTraits::destroy(alloc_, data_ + i);
00618         }
00619         size_ -= count;
00620         return data_ + idxFirst;
00621     }
00622
00623     // --- push_back / emplace_back / pop_back ---
00624
00625     /**
00626      * @brief Prideda kopiją į galą.
00627      */
00628     void push_back(const T& value)
00629     {
00630         if (size_ == cap_)
00631         {
00632             reallocate(growCap());
00633         }
00634         AllocTraits::construct(alloc_, data_ + size_, value);
00635         size_++;
00636     }
00637
00638     /**
00639      * @brief Prideda (move) elementą į galą.
00640      */
00641     void push_back(T&& value)
00642     {
00643         if (size_ == cap_)
00644         {
00645             reallocate(growCap());

```

```

00646     }
00647     AllocTraits::construct(alloc_, data_ + size_, std::move(value));
00648     size_++;
00649 }
00650
00651 /**
00652  * @brief Sukuria elementą vietoje gale.
00653  * @return Nuoroda į sukurtą elementą.
00654  */
00655 template <typename... Args>
00656 reference.emplace_back(Args&&... args)
00657 {
00658     if (size_ == cap_)
00659     {
00660         reallocate(growCap());
00661     }
00662     AllocTraits::construct(alloc_, data_ + size_, std::forward<Args>(args)...);
00663     size_++;
00664     return data_[size_ - 1];
00665 }
00666
00667 /**
00668  * @brief Pašalina paskutinį elementą.
00669  */
00670 void pop_back()
00671 {
00672     assert(size_ > 0 && "pop_back() iškviesta ant tuščio vektoriaus");
00673     AllocTraits::destroy(alloc_, data_ + size_ - 1);
00674     size_--;
00675 }
00676
00677 // --- resize ---
00678
00679 /**
00680  * @brief Pakeičia dydį į count (nauji elementai - value-initialized).
00681  */
00682 void resize(size_type count)
00683 {
00684     if (count < size_)
00685     {
00686         for (size_type i = count; i < size_; ++i)
00687         {
00688             AllocTraits::destroy(alloc_, data_ + i);
00689         }
00690         size_ = count;
00691     }
00692     else if (count > size_)
00693     {
00694         reserve(count);
00695         for (size_type i = size_; i < count; ++i)
00696         {
00697             AllocTraits::construct(alloc_, data_ + i);
00698             ++size_;
00699         }
00700     }
00701 }
00702
00703 /**
00704  * @brief Pakeičia dydį į count, naujus elementus užpildo value.
00705  */
00706 void resize(size_type count, const T& value)
00707 {
00708     if (count < size_)
00709     {
00710         for (size_type i = count; i < size_; ++i)
00711         {
00712             AllocTraits::destroy(alloc_, data_ + i);
00713         }
00714         size_ = count;
00715     }
00716     else if (count > size_)
00717     {
00718         reserve(count);
00719         for (size_type i = size_; i < count; ++i)
00720         {
00721             AllocTraits::construct(alloc_, data_ + i, value);
00722             size_++;
00723         }
00724     }
00725 }
00726
00727 // --- swap ---
00728
00729 /**
00730  * @brief Sukeičia du vektorius vietomis (O(1)).
00731  */
00732 void swap(Vector& other) noexcept

```

```

00733     {
00734         using std::swap;
00735         swap(alloc_, other.alloc_);
00736         swap(data_, other.data_);
00737         swap(size_, other.size_);
00738         swap(cap_, other.cap_);
00739     }
00740
00741 private:
00742     // Vidiniai pagalbiniai metodai insert logikai
00743
00744     //Garantuoja vietos count naujiems elementams (perskirsto jei reikia)
00745     void ensureSpace(size_type count)
00746     {
00747         if (size_ + count > cap_)
00748         {
00749             reallocate(std::max(size_ + count, growCap()));
00750         }
00751     }
00752
00753     /**
00754      * @brief Pastumia elementus iš [idx, size_) dešinėn per count pozicijų.
00755      *
00756      * Elementai į nepatalpintą (uninitialized) sritį - construct,
00757      * į jau egzistuojančią - move-assign.
00758      * Rezultate pozicijos [idx, idx+count) - „inicializuotos, bet perkeltos“
00759      * arba neinicializuotos - tvarkomos insert() metodo.
00760      *
00761      * @pre size_ + count <= cap_ (ensureSpace jau iškviesta)
00762      */
00763     void shiftRight(size_type idx, size_type count)
00764     {
00765         // Perkeliame dešinėn (iš dešinės į kairę, kad neverstume)
00766         size_type i = size_ + count;
00767         while (i-->idx + count)
00768         {
00769             const size_type src = i - count;
00770             if (i >= size_)
00771             {
00772                 AllocTraits::construct(alloc_, data_ + i, std::move(data_[src]));
00773             }
00774             else
00775             {
00776                 data_[i] = std::move(data_[src]);
00777             }
00778         }
00779     }
00780 };
00781
00782 //Non-member palyginimo operatoriai
00783
00784 /// @relates Vector
00785 template <typename T, typename A>
00786 bool operator==(const Vector<T, A>& l, const Vector<T, A>& r)
00787 {
00788     if (l.size() != r.size())
00789     {
00790         return false;
00791     }
00792     for (typename Vector<T, A>::size_type i = 0; i < l.size(); ++i)
00793     {
00794         if (!l[i] == r[i])
00795         {
00796             return false;
00797         }
00798     }
00799     return true;
00800 }
00801
00802 /// @relates Vector
00803 template <typename T, typename A>
00804 bool operator!=(const Vector<T, A>& l, const Vector<T, A>& r) { return !(l == r); }
00805
00806 /// @relates Vector
00807 template <typename T, typename A>
00808 bool operator<(const Vector<T, A>& l, const Vector<T, A>& r) { return
    std::lexicographical_compare(l.begin(), l.end(), r.begin(), r.end()); }
00809
00810 /// @relates Vector
00811 template <typename T, typename A>
00812 bool operator<=(const Vector<T, A>& l, const Vector<T, A>& r) { return !(r < l); }
00813
00814 /// @relates Vector
00815 template <typename T, typename A>
00816 bool operator>(const Vector<T, A>& l, const Vector<T, A>& r) { return r < l; }
00817
00818 /// @relates Vector

```

```

00819 template <typename T, typename A>
00820 bool operator>=(const Vector<T, A>& l, const Vector<T, A>& r) { return !(l < r); }
00821
00822 //Non-member swap
00823
00824 /**
00825  * @brief ADL-friendly swap.
00826  * @relates Vector
00827  */
00828 template <typename T, typename A>
00829 void swap(Vector<T, A>& l, Vector<T, A>& r) noexcept { l.swap(r); }
00830
00831 //Non-member erase / erase_if
00832
00833 /**
00834  * @brief Pašalina visus elementus, lygius value.
00835  * @return Pašalintų elementų skaičius.
00836  * @relates Vector
00837  */
00838 template <typename T, typename A, typename U>
00839 typename Vector<T, A>::size_type erase(Vector<T, A>& c, const U& value)
00840 {
00841     auto it = std::remove(c.begin(), c.end(), value);
00842     const auto n = static_cast<typename Vector<T, A>::size_type>(std::distance(it, c.end()));
00843     c.erase(it, c.end());
00844     return n;
00845 }
00846
00847 /**
00848  * @brief Pašalina visus elementus, tenkinančius pred.
00849  * @return Pašalintų elementų skaičius.
00850  * @relates Vector
00851  */
00852 template <typename T, typename A, typename Pred>
00853 typename Vector<T, A>::size_type erase_if(Vector<T, A>& c, Pred pred)
00854 {
00855     auto it = std::remove_if(c.begin(), c.end(), pred);
00856     const auto n = static_cast<typename Vector<T, A>::size_type>(std::distance(it, c.end()));
00857     c.erase(it, c.end());
00858     return n;
00859 }

```

5.37 Zmogus.cpp Failo Nuoroda

```
#include "Zmogus.h"
```

5.38 Zmogus.cpp

[Eiti į šio failo dokumentaciją.](#)

```

00001 #include "Zmogus.h"
00002
00003 Zmogus::Zmogus(const std::string& v, const std::string& p): vardas(v), pavarde(p)
00004 {
00005 }

```

5.39 Zmogus.h Failo Nuoroda

Abstrakti bazinė klasė žmogui.

```
#include <iostream>
#include <string>
```

Klasės

- class [Zmogus](#)

Abstrakti bazinė klasė, apibrėžianti žmogaus sąsają.

5.39.1 Smulkus aprašymas

Abstrakti bazinė klasė žmogui.

Apibrėžia bendrą sąsają visoms žmogaus tipo klasėms. Negali būti tiesiogiai instantijuojama, nes turi grynąsias virtualias funkcijas.

Autorius

[Studentas](#)

Versija

3.0

Apibrėžimas faile [Zmogus.h](#).

5.40 Zmogus.h

Eiti į šio failo dokumentaciją.

```
00001 /**
00002  * @file Zmogus.h
00003  * @brief Abstrakti bazinė klasė žmogui.
00004  *
00005  * Apibrėžia bendrą sąsają visoms žmogaus tipo klasėms.
00006  * Negali būti tiesiogiai instantijuojama, nes turi grynąsias
00007  * virtualias funkcijas.
00008  *
00009  * @author Studentas
00010  * @version 3.0
00011  */
00012
00013 #pragma once
00014
00015 #include <iostream>
00016 #include <string>
00017
00018 /**
00019  * @class Zmogus
00020  * @brief Abstrakti bazinė klasė, apibrėžianti žmogaus sąsają.
00021  *
00022  * Saugo vardą ir pavardę, bei deklaruoja grynąsias virtualias
00023  * funkcijas, kurias privalo realizuoti išvestinės klasės.
00024  * Realizuoja Rule of Five naudojant '= default' direktyvą.
00025  */
00026
00027 class Zmogus
00028 {
00029 protected:
00030     std::string vardas;
00031     std::string pavarde;
00032
00033 public:
00034     /** @name Rule of Five
00035      * @{
00036      */
00037
00038     /** @brief Numatytasis konstruktorius. Inicializuoja tuščius laukus. */
00039     Zmogus() = default;
00040
00041     /**
```

```

00042     * @brief Konstruktorius su parametrais.
00043     * @param v Vardas.
00044     * @param p Pavardė.
00045     */
00046     Zmogus(const std::string& v, const std::string& p);
00047
00048     /** @brief Kopijavimo konstruktorius. */
00049     Zmogus(const Zmogus&) = default;
00050
00051     /** @brief Kopijavimo priskyrimo operatorius. */
00052     Zmogus& operator=(const Zmogus&) = default;
00053
00054     /** @brief Perkėlimo konstruktorius. */
00055     Zmogus(Zmogus&&) = default;
00056
00057     /** @brief Perkėlimo priskyrimo operatorius. */
00058     Zmogus& operator=(Zmogus&&) = default;
00059
00060     /**
00061     * @brief Virtualus destruktorius.
00062     *
00063     * Užtikrina teisingą išvestinių klasių sunaikinimą per bazinės
00064     * klasės rodyklę.
00065     */
00066     virtual ~Zmogus() = default;
00067
00068
00069     virtual std::string getVardas() const { return vardas; }
00070     virtual std::string getPavarde() const { return pavarde; }
00071
00072     void setVardas (const std::string& v) { vardas = v; }
00073     void setPavarde (const std::string& p) { pavarde = p; }
00074
00075     // -----
00076     /** @name Gynosios virtualios funkcijos
00077     * Išvestinės klasės privalo šias funkcijas realizuoti.
00078     * @{
00079     */
00080
00081     /**
00082     * @brief Apskaičiuoja galutinį įvertinimą.
00083     *
00084     * Konkretus skaičiavimo algoritmas priklauso nuo išvestinės klasės.
00085     */
00086     virtual void suskaiciuotiGalutini() = 0;
00087
00088     /**
00089     * @brief Išspausdina objekto duomenis į srautą.
00090     * @param os Išvesties srautas.
00091     */
00092     virtual void spausdinti(std::ostream& os) const = 0;
00093     /** @} */
00094 };

```